THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms.K.NETHRAVATHI (20524UCSC058), a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of the **academic Advanced Database Management System – Practical (UCSR513)** during year 2026–2027.

The practical examination will be held on **16.10.2026**

Course Inchage: Dr. S Manimekalai

Head of the Department: Ms.K.Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO: 01 Creation of DDL Operations using Products Table

DATE:05.08.2026

Aim

To create a Products table, alter the table structure, insert product records, display all records, and retrieve products based on their price.

Software Used

Oracle / MySQL / SQLite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects such as tables.

DDL COMMANDS:

- CREATE
- ALTER
- DROP
- TRUNCATE
- RENAME

CREATE

CREATE command is used to create a new table and define its columns and constraints.

Syntax

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

SQL> CREATE TABLE products (

id INTEGER PRIMARY KEY,

product_name TEXT,

price INTEGER

);

Table Created

ALTER

ALTER command is used to modify the structure of an existing table.

It has methods such as ADD, MODIFY, and DROP.

ADD

ADD command is used to add a new column to the table.

Syntax

ALTER TABLE table_name ADD column_name datatype;

Query

SQL> ALTER TABLE products ADD category VARCHAR(100);

Table Altered

MODIFY

MODIFY command is used to change the data type or size of an existing column.

Syntax

ALTER TABLE table_name MODIFY column_name datatype;

Query – Oracle

SQL> ALTER TABLE products MODIFY product_name VARCHAR(150);

Query – MySQL

SQL> ALTER TABLE products MODIFY product_name VARCHAR(150);

Table Altered

DROP COLUMN

DROP COLUMN is used to remove a column from a table.

Syntax

ALTER TABLE table_name DROP COLUMN column_name;

Query

SQL> ALTER TABLE products DROP COLUMN category;

Column Dropped

RENAME

RENAME command is used to change the name of a table.

Query

SQL> RENAME TABLE products TO items;

Table Renamed

OR

ALTER TABLE products RENAME TO items;

Column Rename

ALTER TABLE products RENAME COLUMN product_name TO item_name;

Column Renamed

TRUNCATE

TRUNCATE command is used to delete all records from a table while keeping the table structure.

Syntax

TRUNCATE TABLE table_name;

Query

SQL> TRUNCATE TABLE products;

Table Truncated

DROP

DROP command is used to permanently delete the table and all its data.

Syntax

DROP TABLE table_name;

Query

SQL> DROP TABLE products;

Table Dropped

Explanation

- Create a table named products.
- Add a new column using ALTER ADD.
- Modify the column structure.
- Drop an unwanted column.
- Insert product details into the table.
- Display all product records.
- Retrieve products whose price is 500 or above.
- Sort the products by price.

Create Table

```
CREATE TABLE products (
   id INTEGER PRIMARY KEY,
   product_name TEXT,
   price INTEGER
);
```

Table Created

Sample Data

INSERT INTO products VALUES (1, 'Laptop', 55000);

INSERT INTO products VALUES (2, 'Smartphone', 25000);

INSERT INTO products VALUES (3, 'Headphones', 1500);

INSERT INTO products VALUES (4, 'Smart Watch', 4500);

INSERT INTO products VALUES (5, 'Keyboard', 800);

INSERT INTO products VALUES (6, 'Tablet', 18000);

Records Inserted

SQL Query

Query 1: Display all product records

SQL> SELECT * FROM products;

Resultant Query

id	product_name	price
1	Laptop	55000
2	Smartphone	25000
3	Headphones	1500
4	Smart Watch	4500
5	Keyboard	800
6	Tablet	18000

Query 2: Display products with price 5000 or above

SQL> SELECT *

FROM products

WHERE price >= 5000

ORDER BY price;

Resultant Query

id	product_name	price
6	Tablet	18000
2	Smartphone	25000
1	Laptop	55000

RESULT

Thus, the Products table was created successfully, the table structure was modified using DDL commands, product records were inserted, all product details were displayed, and the products with a price of 5000 or above were retrieved successfully.

EX. NO: 02 Creation of DML Operations using Hospital Patients Table

DATE: 07.08.2026

Aim

To create a **Hospital Patients** table, insert patient records, display all records, update patient details, delete a patient record, and retrieve patients based on their age.

Software Used

Oracle / MySQL / SQLite3

Description

DML (**Data Manipulation Language**) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query

SQL> INSERT INTO patients VALUES (1, 'Arun', 35);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

Query

SQL> UPDATE patients

SET age = 36

WHERE patient_id = 1;

1 row updated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM patients

WHERE patient_id = 5;

1 row deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM patients;

Explanation

- Create a table named **patients**.
- Insert patient details into the table.
- Display all patient records.
- Update the age of a patient.
- Delete a patient record.
- Retrieve patients whose age is **30 or above** and sort them by age.

Create Table

```
CREATE TABLE patients (
    patient_id INTEGER PRIMARY KEY,
    patient_name TEXT,
    age INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO patients VALUES (1, 'Arun', 35);
INSERT INTO patients VALUES (2, 'Meena', 28);
INSERT INTO patients VALUES (3, 'Kavin', 42);
INSERT INTO patients VALUES (4, 'Divya', 25);
INSERT INTO patients VALUES (5, 'Rahul', 50);
INSERT INTO patients VALUES (6, 'Priya', 32);
```

Records Inserted

SQL Query

Query 1: Display all patient records

SQL> SELECT * FROM patients;

patient_id	patient_name	age
1	Arun	35
2	Meena	28
3	Kavin	42
4	Divya	25
5	Rahul	50
6	Priya	32

Query 2: Update the age of Arun

SQL> UPDATE patients

SET age = 36

WHERE patient_id = 1;

1 row updated.

Updated Record

patient_id	patient_name	age
1	Arun	35

Query 3: Delete Rahul's record

SQL> DELETE FROM patients

WHERE patient_id = 5;

1 row deleted**.**

Remaining Records

patient_id	patient_name	age
1	Arun	35
2	Meena	28

3	Kavin	42
4	Divya	25
6	Priya	32

Query 4: Display patients whose age is 30 or above

SQL> SELECT *

FROM patients

WHERE age >= 30

ORDER BY age;

Resultant Query

patient_id	patient_name	age
6	Priya	32
1	Arun	35
3	Kavin	42

RESULT

Thus, the Hospital Patients table was created successfully, records were inserted, patient details were displayed, the patient age was updated, a patient record was deleted, and the patients whose age was 30 or above were retrieved successfully.

EX. NO: 03 Creation of DML Operations using Doctors Table

DATE: 08.08.2026

Aim

To create a Doctors table, insert doctor records, display all records, update doctor details, delete a doctor record, and retrieve doctors based on their salary.

Software Used

Oracle / MySQL / SQLite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query

SQL> INSERT INTO doctors VALUES (1, 'Anitha', 'Cardiology', 85000);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

Query

SQL> UPDATE doctors

SET salary = 90000

WHERE doctor_id = 1;

1 row updated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM doctors

WHERE doctor_id = 5;

1 row deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM doctors;

Explanation

- Create a table named **doctors**.
- Insert doctor details into the table.
- Display all doctor records.
- Update the salary of a doctor.
- Delete a doctor record.
- Retrieve doctors whose salary is **80000 or above** and sort them by salary.

Create Table

```
CREATE TABLE doctors (
   doctor_id INTEGER PRIMARY KEY,
   doctor_name TEXT,
   specialization TEXT,
   salary INTEGER
);
```

Table Created

Sample Data

INSERT INTO doctors VALUES (1, 'Anitha', 'Cardiology', 85000);

INSERT INTO doctors VALUES (2, 'Kumar', 'Neurology', 75000);

INSERT INTO doctors VALUES (3, 'Priya', 'Pediatrics', 92000);

INSERT INTO doctors VALUES (4, 'Raj', 'Orthopedics', 68000);

INSERT INTO doctors VALUES (5, 'Meena', 'Dermatology', 80000);

INSERT INTO doctors VALUES (6, 'Divya', 'Gynecology', 95000);

Records Inserted

SQL Query

Query 1: Display all doctor records

SQL> SELECT * FROM doctors;

Resultant Query

doctor_id	doctor_name	specialization	salary
1	Anitha	Cardiology	85000
2	Kumar	Neurology	75000
3	Priya	Pediatrics	92000
4	Raj	Orthopedics	68000
5	Meena	Dermatology	80000
6	Divya	Gynecology	95000

Query 2: Update the salary of Anitha

SQL> UPDATE doctors

SET salary = 90000

WHERE doctor_id = 1;

1 row updated.

Updated Record

doctor_id	doctor_name	specialization	salary
1	Anitha	Cardiology	85000

Query 3: Delete Meena's record

SQL> DELETE FROM doctors

WHERE doctor_id = 5;

1 row deleted.

Remaining Records

doctor_id	doctor_name	specialization	salary

1	Anitha	Cardiology	85000
2	Kumar	Neurology	75000
3	Priya	Pediatrics	92000
4	Raj	Orthopedics	68000
6	Divya	Gynecology	95000

Query 4: Display doctors whose salary is 80000 or above

SQL> SELECT *

FROM doctors

WHERE salary >= 80000

ORDER BY salary;

Resultant Query

doctor_id	doctor_name	specialization	salary
1	Anitha	Cardiology	85000
3	Priya	Pediatrics	92000
6	Divya	Gynecology	95000

RESULT

Thus, the Doctors table was created successfully, records were inserted, doctor details were displayed, the doctor salary was updated, a doctor record was deleted, and the doctors who received a salary of 80000 or above were successfully retrieved.

EX. NO: 04 Creation of Transaction Operations using Library Books Table

DATE: 10.08.2026

Aim

To create a Library Books table and perform transaction operations such as COMMIT, ROLLBACK, and SAVEPOINT on the table.

Software Used

Oracle / MySQL / SQLite3

Description

TCL (Transaction Control Language) is used to manage transactions in a database. It helps to save changes permanently or undo changes made during a transaction.

TRANSACTION COMMANDS:

- **COMMIT**
- **ROLLBACK**
- **SAVEPOINT**

COMMIT

COMMIT command is used to permanently save the changes made in a transaction.

Syntax

COMMIT;

Query

SQL> INSERT INTO library_books

VALUES (6, 'Machine Learning', 'Meena', 800);

SQL> COMMIT;

Transaction committed.

ROLLBACK

ROLLBACK command is used to undo changes made after the last COMMIT**.**

Syntax

ROLLBACK;

Query

SQL> UPDATE library_books

SET price = 900

WHERE book_id = 6;

SQL> ROLLBACK;

Transaction rolled back.

SAVEPOINT

SAVEPOINT command is used to create a temporary point in a transaction. We can roll back to that particular point.

Syntax

SAVEPOINT savepoint_name;

Query

SQL> SAVEPOINT book_save;

Savepoint created.

Explanation

- Create a table named library_books.
- Insert book details into the table.
- Display all book records.
- Use COMMIT to permanently save the inserted record.
- Use ROLLBACK to undo an update.

- Use SAVEPOINT to create a point within a transaction.
- Roll back the transaction to the created savepoint**.**

Create Table

```
CREATE TABLE library_books (
    book_id INTEGER PRIMARY KEY,
    book_name TEXT,
    author TEXT,
    price INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO library_books VALUES
(1, 'Database Systems', 'Kumar', 500);
INSERT INTO library_books VALUES
(2, 'Python Programming', 'Anitha', 450);
INSERT INTO library_books VALUES
(3, 'Java Basics', 'Raj', 550);
INSERT INTO library_books VALUES
(4, 'Web Development', 'Priya', 600);
INSERT INTO library_books VALUES
(5, 'Data Science', 'Arun', 700);
```

Records Inserted

SQL Query

Query 1: Display all book records

SQL> SELECT * FROM library_books;

Resultant Query

book_id	book_name	author	price
1	Database Systems	Kumar	500
2	Python Programming	Anitha	450
3	Java Basics	Raj	550
4	Web Development	Priya	600
5	Data Science	Arun	700

Query 2: Insert a new book and COMMIT

SQL> INSERT INTO library_books

VALUES (6, 'Machine Learning', 'Meena', 800);

SQL> COMMIT;

Transaction committed.

Updated Record

book_id	book_name	author	price
6	Machine learning	meena	800

Query 3: Update price and ROLLBACK

SQL> UPDATE library_books

SET price = 900

WHERE book_id = 6;

SQL> ROLLBACK;

Transaction rolled back.

Record After Rollback

book_id	book_name	author	price
6	Machine learning	meena	800

The price returns to 800 because the update was rolled back**.**

Query 4: Use SAVEPOINT

SQL> INSERT INTO library_books

VALUES (7, 'Cloud Computing', 'Divya', 650);

SQL> SAVEPOINT book_save;

SQL> INSERT INTO library_books

VALUES (8, 'Artificial Intelligence', 'Kavin', 750);

SQL> ROLLBACK TO book_save;

Transaction rolled back to savepoint.

Resultant Query

book_id	book_name	author	price
6	Machine learning	meena	800
7	Cloud computing	Divya	

The Cloud Computing record remains because it was inserted before the savepoint. The Artificial Intelligence record is removed because it was inserted after the savepoint.

RESULT

Thus, the Library Books table was created successfully, and the transaction operations COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT were successfully performed**.**

EX. NO: 05 Creation of Join Operations using Students and Courses Tables

DATE: 16.08.2026

Aim

To create Students and Courses tables and perform different types of join operations to retrieve related information from both tables.

Software Used

Oracle / MySQL / SQLite3

Description

A JOIN is used to combine records from two or more tables based on a related column between them.

TYPES OF JOIN

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN

Table 1: STUDENTS

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT,
    course_id INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO students VALUES (1, 'Kavin', 101);
INSERT INTO students VALUES (2, 'Meena', 102);
```

INSERT INTO students VALUES (3, 'Arun', 103);

INSERT INTO students VALUES (4, 'Divya', 101);

INSERT INTO students VALUES (5, 'Rahul', 104);

Records Inserted

student_id	student_name	course_id
1	Kavin	101
2	Meena	102
3	Arun	103
4	Divya	101
5	Rahul	104

Table 2: COURSES

```
CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY,
    course_name TEXT,
    duration INTEGER
);
```

Table Created

Sample Data

INSERT INTO courses VALUES (101, 'Computer Science', 3);

INSERT INTO courses VALUES (102, 'Data Science', 2);

INSERT INTO courses VALUES (103, 'Artificial Intelligence', 3);

INSERT INTO courses VALUES (105, 'Cyber Security', 2);

Records Inserted

Course table

course_id	course_name	duration
101	Computer Science	3
102	Data Science	2
103	Artificial Intelligence	3
105	Cyber Security	2

Query 1: INNER JOIN

INNER JOIN returns only the records that have matching values in both tables.

Query

```
SELECT students.student_id,
    students.student_name,
    courses.course_name,
    courses.duration
FROM students
INNER JOIN courses
ON students.course_id = courses.course_id;
```

Resultant Query

student_id	student_name	course_name	duration
1	Kavin	Computer Science	3
2	Meena	Data Science	2
3	Arun	Artificial Intelligence	3

4	Divya	Computer Science	3

Query 2: LEFT JOIN

LEFT JOIN returns all records from the Students table and matching records from the Courses table.

Query

```
SELECT students.student_id,
    students.student_name,
    courses.course_name
FROM students
LEFT JOIN courses
ON students.course_id = courses.course_id;
```

Resultant Query

student_id	student_name	course_name
1	Kavin	Computer Science
2	Meena	Data Science
3	Arun	Artificial Intelligence
4	Divya	Computer Science
5	Rahul	NULL

Query 3: RIGHT JOIN

RIGHT JOIN returns all records from the Courses table and matching records from the Students table.

Query

```
SELECT students.student_name,
    courses.course_id,
    courses.course_name
FROM students
RIGHT JOIN courses
ON students.course_id = courses.course_id;
```

Resultant Query

student_name	course_id	course_name
Kavin	101	Computer Science
Divya	101	Computer Science
Meena	102	Data Science
Arun	103	Artificial Intelligence
NULL	105	Cyber Security

Query 4: FULL OUTER JOIN

FULL OUTER JOIN returns all matching and non-matching records from both tables.

Query

```
SELECT students.student_name,
    courses.course_name
FROM students
FULL OUTER JOIN courses
ON students.course_id = courses.course_id;
```

Resultant Query

student_name	**course_name**

Kavin	Computer Science
Divya	Computer Science
Meena	Data Science
Arun	Artificial Intelligence
Rahul	NULL
NULL	Cyber Security



RESULT

Thus, the Students and Courses tables were created successfully, and different JOIN operations such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN were successfully performed to retrieve related information from both tables.

EX. NO: 06 Creation of SQL Functions using Books Table

DATE: 18.08.2026

Aim

To create a Books table, insert book records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / SQLite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL Function Types

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions perform calculations on a group of rows and return a single result.

Common Aggregate Functions

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Syntax

SELECT aggregate_function(column_name)

FROM table_name;

STRING FUNCTIONS

String functions are used to perform operations on text or character values.

Common String Functions

- UPPER()
- LOWER()
- LENGTH()

Syntax

SELECT string_function(column_name)

FROM table_name;

DATE FUNCTIONS

Date functions are used to perform operations on date values.

Common Date Functions

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

SELECT date_function(date_column)

FROM table_name;

Explanation

- Create a table named books.
- Insert book details into the table.
- Display all book records.
- Calculate the total number of books using COUNT().
- Calculate the total and average price using SUM() and AVG().
- Find the highest and lowest book price using MAX() and MIN().
- Display book titles in uppercase and lowercase using UPPER() and LOWER().
- Find the length of book titles using LENGTH().
- Display the current date.
- Retrieve books published after a particular date.

Create Table

```
CREATE TABLE books (
   book_id INTEGER PRIMARY KEY,
   book_title TEXT,
   author TEXT,
   price INTEGER,
   published_date DATE
);
```

Table Created

Sample Data

```
INSERT INTO books VALUES
(1, 'Python Basics', 'Ravi', 450, '2026-01-10');

INSERT INTO books VALUES
(2, 'Database Systems', 'Priya', 600, '2026-02-15');

INSERT INTO books VALUES
(3, 'Web Development', 'Arun', 550, '2026-03-05');

INSERT INTO books VALUES
(4, 'Artificial Intelligence', 'Divya', 750, '2026-04-12');

INSERT INTO books VALUES
(5, 'Computer Networks', 'Karthik', 500, '2026-05-20');

INSERT INTO books VALUES
(6, 'Data Science', 'Meena', 650, '2026-06-18');
```

Records Inserted

SQL QUERIES

Query 1: Display all book records

```
SELECT * FROM books;
```

Resultant Query

book_id	book_title	author	price	published_date
1	Python Basics	Ravi	450	2026-01-10
2	Database Systems	Priya	600	2026-02-15
3	Web Development	Arun	550	2026-03-05
4	Artificial Intelligence	Divya	750	2026-04-12

5	Computer Networks	Karthik	500	2026-05-20
6	Data Science	Meena	650	2026-06-18

AGGREGATE FUNCTIONS

Query 2: Count the total number of books

SELECT COUNT(*) AS Total_Books

FROM books;

Result:

Total_Books
6

Query 3: Calculate the total price of all books

SELECT SUM(price) AS Total_Price

FROM books;

Result:

Total_Price
3500

Query 4: Calculate the average price of books

SELECT AVG(price) AS Average_Price

FROM books;

Result:

Average_Price
583.33

Query 5: Find the highest and lowest book price

```
SELECT
    MAX(price) AS Highest_Price,
    MIN(price) AS Lowest_Price
FROM books;
```

Result:

Highest_Price	Lowest_Price
750	450

STRING FUNCTIONS

Query 6: Display book titles in uppercase

```
SELECT book_title, UPPER(book_title) AS Upper_Title
FROM books;
```

Result:

book_title	Upper_Title
Python Basics	PYTHON BASICS
Database Systems	DATABASE SYSTEMS
Web Development	WEB DEVELOPMENT
Artificial Intelligence	ARTIFICIAL INTELLIGENCE
Computer Networks	COMPUTER NETWORKS

Query 7: Display book titles in lowercase

```
SELECT book_title, LOWER(book_title) AS Lower_Title
```

FROM books;

Result:

book_title	Lower_Title
Python Basics	python basics
Database Systems	database systems
Web Development	web development
Artificial Intelligence	artificial intelligence
Computer Networks	computer networks
Data Science	data science

Query 8: Display the length of each book title

SELECT book_title, LENGTH(book_title) AS Title_Length

FROM books;

Result:

book_title	Title_Length
Python Basics	13
Database Systems	16
Web Development	15
Artificial Intelligence	23
Computer Networks	18
Data Science	12

DATE FUNCTIONS

Query 9: Display the current date

SELECT CURRENT_DATE AS Current_Date;

Result:

Current_Date

Current system date

Query 10: Display books published after March 2026

SELECT book_title, published_date

FROM books

WHERE published_date > '2026-03-31';

Result:

book_title	published_date
Artificial Intelligence	2026-04-12
Computer Networks	2026-05-20
Data Science	2026-06-18

RESULT

Thus, the Books table was created successfully, book records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO: 07 Creation of Subqueries and Nested Queries using Students

DATE: 21.08.2026

Aim

To create a Students table, insert student records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / SQLite3

Description

A Subquery is a query written inside another SQL query. It is used to obtain an intermediate result and use that result in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES

- Single Row Subquery
- Multiple Row Subquery
- Nested Subquery

SINGLE ROW SUBQUERY

A Single Row Subquery returns one value or one row. It is commonly used with operators such as =, >, <, >=, and <=.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name > (
    SELECT column_name
    FROM table_name
```

WHERE condition

);

MULTIPLE ROW SUBQUERY

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL.

Syntax

SELECT column_name

FROM table_name

WHERE column_name IN (

SELECT column_name

FROM table_name

WHERE condition

);

NESTED SUBQUERY

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.

Syntax

SELECT column_name

FROM table_name

WHERE column_name = (

SELECT column_name

FROM table_name

WHERE condition

);

Explanation

- Create a table named students.
- Insert student details into the table.
- Display all student records.
- Find students whose marks are greater than the average marks.
- Find students who belong to the CS or BCA department.
- Find the student who has the highest mark.
- Find students whose marks are greater than a particular student's marks.

Create Table

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT,
    department TEXT,
    marks INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO students VALUES (1, 'Kavin', 'CS', 85);

INSERT INTO students VALUES (2, 'Meena', 'BCA', 72);

INSERT INTO students VALUES (3, 'Arun', 'BCom', 68);

INSERT INTO students VALUES (4, 'Divya', 'CS', 92);

INSERT INTO students VALUES (5, 'Rahul', 'BBA', 60);
```

INSERT INTO students VALUES (6, 'Priya', 'BCA', 80);

Records Inserted

SQL Query

Query 1: Display all student records

SELECT * FROM students;

Resultant Query

student_id	student_name	department	marks
1	Kavin	CS	85
2	Meena	BCA	72
3	Arun	BCom	68
4	Divya	CS	92
5	Rahul	BBA	60
6	Priya	BCA	80

Query 2: Display students whose marks are greater than the average marks

```
SELECT student_name, marks
FROM students
WHERE marks > (
   SELECT AVG(marks)
   FROM students
);
```

Average marks = 76.17

Resultant Query

student_name	marks
Kavin	85
Divya	92
Priya	80

Query 3: Display students who belong to CS or BCA department

```
SELECT student_name, department
FROM students
WHERE department IN (
    SELECT department
    FROM students
    WHERE department IN ('CS', 'BCA')
);
```

Resultant Query

student_name	department
Kavin	CS
Meena	BCA
Divya	CS
Priya	BCA

Query 4: Display the student with the highest marks

```
SELECT student_name, marks
FROM students
WHERE marks = (
    SELECT MAX(marks)
```

FROM students

);

Resultant Query

student_name	marks
Divya	92

Query 5: Display students whose marks are greater than Meena's marks

SELECT student_name, marks

FROM students

WHERE marks > (

SELECT marks

FROM students

WHERE student_name = 'Meena'

);

Meena's marks = 72

Resultant Query

student_name	marks
Kavin	85
Divya	92
Priya	80

Additional Nested Query

Query 6: Display the student who scored the same marks as the highest-scoring CS student

SELECT student_name, marks

```
FROM students
WHERE marks = (
    SELECT MAX(marks)
    FROM students
    WHERE department = 'CS'
);
```

Resultant Query

student_name	marks
Divya	92

RESULT

Thus, the Students table was created successfully, student records were inserted, and the required student details were retrieved successfully using Subqueries and Nested Queries.

EX. NO: 08 Creation of Views, Indexes and Sequences using Product Table

DATE: 23.08.2026

Aim

To create and perform operations on **Views, Indexes, and Sequences** using a products table and SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View

A View is a virtual table created from the result of a SELECT query. It does not store the actual data separately.

Index

An Index is used to improve the speed of searching and retrieving data from a table.

Sequence

A Sequence is a database object used to automatically generate unique numeric values.

1. VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

CREATE VIEW view_name AS

SELECT column1, column2

FROM table_name

WHERE condition;

Create Table

```
CREATE TABLE products (
    product_id NUMBER PRIMARY KEY,
    product_name VARCHAR2(50),
    category VARCHAR2(30),
    price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO products VALUES (201, 'Laptop', 'Electronics', 55000);
INSERT INTO products VALUES (202, 'Mobile', 'Electronics', 25000);
INSERT INTO products VALUES (203, 'Keyboard', 'Accessories', 2000);
INSERT INTO products VALUES (204, 'Monitor', 'Electronics', 15000);
INSERT INTO products VALUES (205, 'Mouse', 'Accessories', 1000);
COMMIT;
```

Records Inserted

Create View

Create a view to display only products belonging to the **Electronics** category.

```
CREATE VIEW electronic_products AS
SELECT product_id, product_name, price
FROM products
WHERE category = 'Electronics';
```

View Created

Display View

```
SELECT * FROM electronic_products;
```

Resultant Query

product_id	product_name	price
201	Laptop	55000
202	Mobile	25000
204	Monitor	15000

Explanation

The electronic_products view displays only the products that belong to the **Electronics** category.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

CREATE INDEX index_name

ON table_name(column_name);

Create Index

CREATE INDEX idx_product_name

ON products(product_name);

Index Created

Use Index in Search

SELECT *

FROM products

WHERE product_name = 'Laptop';

Resultant Query

product_id	product_name	category	price

201	Laptop	Electronics	55000

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY increment_value;

Create Sequence

CREATE SEQUENCE product_seq

START WITH 206

INCREMENT BY 1;

Sequence Created

Insert Record Using Sequence

INSERT INTO products

VALUES (product_seq.NEXTVAL, 'Printer', 'Office', 12000);

COMMIT;

Record Inserted

Display Sequence Value

SELECT product_seq.CURRVAL AS current_sequence

FROM dual;

Resultant Query

current_sequence

206

Insert Another Record Using Sequence

INSERT INTO products

VALUES (product_seq.NEXTVAL, 'Scanner', 'Office', 9000);

COMMIT;

Record Inserted

The second product receives sequence value **207**.

Display All Product Records

SELECT *

FROM products

ORDER BY product_id;

Resultant Query

product_id	product_name	category	price
201	Laptop	Electronics	55000
202	Mobile	Electronics	25000
203	Keyboard	Accessories	2000
204	Monitor	Electronics	15000
205	Mouse	Accessories	1000
206	Printer	Office	12000
207	Scanner	Office	9000

Important Commands

Drop View

DROP VIEW electronic_products;

View Dropped

Drop Index

DROP INDEX idx_product_name;

Index Dropped

Drop Sequence

DROP SEQUENCE product_seq;

Sequence Dropped

RESULT

Thus, the Products table was created successfully, and the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected product records, the Index was used to improve data retrieval, and the Sequence was used to generate unique product IDs automatically.

EX. NO: 09 Creation of Stored Procedures and Functions using Accounts Table

DATE:26.08.2026

Aim

To create and execute **Stored Procedures and Functions** using PL/SQL with an accounts table.

Software Used

Oracle SQL / SQL Command Line

Description

Stored Procedure

A Stored Procedure is a named PL/SQL program stored in the database. It performs a specific task and may or may not return a value.

Function

A Function is a named PL/SQL program that performs a specific operation and always returns a single value.

1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and retrieve or modify data.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
(parameter_name IN datatype)
IS
BEGIN
   statements;
END;
/
```

Create Table

```
CREATE TABLE accounts (
    account_id NUMBER PRIMARY KEY,
    customer_name VARCHAR2(50),
    account_type VARCHAR2(30),
    balance NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO accounts VALUES (1001, 'Kavin', 'Savings', 45000);
INSERT INTO accounts VALUES (1002, 'Meena', 'Savings', 32000);
INSERT INTO accounts VALUES (1003, 'Arun', 'Current', 75000);
INSERT INTO accounts VALUES (1004, 'Divya', 'Savings', 60000);
INSERT INTO accounts VALUES (1005, 'Priya', 'Current', 85000);
COMMIT;
```

Records Inserted

Display Account Records

```
SELECT * FROM accounts;
```

Resultant Query

account_id	customer_name	account_type	balance
1001	Kavin	Savings	45000
1002	Meena	Savings	32000
1003	Arun	Current	75000

1004	Divya	Savings	60000
1005	Priya	Current	85000

Create Stored Procedure

The following procedure displays the details of a customer based on the account ID.

```
CREATE OR REPLACE PROCEDURE display_account (
    p_account_id IN NUMBER
)
IS
    v_name accounts.customer_name%TYPE;
    v_type accounts.account_type%TYPE;
    v_balance accounts.balance%TYPE;
BEGIN
    SELECT customer_name, account_type, balance
    INTO v_name, v_type, v_balance
    FROM accounts
    WHERE account_id = p_account_id;

    DBMS_OUTPUT.PUT_LINE('Customer Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Account Type: ' || v_type);
    DBMS_OUTPUT.PUT_LINE('Balance: ' || v_balance);
END;
/
```

Procedure Created

Execute Stored Procedure

```
SET SERVEROUTPUT ON;


EXEC display_account(1003);
```

Output

Customer Name: Arun

Account Type: Current

Balance: 75000

Explanation

The display_account procedure accepts an **account ID** as an input parameter and retrieves the corresponding customer name, account type, and balance from the accounts table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

```
CREATE OR REPLACE FUNCTION function_name

(parameter_name IN datatype)

RETURN datatype

IS

BEGIN

   statements;

   RETURN value;

END;

/
```

Create Function

The following function returns the **account category** based on the account balance.

```
CREATE OR REPLACE FUNCTION get_account_category (
    p_balance IN NUMBER
)
RETURN VARCHAR2
IS
    v_category VARCHAR2(20);
BEGIN
    IF p_balance >= 75000 THEN
        v_category := 'Premium';
    ELSIF p_balance >= 50000 THEN
        v_category := 'Gold';
    ELSIF p_balance >= 30000 THEN
        v_category := 'Regular';
    ELSE
        v_category := 'Basic';
    END IF;

    RETURN v_category;
END;
/
```

Function Created

Execute Function

```
SELECT account_id,
       customer_name,
       balance,
```

get_account_category(balance) AS category

FROM accounts;

Resultant Query

account_id	customer_name	balance	category
1001	Kavin	45000	Regular
1002	Meena	32000	Regular
1003	Arun	75000	Premium
1004	Divya	60000	Gold
1005	Priya	85000	Premium

Explanation

The get_account_category function accepts the account balance as input and returns the corresponding category such as **Basic, Regular, Gold, or Premium**.

Important Commands

Display Stored Procedure

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'DISPLAY_ACCOUNT';

Resultant Query

object_name	object_type
DISPLAY_ACCOUNT	PROCEDURE

Display Stored Function

SELECT object_name, object_type

FROM user_objects

WHERE object_name = 'GET_ACCOUNT_CATEGORY';

Resultant Query

object_name	object_type
GET_ACCOUNT_CATEGORY	FUNCTION

Drop Procedure

DROP PROCEDURE display_account;

Drop Function

DROP FUNCTION get_account_category;

Function Dropped

RESULT

Thus, the Accounts table was created successfully, and the Stored Procedure and Function were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve customer account details, and the Function was used to determine the account category based on the account balance.

EX. NO: 10 Creation of Triggers using Library Books Table

DATE: 28.08.2026

Aim

To create and execute **Triggers** using PL/SQL to automatically perform an action when an INSERT or UPDATE operation occurs on the library_books table.

Software Used

Oracle SQL / SQL Command Line

Description

A **Trigger** is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically validate data, prevent invalid values, and maintain audit records when a particular event takes place.

1. CREATE TABLE

```
CREATE TABLE library_books (
    book_id NUMBER PRIMARY KEY,
    book_name VARCHAR2(50),
    available_copies NUMBER,
    price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO library_books VALUES (301, 'Python Programming', 10, 450);
INSERT INTO library_books VALUES (302, 'Database Systems', 15, 600);
INSERT INTO library_books VALUES (303, 'Web Development', 8, 550);
INSERT INTO library_books VALUES (304, 'Artificial Intelligence', 12, 750);
INSERT INTO library_books VALUES (305, 'Computer Networks', 20, 500);
```

COMMIT;

Records Inserted

Display Book Records

SELECT * FROM library_books;

Resultant Query

book_id	book_name	available_copies	price
301	Python Programming	10	450
302	Database Systems	15	600
303	Web Development	8	550
304	Artificial Intelligence	12	750
305	Computer Networks	20	500

2. BEFORE INSERT TRIGGER

Definition

A **BEFORE INSERT Trigger** is automatically executed before a new record is inserted into a table.

Syntax

CREATE OR REPLACE TRIGGER trigger_name

BEFORE INSERT ON table_name

FOR EACH ROW

BEGIN

statements;

END;

/

Create Trigger

The following trigger checks whether the number of available copies is negative before inserting a book.

```
CREATE OR REPLACE TRIGGER check_book_copies

BEFORE INSERT ON library_books

FOR EACH ROW

BEGIN

   IF :NEW.available_copies < 0 THEN

      RAISE_APPLICATION_ERROR(

         -20001,

         'Available copies cannot be negative'

      );

   END IF;

END;

/
```

Trigger Created

Execute Trigger

```
INSERT INTO library_books

VALUES (306, 'Java Programming', -5, 650);
```

Output

ORA-20001: Available copies cannot be negative

The trigger prevents the insertion of an invalid number of copies.

3. BEFORE UPDATE TRIGGER

Definition

A **BEFORE UPDATE Trigger** is automatically executed before an existing record is updated.

Create Trigger

The following trigger prevents the book price from being updated to a negative value.

```
CREATE OR REPLACE TRIGGER check_book_price
BEFORE UPDATE OF price ON library_books
FOR EACH ROW
BEGIN
   IF :NEW.price < 0 THEN
      RAISE_APPLICATION_ERROR(
         -20002,
         'Book price cannot be negative'
      );
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
UPDATE library_books
SET price = -100
WHERE book_id = 303;
```

Output

ORA-20002: Book price cannot be negative

The trigger prevents the invalid price from being updated.

4. AFTER INSERT TRIGGER

Definition

An **AFTER INSERT Trigger** is automatically executed after a new record is successfully inserted into a table.

Create Audit Table

```
CREATE TABLE library_audit (
   audit_id NUMBER,
   book_id NUMBER,
   action_type VARCHAR2(20)
);
```

Table Created

Create Sequence

Before executing the audit trigger, create the required sequence.

```
CREATE SEQUENCE library_audit_seq
START WITH 1
INCREMENT BY 1;
```

Sequence Created

Create Trigger

```
CREATE OR REPLACE TRIGGER library_insert_audit
AFTER INSERT ON library_books
FOR EACH ROW
BEGIN
   INSERT INTO library_audit
   VALUES (
      library_audit_seq.NEXTVAL,
      :NEW.book_id,
      'INSERT'
```

```
    );
END;
/
```

Trigger Created

Insert New Book

```
INSERT INTO library_books
VALUES (306, 'Java Programming', 7, 650);

COMMIT;
```

Record Inserted

Display Audit Records

```
SELECT * FROM library_audit;
```

Resultant Query

audit_id	book_id	action_type
1	306	INSERT

The library_insert_audit trigger automatically records the newly inserted book in the audit table.

5. DISPLAY FINAL BOOK RECORDS

```
SELECT *
FROM library_books
ORDER BY book_id;
```

Resultant Query

book_id	book_name	available_copies	price
301	Python Programming	10	450

302	Database Systems	15	600
303	Web Development	8	550
304	Artificial Intelligence	12	750
305	Computer Networks	20	500
306	Java Programming	7	650

Important Commands

Display Triggers

SELECT trigger_name, status

FROM user_triggers;

Resultant Query

trigger_name	**status**
CHECK_BOOK_COPIES	ENABLED
CHECK_BOOK_PRICE	ENABLED
LIBRARY_INSERT_AUDIT	ENABLED

Drop Trigger

DROP TRIGGER check_book_copies;

Trigger Dropped

DROP TRIGGER check_book_price;

Trigger Dropped

DROP TRIGGER library_insert_audit;

Trigger Dropped

RESULT

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers automatically validated the number of available book copies and book price before insertion or modification, and recorded newly inserted book details in the audit table.

EX. NO:11 — Creation of Cursor Implementation using Patient Table

DATE: 01.09.2026

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process patient records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

Types of Cursors

1. Implicit Cursor
2. Explicit Cursor

In this practical, an **Explicit Cursor** is implemented.

1. CREATE TABLE

```
CREATE TABLE patient_details (
   patient_id NUMBER PRIMARY KEY,
   patient_name VARCHAR2(50),
   disease VARCHAR2(50),
   treatment_cost NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO patient_details VALUES (401, 'Arun', 'Fever', 1500);
```

INSERT INTO patient_details VALUES (402, 'Priya', 'Diabetes', 3500);

INSERT INTO patient_details VALUES (403, 'Kavin', 'Cold', 1000);

INSERT INTO patient_details VALUES (404, 'Divya', 'Heart Disease', 8000);

INSERT INTO patient_details VALUES (405, 'Meena', 'Asthma', 4500);

COMMIT;

Records Inserted

Display Patient Records

SELECT * FROM patient_details;

Resultant Query

patient_id	patient_name	disease	treatment_cost
401	Arun	Fever	1500
402	Priya	Diabetes	3500
403	Kavin	Cold	1000
404	Divya	Heart Disease	8000
405	Meena	Asthma	4500

2. EXPLICIT CURSOR

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and process them one by one.

Syntax

```
DECLARE
   CURSOR cursor_name IS
      SELECT statement;
```

```
BEGIN
   OPEN cursor_name;

   LOOP
      FETCH cursor_name INTO variables;

      EXIT WHEN cursor_name%NOTFOUND;

      statements;
   END LOOP;

   CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves each patient name and treatment cost from the patient_details table.

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR patient_cursor IS
      SELECT patient_name, treatment_cost
      FROM patient_details;

   v_patient_name patient_details.patient_name%TYPE;
```

```
    v_cost patient_details.treatment_cost%TYPE;

BEGIN
    OPEN patient_cursor;

    LOOP
        FETCH patient_cursor INTO v_patient_name, v_cost;

        EXIT WHEN patient_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Patient: ' || v_patient_name ||
            ' | Treatment Cost: ₹' || v_cost
        );
    END LOOP;

    CLOSE patient_cursor;
END;
/
```

Output

Patient: Arun | Treatment Cost: ₹1500

Patient: Priya | Treatment Cost: ₹3500

Patient: Kavin | Treatment Cost: ₹1000

Patient: Divya | Treatment Cost: ₹8000

Patient: Meena | Treatment Cost: ₹4500

3. CURSOR WITH CONDITION

The following cursor displays only the patients whose treatment cost is greater than ₹3000.

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR patient_cursor IS
      SELECT patient_id, patient_name, treatment_cost
      FROM patient_details
      WHERE treatment_cost > 3000;

   v_patient_id patient_details.patient_id%TYPE;
   v_patient_name patient_details.patient_name%TYPE;
   v_cost patient_details.treatment_cost%TYPE;

BEGIN
   OPEN patient_cursor;

   LOOP
      FETCH patient_cursor
      INTO v_patient_id, v_patient_name, v_cost;

      EXIT WHEN patient_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         'Patient ID: ' || v_patient_id ||
```

```
        ' | Patient: ' || v_patient_name ||
        ' | Treatment Cost: ₹' || v_cost
      );
  END LOOP;

  CLOSE patient_cursor;
END;
/
```

Resultant Query

patient_id	patient_name	treatment_cost
402	Priya	3500
404	Divya	8000
405	Meena	4500

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Cursor Attribute	Description
%FOUND	Returns TRUE if the last fetch returned a row
%NOTFOUND	Returns TRUE if the last fetch did not return a row
%ROWCOUNT	Returns the number of rows processed
%ISOPEN	Returns TRUE if the cursor is currently open

5. CURSOR USING %ROWCOUNT

The following program displays the total number of patients processed by the cursor.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
  CURSOR patient_cursor IS
    SELECT patient_name, treatment_cost
    FROM patient_details;

  v_patient_name patient_details.patient_name%TYPE;
  v_cost patient_details.treatment_cost%TYPE;

BEGIN
  OPEN patient_cursor;

  LOOP
    FETCH patient_cursor INTO v_patient_name, v_cost;

    EXIT WHEN patient_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
      'Patient: ' || v_patient_name ||
      ' | Treatment Cost: ₹' || v_cost
    );
  END LOOP;

  DBMS_OUTPUT.PUT_LINE(
    'Total Patients Processed: ' ||
```

```
        patient_cursor%ROWCOUNT
    );


    CLOSE patient_cursor;
END;
/
```

Output

Patient: Arun | Treatment Cost: ₹1500

Patient: Priya | Treatment Cost: ₹3500

Patient: Kavin | Treatment Cost: ₹1000

Patient: Divya | Treatment Cost: ₹8000

Patient: Meena | Treatment Cost: ₹4500

Total Patients Processed: 5

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process patient records one by one, apply conditions, and count the number of processed records**.**

EX. NO:12 — Creation of PL/SQL Exception Handling using Vehicle Service Table

DATE: 05.09.2026

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

Basic Structure

```
DECLARE
   declarations;
BEGIN
   statements;
EXCEPTION
   WHEN exception_name THEN
      statements;
END;
/
```

Types of Exceptions

Type	Description
Predefined Exception	Built-in exceptions provided by Oracle
User-Defined Exception	Exception created by the programmer

1. CREATE TABLE

```
CREATE TABLE vehicle_service (
   vehicle_id NUMBER PRIMARY KEY,
   owner_name VARCHAR2(50),
```

```
    vehicle_model VARCHAR2(50),
    service_cost NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO vehicle_service VALUES (601, 'Arun', 'Honda City', 5000);

INSERT INTO vehicle_service VALUES (602, 'Priya', 'Hyundai i20', 3500);

INSERT INTO vehicle_service VALUES (603, 'Kavin', 'Toyota Innova', 8000);

INSERT INTO vehicle_service VALUES (604, 'Divya', 'Maruti Swift', 2500);

INSERT INTO vehicle_service VALUES (605, 'Meena', 'Tata Nexon', 6000);

COMMIT;
```

Records Inserted

Display Vehicle Records

```
SELECT * FROM vehicle_service;
```

Resultant Query

vehicle_id	owner_name	vehicle_model	service_cost
601	Arun	Honda City	5000
602	Priya	Hyundai i20	3500
603	Kavin	Toyota Innova	8000

604	Divya	Maruti Swift	2500
605	Meena	Tata Nexon	6000

2. NO_DATA_FOUND EXCEPTION

Definition

NO_DATA_FOUND occurs when a SELECT INTO statement does not return any record.

Program

```
SET SERVEROUTPUT ON;

DECLARE
   v_owner vehicle_service.owner_name%TYPE;
   v_cost vehicle_service.service_cost%TYPE;

BEGIN
   SELECT owner_name, service_cost
   INTO v_owner, v_cost
   FROM vehicle_service
   WHERE vehicle_id = 699;

   DBMS_OUTPUT.PUT_LINE('Owner Name: ' || v_owner);
   DBMS_OUTPUT.PUT_LINE('Service Cost: ₹' || v_cost);

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Vehicle not found.');
```

```
END;
/
```

Output

Exception: Vehicle not found.

Exception Handled Successfully

3. TOO_MANY_ROWS EXCEPTION

Definition

TOO_MANY_ROWS occurs when a SELECT INTO statement returns more than one record.

Program

```
SET SERVEROUTPUT ON;

DECLARE
   v_owner vehicle_service.owner_name%TYPE;

BEGIN
   SELECT owner_name
   INTO v_owner
   FROM vehicle_service
   WHERE service_cost > 3000;

   DBMS_OUTPUT.PUT_LINE('Owner Name: ' || v_owner);

EXCEPTION
   WHEN TOO_MANY_ROWS THEN
```

```
        DBMS_OUTPUT.PUT_LINE(
            'Exception: More than one vehicle found.'
        );
END;
/
```

Output

Exception: More than one vehicle found.

Exception Handled Successfully

4. ZERO_DIVIDE EXCEPTION

Definition

ZERO_DIVIDE occurs when a number is divided by zero.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    v_cost NUMBER := 5000;
    v_divisor NUMBER := 0;
    v_result NUMBER;

BEGIN
    v_result := v_cost / v_divisor;

    DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
```

```
EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Cannot divide by zero.'
      );
END;
/
```

Output

Exception: Cannot divide by zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, the service payment must be equal to or greater than the service cost.

Program

```
SET SERVEROUTPUT ON;

DECLARE
   v_cost vehicle_service.service_cost%TYPE;
   v_payment NUMBER := 2000;

   insufficient_payment EXCEPTION;
```

```
BEGIN
   SELECT service_cost
   INTO v_cost
   FROM vehicle_service
   WHERE vehicle_id = 601;

   IF v_payment < v_cost THEN
      RAISE insufficient_payment;
   END IF;

   DBMS_OUTPUT.PUT_LINE('Payment Successful.');

EXCEPTION
   WHEN insufficient_payment THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Insufficient payment amount.'
      );
END;
/
```

Output

Exception: Insufficient payment amount.

User-Defined Exception Handled Successfully

6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program performs a valid payment when sufficient payment is available.

```
SET SERVEROUTPUT ON;

DECLARE
   v_cost vehicle_service.service_cost%TYPE;
   v_payment NUMBER := 6000;

BEGIN
   SELECT service_cost
   INTO v_cost
   FROM vehicle_service
   WHERE vehicle_id = 601;

   IF v_payment < v_cost THEN
      RAISE_APPLICATION_ERROR(
         -20001,
         'Insufficient payment amount'
      );
   END IF;
    DBMS_OUTPUT.PUT_LINE('Payment Successful.');

   DBMS_OUTPUT.PUT_LINE(
      'Paid Amount: ₹' || v_payment
   );

EXCEPTION
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```

Output

Payment Successful.

Paid Amount: ₹6000

Display Vehicle Record

```
SELECT *
FROM vehicle_service
WHERE vehicle_id = 601;
```

Resultant Query

vehicle_id	owner_name	vehicle_model	service_cost
601	Arun	Honda City	5000

Important Predefined Exceptions

Exception	Description
NO_DATA_FOUND	No record is found
TOO_MANY_ROWS	More than one record is returned
ZERO_DIVIDE	Division by zero occurs
DUP_VAL_ON_INDEX	Duplicate value is inserted into a unique/primary key column
VALUE_ERROR	Invalid value or conversion error occurs

Result

Thus, PL/SQL Exception Handling was successfully implemented using the VEHICLE_SERVICE table. The NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, and user-defined exceptions were handled successfully using appropriate exception-handling mechanisms.

EX. NO:13 — Creation of Database Normalization using Product Supplier Table

DATE: 08.09.2026

Aim

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and improve data integrity.

Software Used

Oracle SQL / SQL Command Line

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

Major Normal Forms

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. UNNORMALIZED TABLE

Consider the following **Product Supplier** table.

The table contains repeated supplier information. For example, supplier **Ravi** is repeated for products supplied by the same supplier.

product_id	product_name	supplier_id	supplier_name	category	price
601	Laptop	S01	Ravi	Electronics	55000
602	Mobile	S02	Kumar	Electronics	25000
603	Keyboard	S01	Ravi	Accessories	2000
604	Monitor	S03	Priya	Electronics	15000

2. FIRST NORMAL FORM (1NF)

Definition

A relation is in **1NF** when:

- Each column contains atomic values.
- There are no repeating groups.
- Each row is uniquely identifiable.

The above table contains atomic values, so it can be considered a **1NF table**.

1NF Table

product_id	product_name	supplier_id	supplier_name	category	price
601	Laptop	S01	Ravi	Electronics	55000
602	Mobile	S02	Kumar	Electronics	25000
603	Keyboard	S01	Ravi	Accessories	2000
604	Monitor	S03	Priya	Electronics	15000

Primary Key: product_id

3. SECOND NORMAL FORM (2NF)

Definition

A relation is in **2NF** when:

1. It is already in 1NF.
2. There is no partial dependency.
3. Every non-key attribute must depend on the complete primary key.

To demonstrate the decomposition, consider the product-supplier relationship with the composite key:

Primary Key: (product_id, supplier_id)

Some attributes depend only on one part of the composite key:

- product_name depends only on product_id.
- price depends only on product_id.
- supplier_name depends only on supplier_id.

Therefore, the table is divided into separate tables.

Product Table

product_id	**product_name**	**category**	**price**
601	Laptop	Electronics	55000
602	Mobile	Electronics	25000
603	Keyboard	Accessories	2000
604	Monitor	Electronics	15000

Supplier Table

supplier_id	**supplier_name**
S01	Ravi
S02	Kumar
S03	Priya

Product_Supplier Table

product_id	**supplier_id**
601	S01
602	S02
603	S01
604	S03

The partial dependencies are removed.

Therefore, the tables are in 2NF.

4. THIRD NORMAL FORM (3NF)

Definition

A relation is in **3NF** when:

1. It is already in 2NF.
2. There is no transitive dependency.
3. Non-key attributes must depend only on the primary key.

Suppose the Supplier table contains:

Supplier_ID → Supplier_Name, Supplier_City

and:

Supplier_City → Supplier_State

Here, Supplier_State depends indirectly on Supplier_ID through Supplier_City.

Therefore, to remove the transitive dependency, supplier location information can be separated.

Supplier Table

supplier_id	**supplier_name**	**city_id**
S01	Ravi	CT01
S02	Kumar	CT02
S03	Priya	CT03

City Table

city_id	**city_name**	**state**
CT01	Chennai	Tamil Nadu
CT02	Madurai	Tamil Nadu
CT03	Coimbatore	Tamil Nadu

Product Table

product_id	product_name	category	price
601	Laptop	Electronics	55000
602	Mobile	Electronics	25000
603	Keyboard	Accessories	2000
604	Monitor	Electronics	15000

Product_Supplier Table

product_id	supplier_id
601	S01
602	S02
603	S01
604	S03

Therefore, the tables are in 3NF.

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in BCNF when, for every non-trivial functional dependency:

The determinant must be a candidate key.

BCNF is a stronger version of 3NF.

Example

Consider the following Supplier_Product table:

supplier_id	product_id
S01	601
S01	603
S02	602
S03	604

Assume that each supplier is assigned a particular product category.

For example:

Supplier_ID → Product_Category

If supplier_id is not a candidate key of the Supplier_Product relationship because a supplier can supply multiple products, this dependency can cause a BCNF violation.

Therefore, the relationship can be decomposed into separate tables.

Supplier Category Table

supplier_id	category
S01	Accessories
S02	Electronics
S03	Electronics

Supplier_Product Table

supplier_id	product_id
S01	601
S01	603
S02	602
S03	604

Product Table

product_id	product_name	price
601	Laptop	55000
602	Mobile	25000
603	Keyboard	2000
604	Monitor	15000

After decomposition, the determinants are treated as keys in their corresponding relations.

Therefore, the relations are decomposed to satisfy BCNF.

Normalization Summary

Normal Form	Main Condition	Action

1NF	Atomic values	Remove repeating groups
2NF	No partial dependency	Separate partially dependent attributes
3NF	No transitive dependency	Separate indirectly dependent attributes
BCNF	Every determinant is a candidate key	Further decompose relations when required

Result

Thus, the Database Normalization process was studied and implemented using the Product–Supplier tables up to 1NF, 2NF, 3NF, and BCNF. The normalization process reduces data redundancy, removes dependency-related anomalies, and improves database consistency and integrity**.**

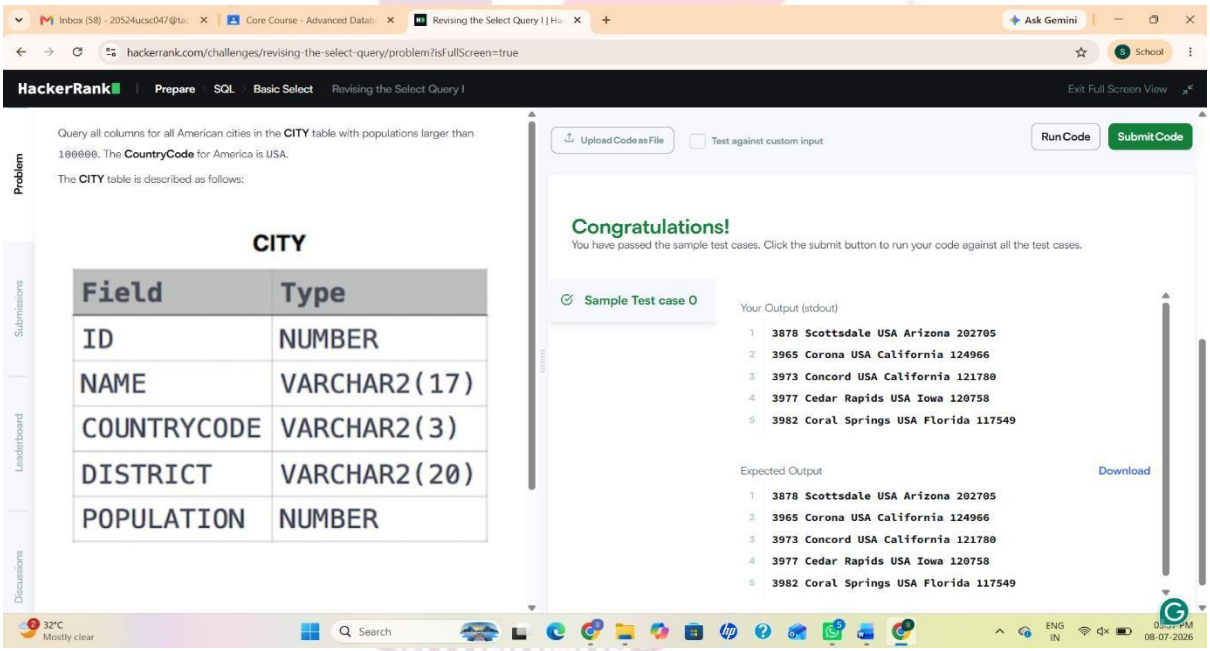
HACKRANK

1.REVISING THE SELECT QUERY I

PROGRAM

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION >
100000;
```

OUTPUT



2.SELECT BY ID

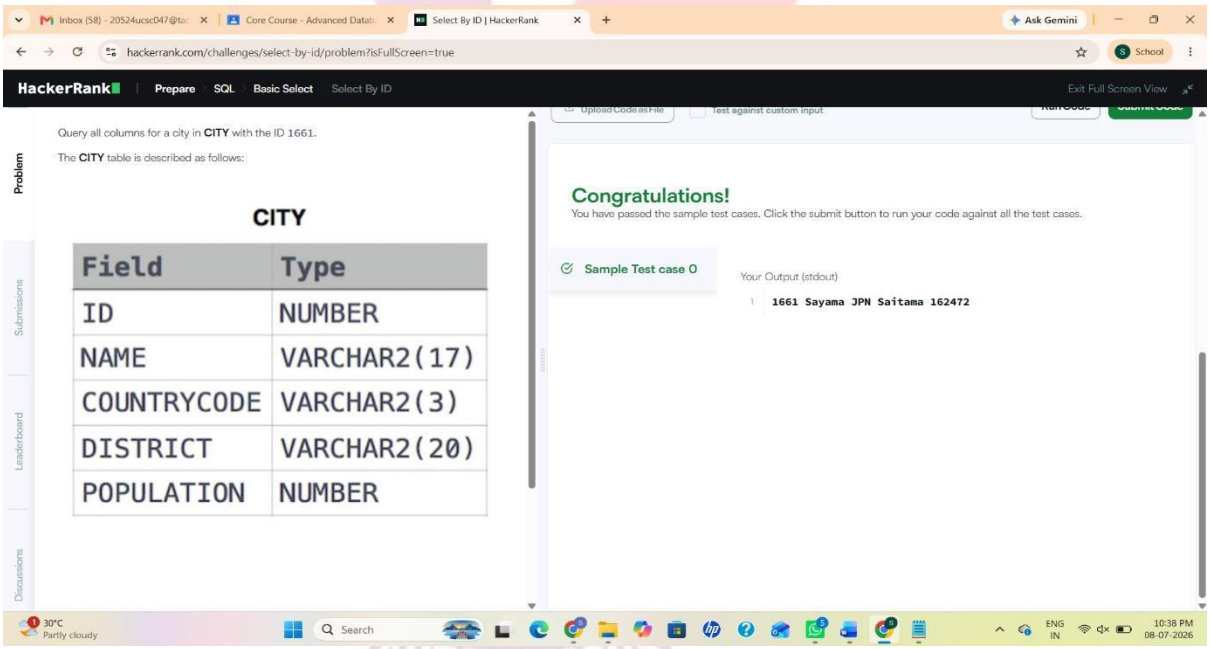
PROGRAM

```
SELECT t.*

FROM CITY t

WHERE t.ID=1661;
```

OUTPUT

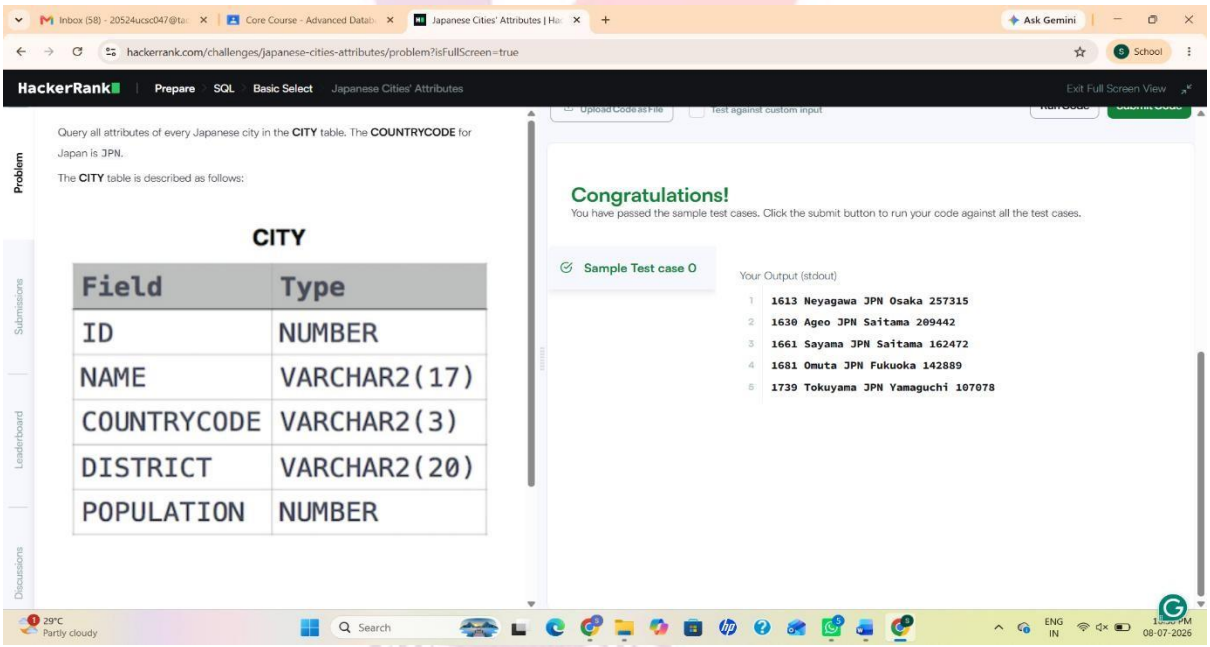


3.JAPANESE CITIES' ATTRIBUTES

PROGRAM

```
SELECT t.*
FROM CITY t
WHERE t.COUNTRYCODE = 'JPN';
```

OUTPUT



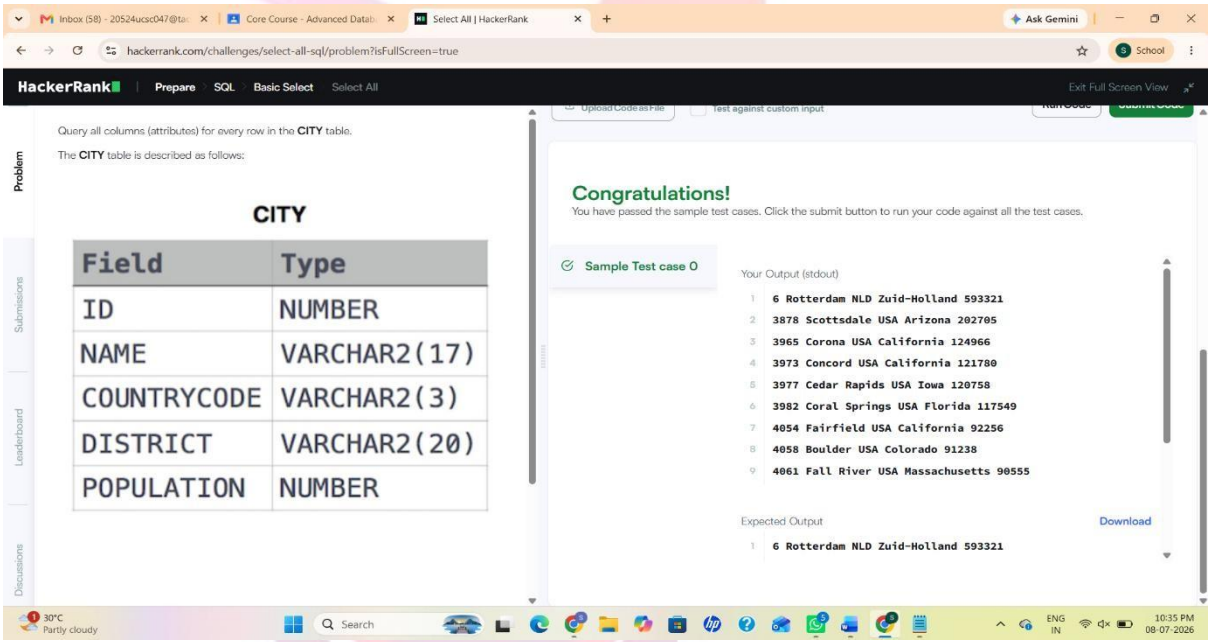
4.SELECT ALL

PROGRAM

```
SELECT t.*

FROM CITY t;
```

OUTPUT



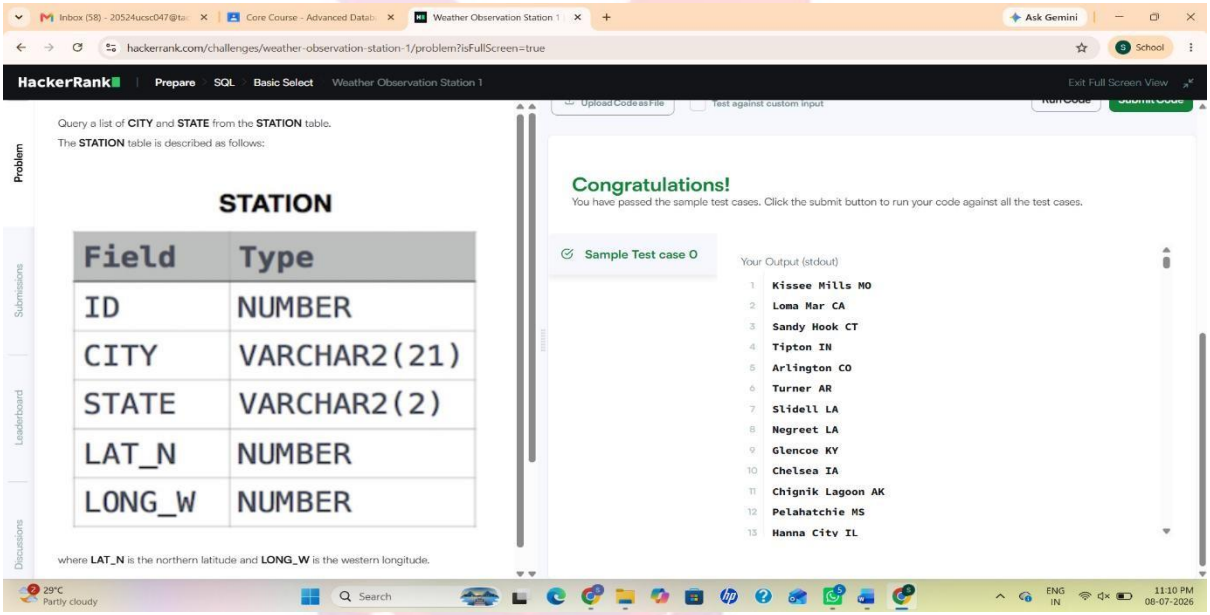
5. WEATHER OBSERVATION STATION 1

PROGRAM

```
SELECT t.CITY,t.STATE

FROM STATION t;
```

OUTPUT

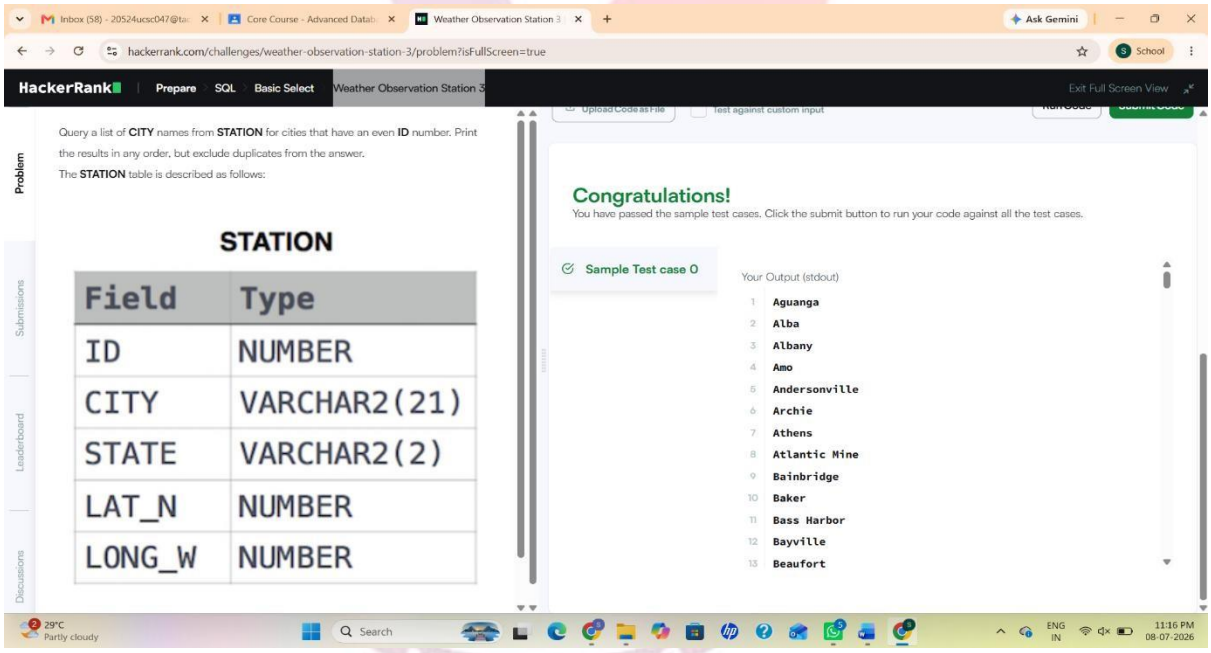


6. WEATHER OBSERVATION STATION 3

PROGRAM

```
SELECT DISTINCT t.CITY
FROM STATION t
WHERE MOD(t.ID,2) = 0;
```

OUTPUT



7.JAPANESE CITIES' NAMES

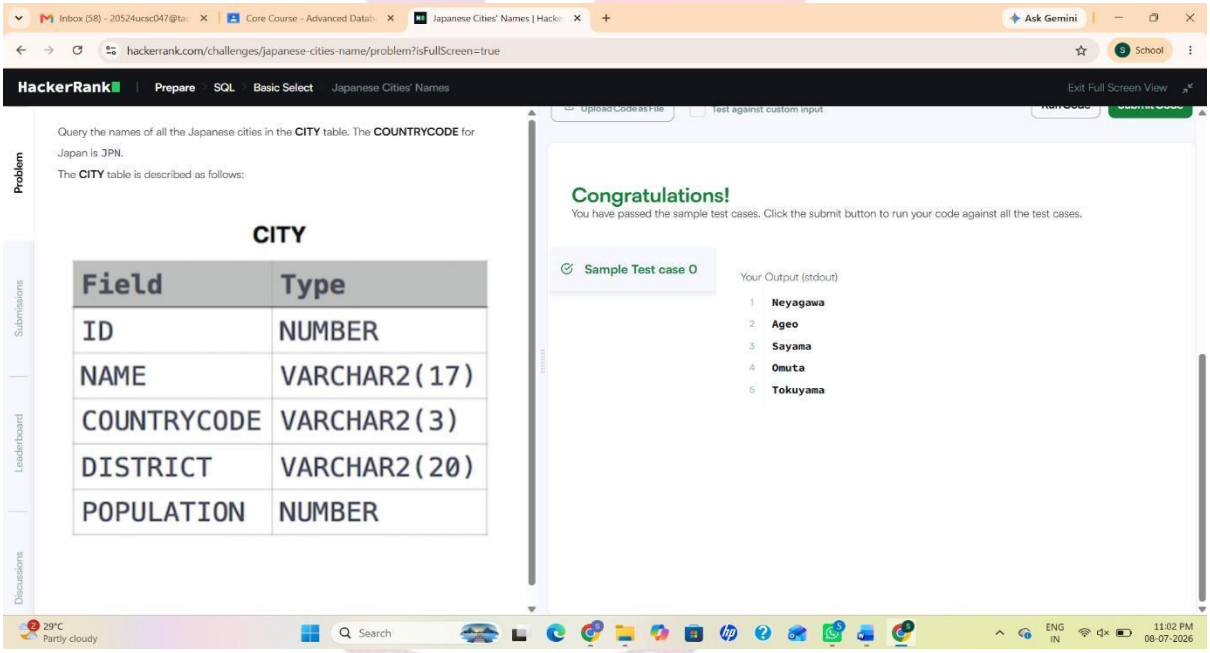
PROGRAM

```
SELECT t.NAME

FROM CITY t

WHERE t.COUNTRYCODE = 'JPN';
```

OUTPUT



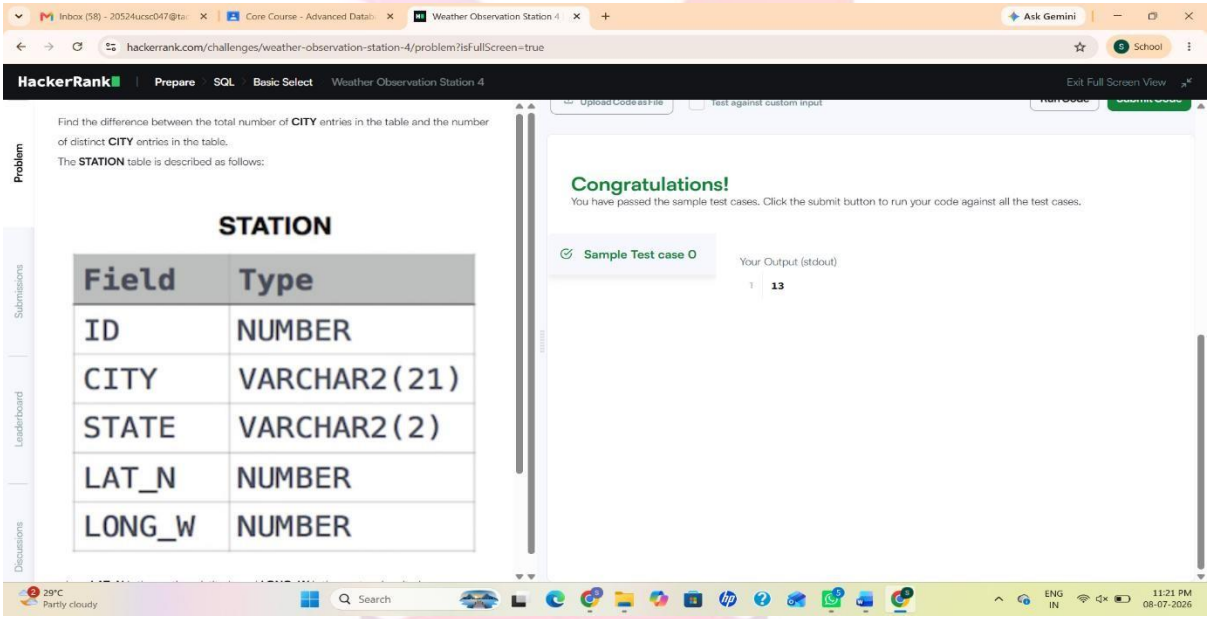
8.WEATHER OBSERVATION STATION 4

PROGRAM

```
SELECT COUNT(t.CITY) - COUNT(DISTINCT t.CITY)

FROM STATION t;
```

OUTPUT



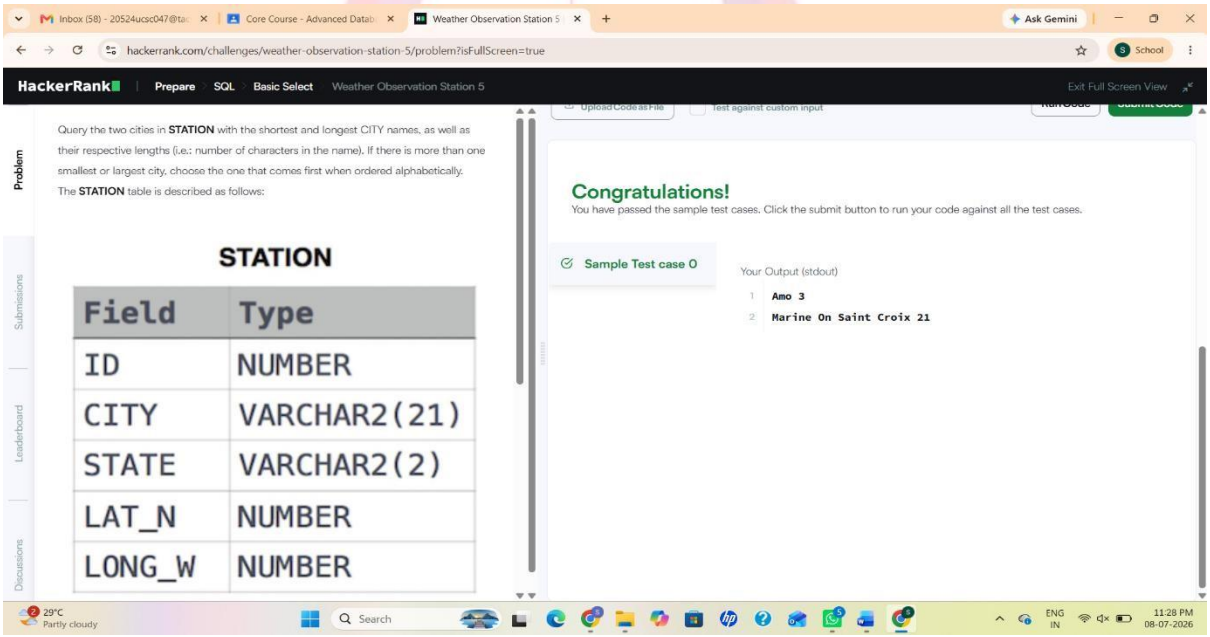
9. WEATHER OBSERVATION STATION 5

PROGRAM

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY), CITY
FETCH FIRST 1 ROW ONLY;
```

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY
FETCH FIRST 1 ROW ONLY;
```

OUTPUT

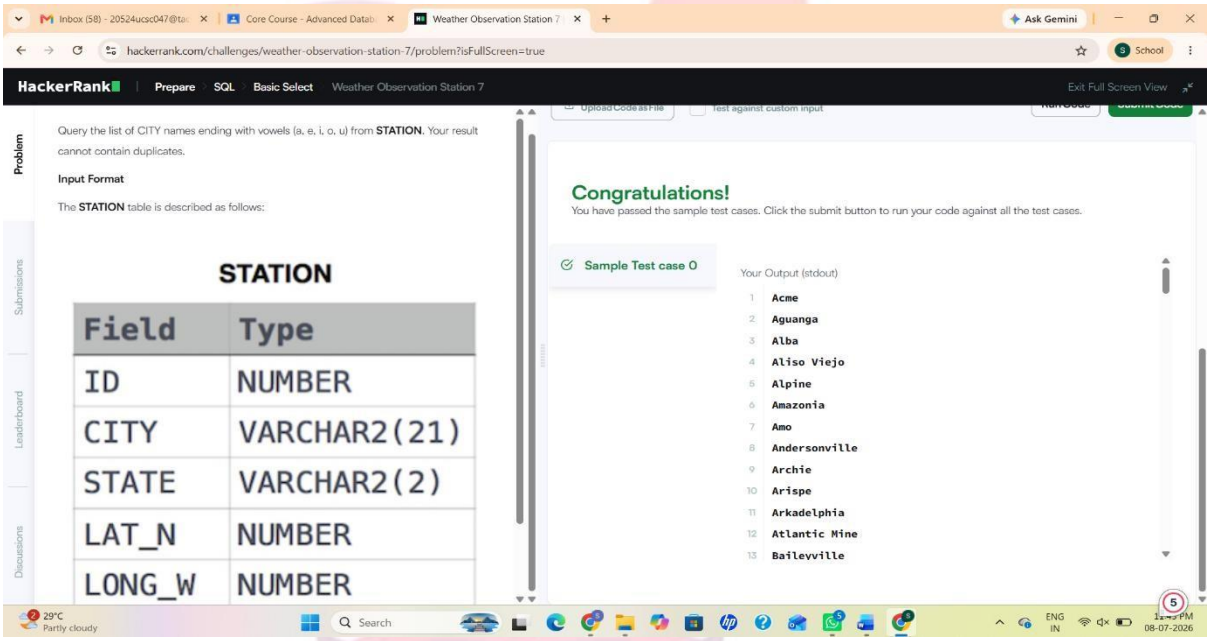


10. WEATHER OBSERVATION STATION 7

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE
UPPER(SUBSTR(CITY,LENGTH(CITY),1)) IN ('A','E','I','O','U');
```

OUTPUT

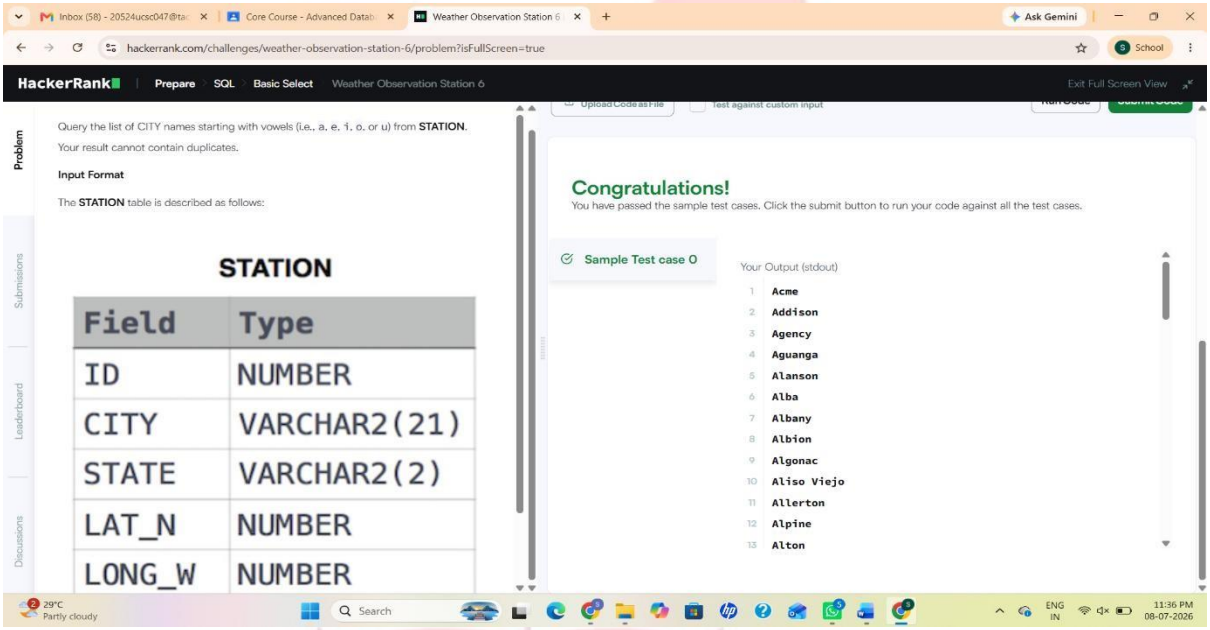


11.WEATHER OBSERVATION STATION 6

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) IN
('A','E','I','O','U');
```

OUTPUT

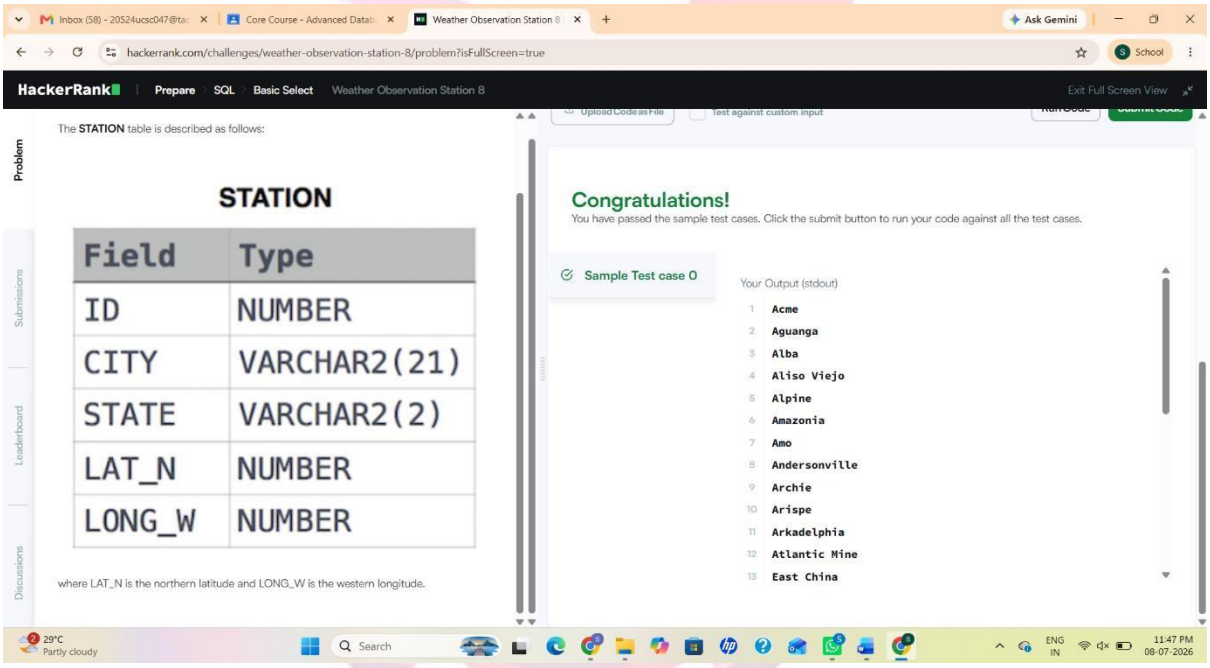


12.WHETHER OBSERVATION STATION

PROGRAM

('A','E','I','O','U') AND UPPER(SUBSTR(CITY,LENGTH(CITY),1)) IN ('A','E','I','O','U');
OUTPUT SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) IN

OUTPUT

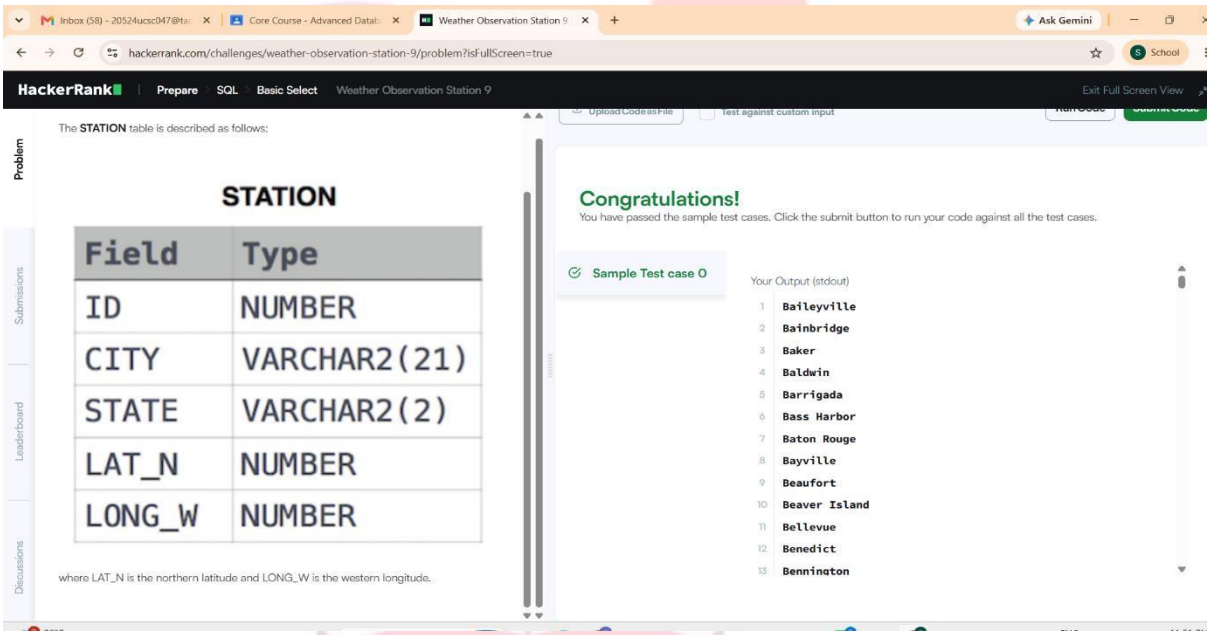


13.WHETHER OBSERVATION STATION

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT
IN ('A','E','I','O','U');
```

OUTPUT

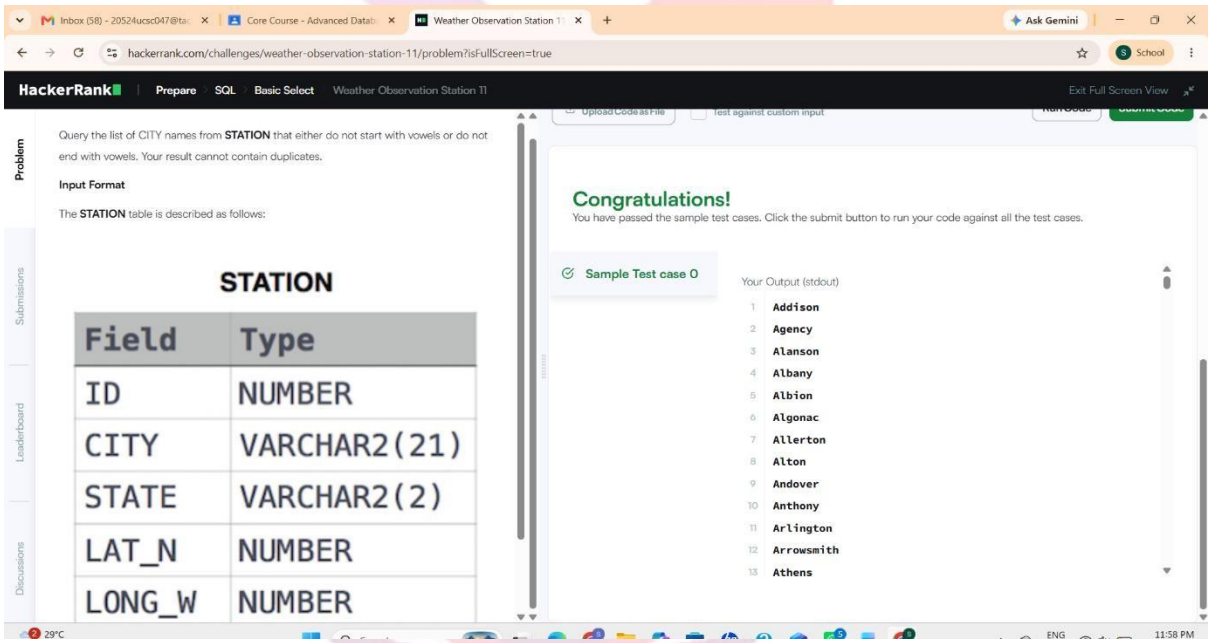


14.WEATHER OBSERVATION STATION 11

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT
IN ('A','E','I','O','U') OR UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN
('A','E','I','O','U');
```

OUTPUT

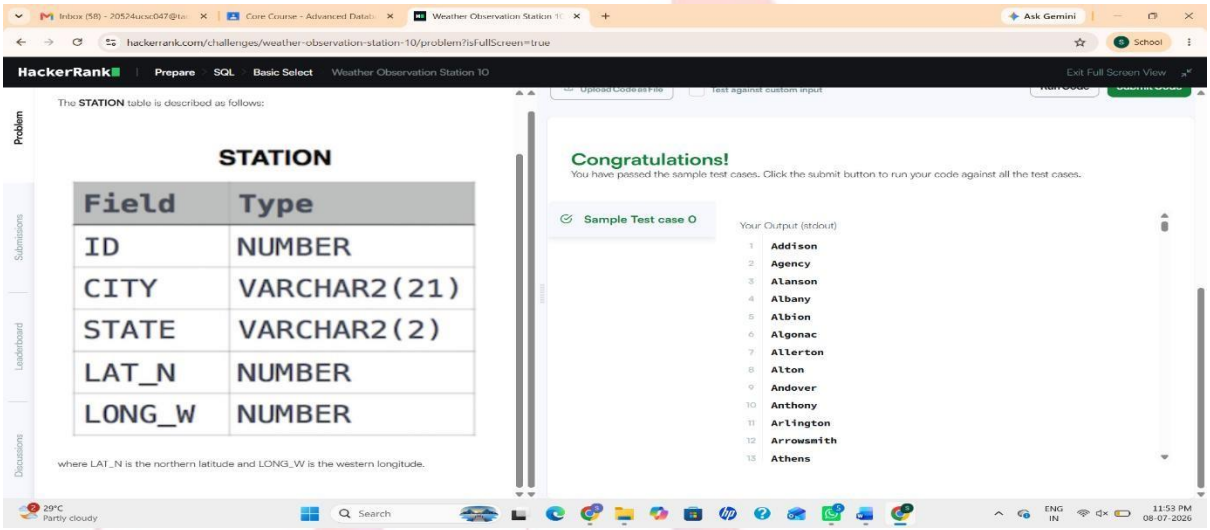


15.WHETHER OBSERVATION STATION

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE
UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN ('A','E','I','O','U');
```

OUTPUT

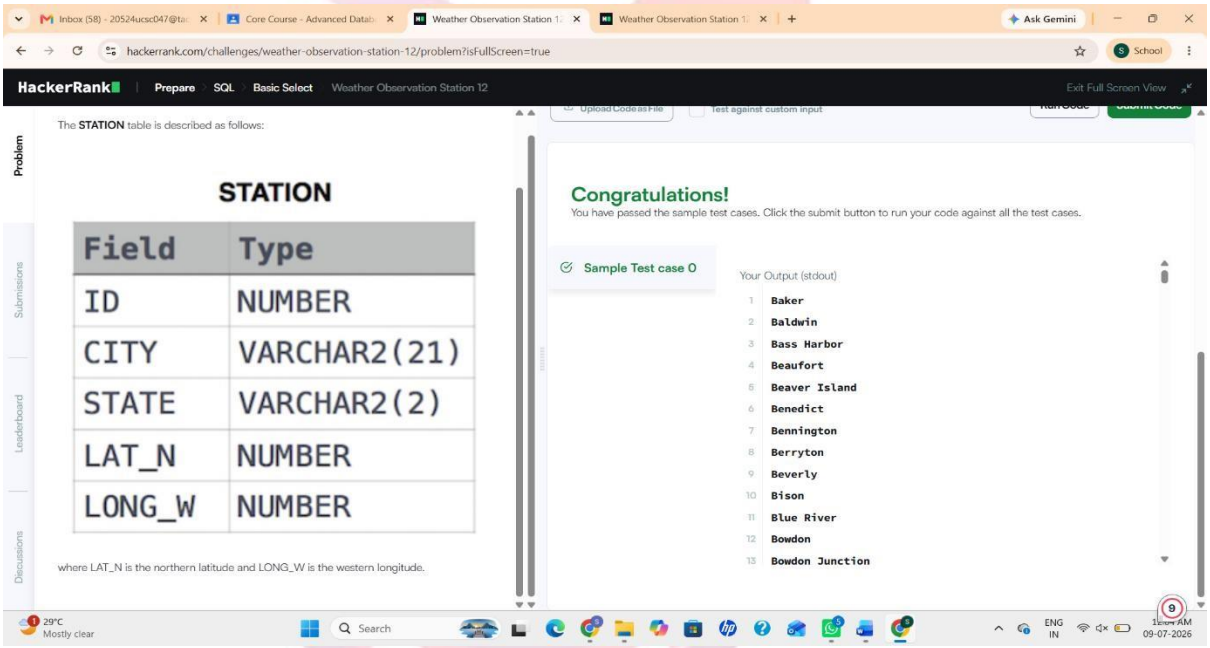


16.WEATHER OBSERVATION STATION 12

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT
IN ('A','E','I','O','U') AND UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN
('A','E','I','O','U');
```

OUTPUT

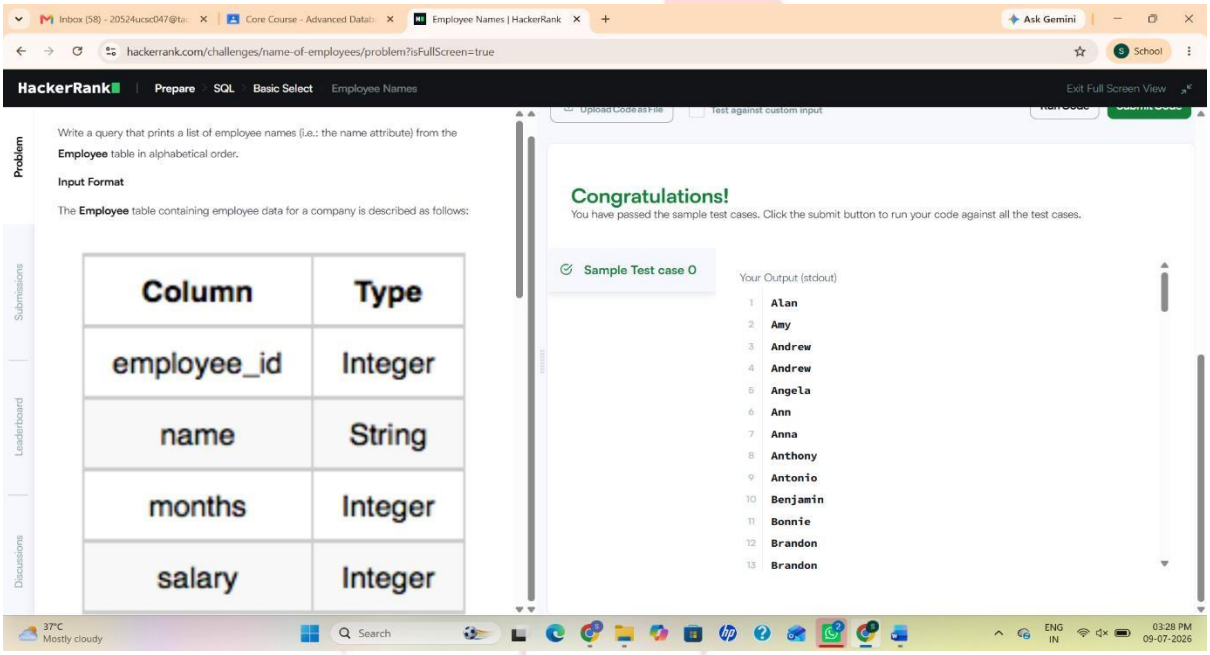


17.EMPLOYEE NAMES

PROGRAM

```
SELECT name FROM Employee ORDER BY name;
```

OUTPUT

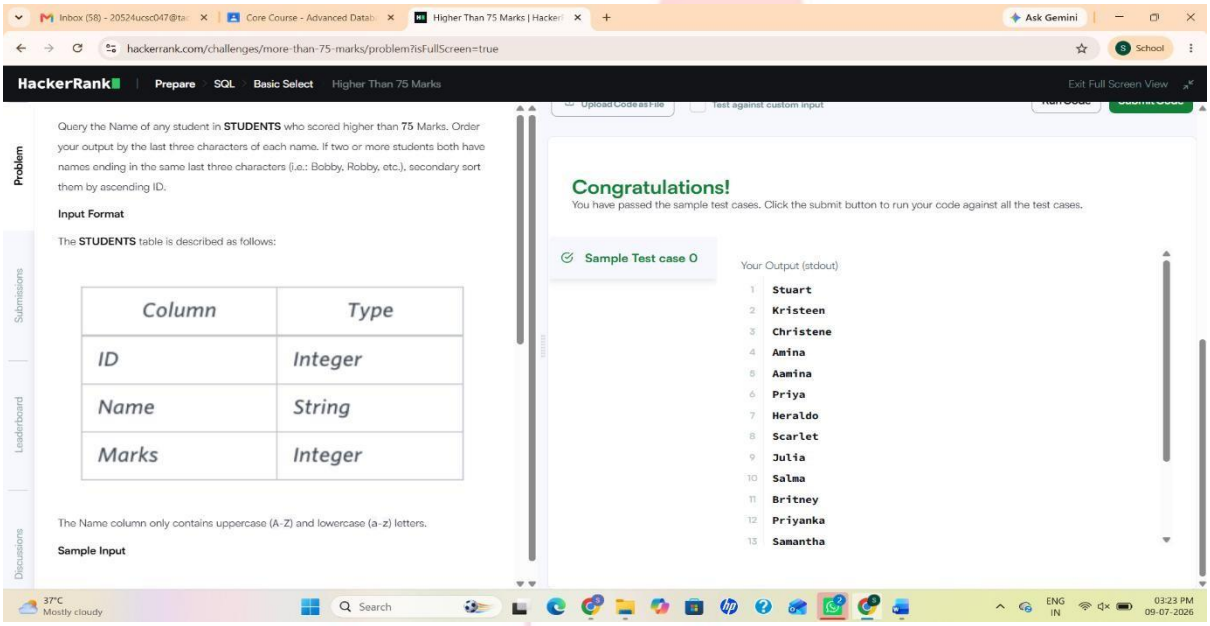


18. HIGHER THAN 75 MARKS

PROGRAM

```
SELECT Name FROM STUDENTS WHERE Marks > 75 ORDER BY SUBSTR(Name, LENGTH(Name)-2, 3), ID;
```

OUTPUT

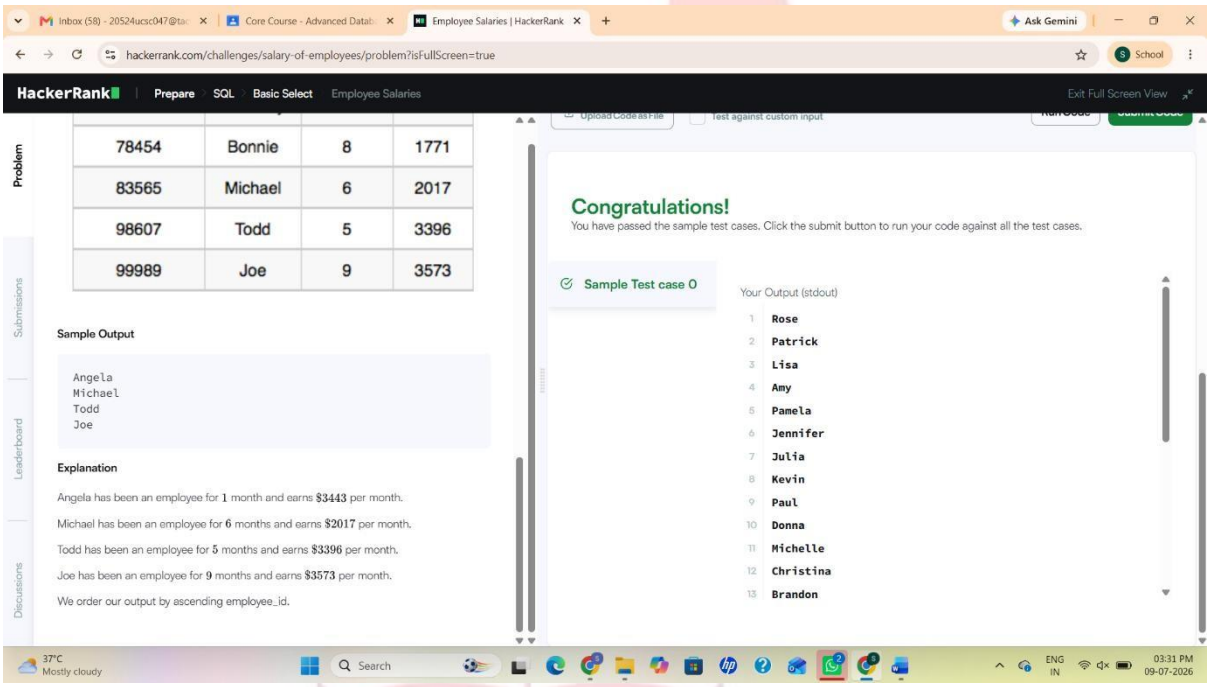


19.EMPLOYEE SALARIES

PROGRAM

```
SELECT name FROM Employee WHERE salary > 2000 AND months < 10 ORDER BY employee_id;
```

OUTPUT

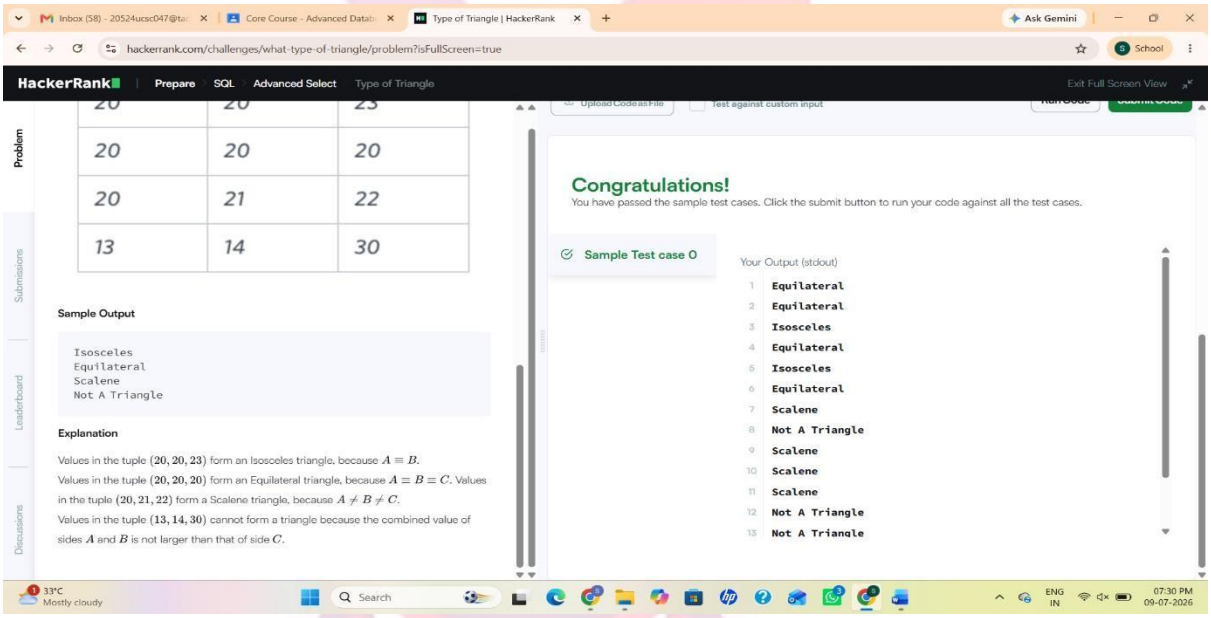


20.TYPES OF TRIANGLE

PROGRAM

```
SELECT CASE WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
WHEN A = B AND B = C THEN 'Equilateral' WHEN A = B OR B = C OR A = C THEN 'Isosceles'
ELSE 'Scalene' END FROM TRIANGLES;
```

OUTPUT



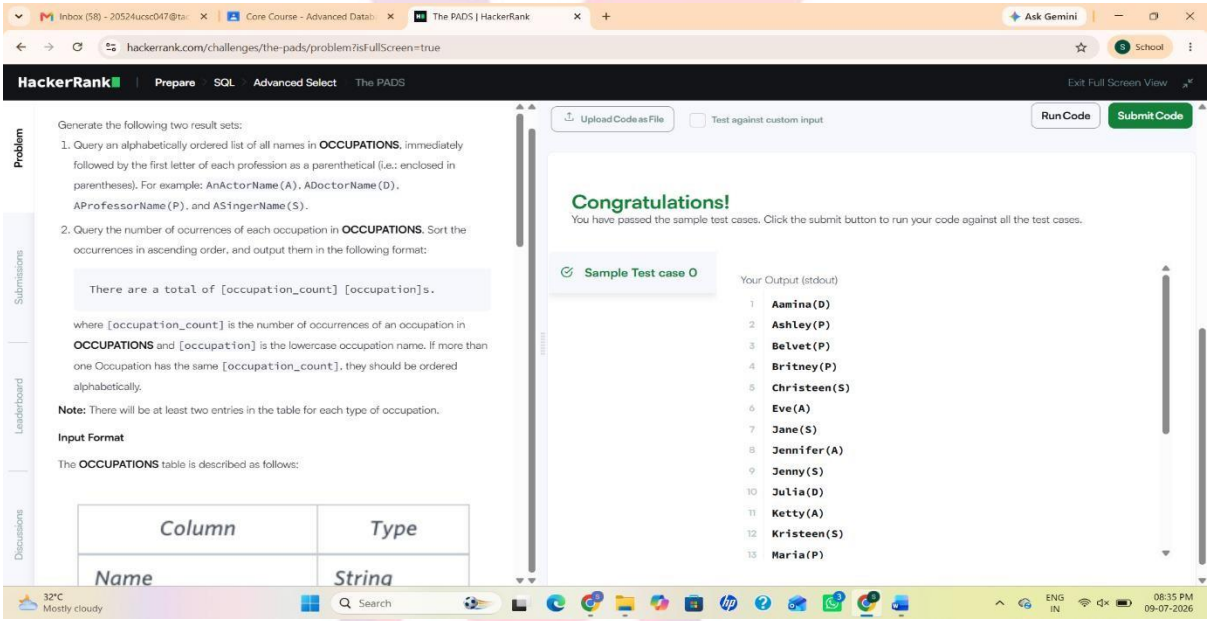
21.THE PADS

PROGRAM

```
SELECT NAME || '(' || SUBSTR(OCCUPATION,1,1) || ')' FROM OCCUPATIONS ORDER BY NAME;

SELECT 'There are a total of ' || COUNT(*) || ' ' || LOWER(OCCUPATION) || 's.' FROM OCCUPATIONS GROUP BY OCCUPATION ORDER BY COUNT(*), OCCUPATION;
```

OUTPUT



22.REVISING AGGREGATIONS – THE COUNT FUNCTION

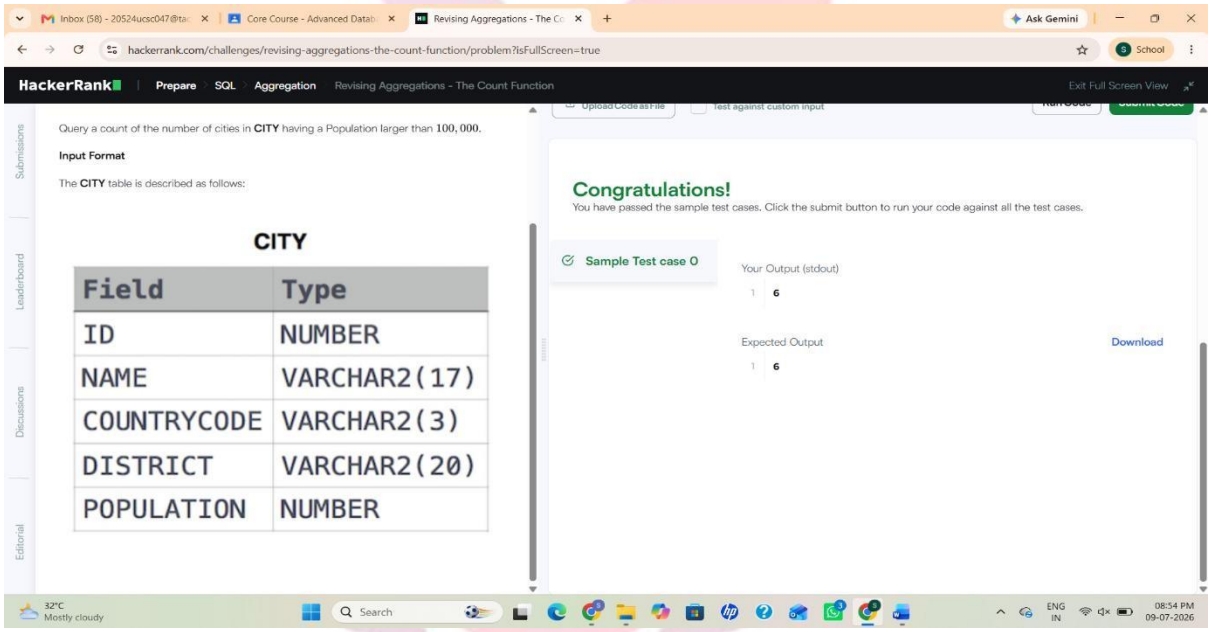
PROGRAM

```
SELECT COUNT(*)

FROM CITY

WHERE POPULATION > 100000;
```

OUTPUT



23. REVISING AGGREGATIONS – THE SUM FUNCTION

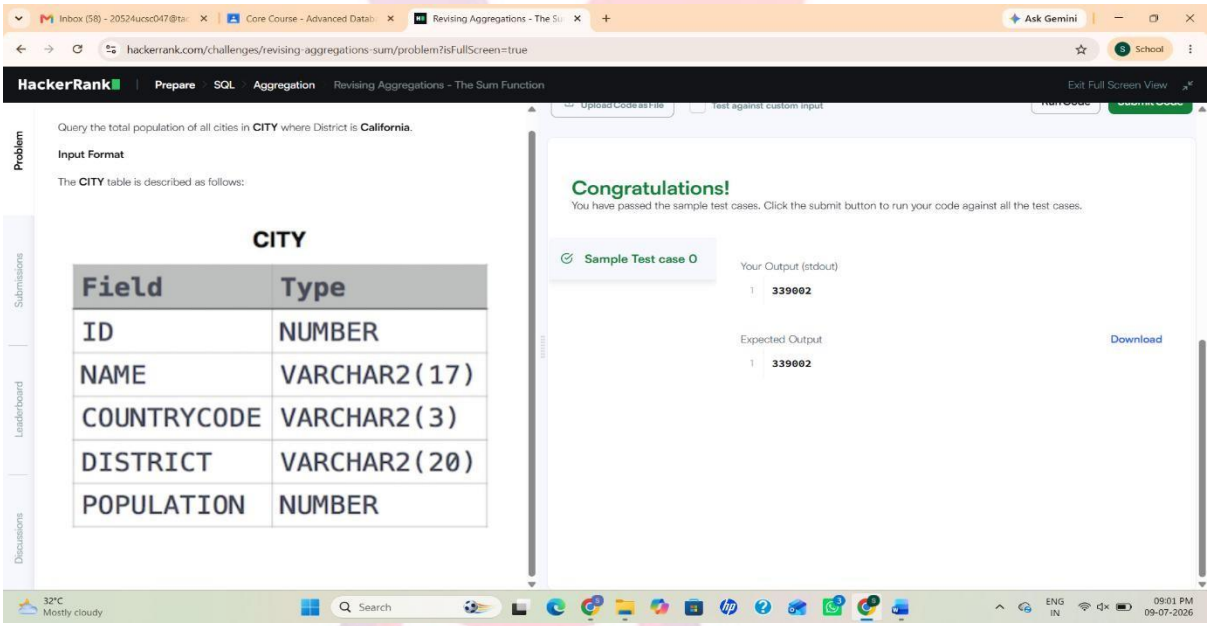
PROGRAM

```
SELECT SUM(POPULATION)

FROM CITY

WHERE DISTRICT = 'California';
```

OUTPUT

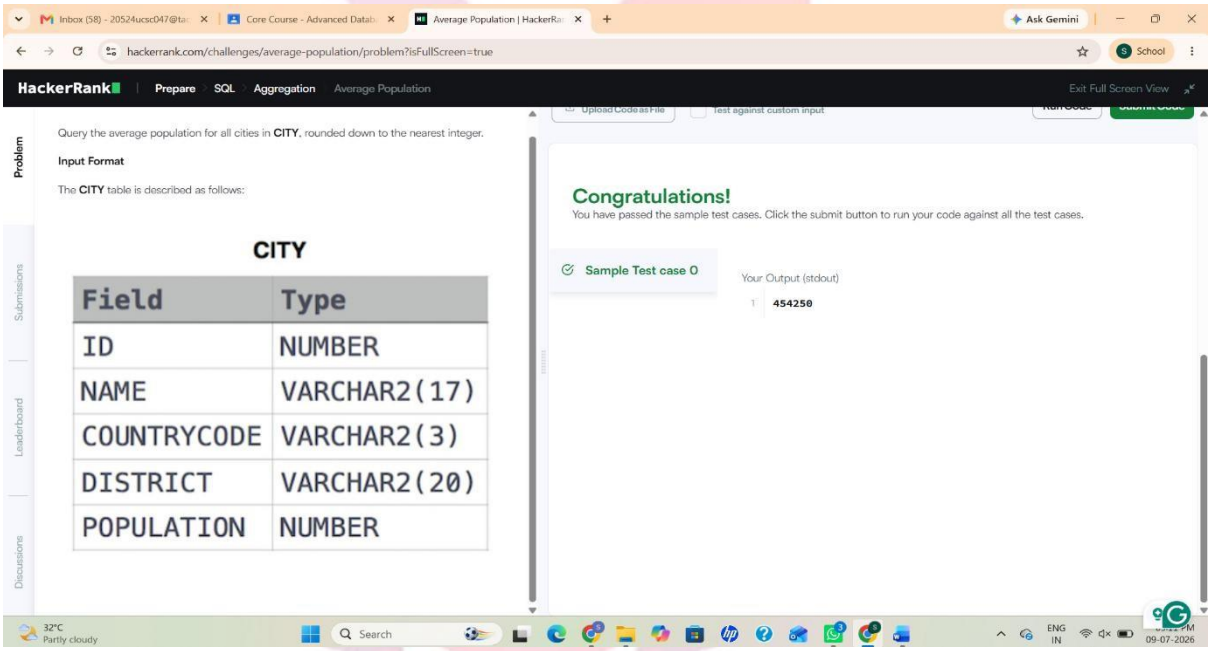


24.AVERGE POPULATION

PROGRAM

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT



25. REVISING AGGREGATIONS – AVERAGES

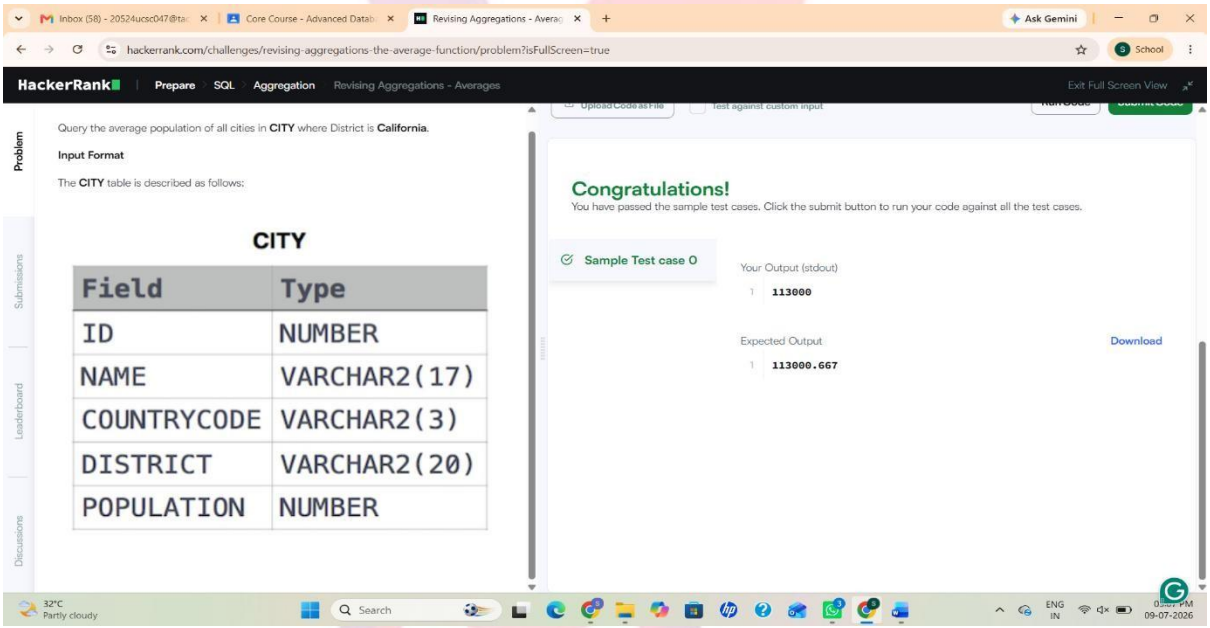
PROGRAM

```
SELECT AVG(POPULATION)

FROM CITY

WHERE DISTRICT = 'California';
```

OUTPUT



26.JAPAN POPULATION

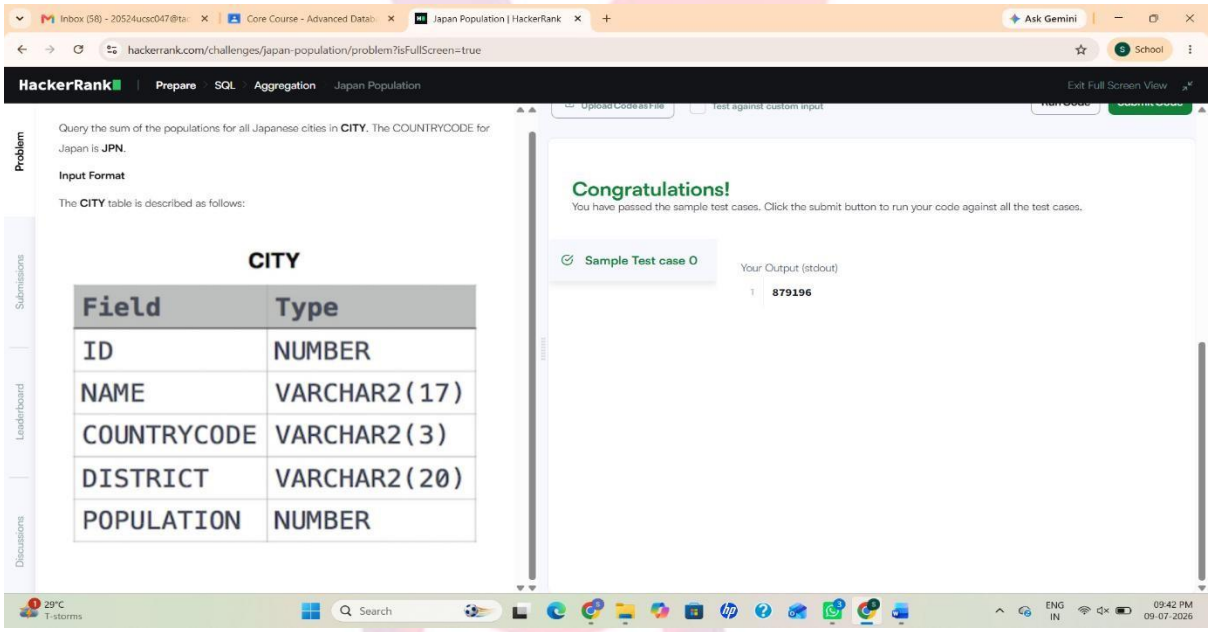
PROGRAM

```
SELECT SUM(POPULATION)

FROM CITY

WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



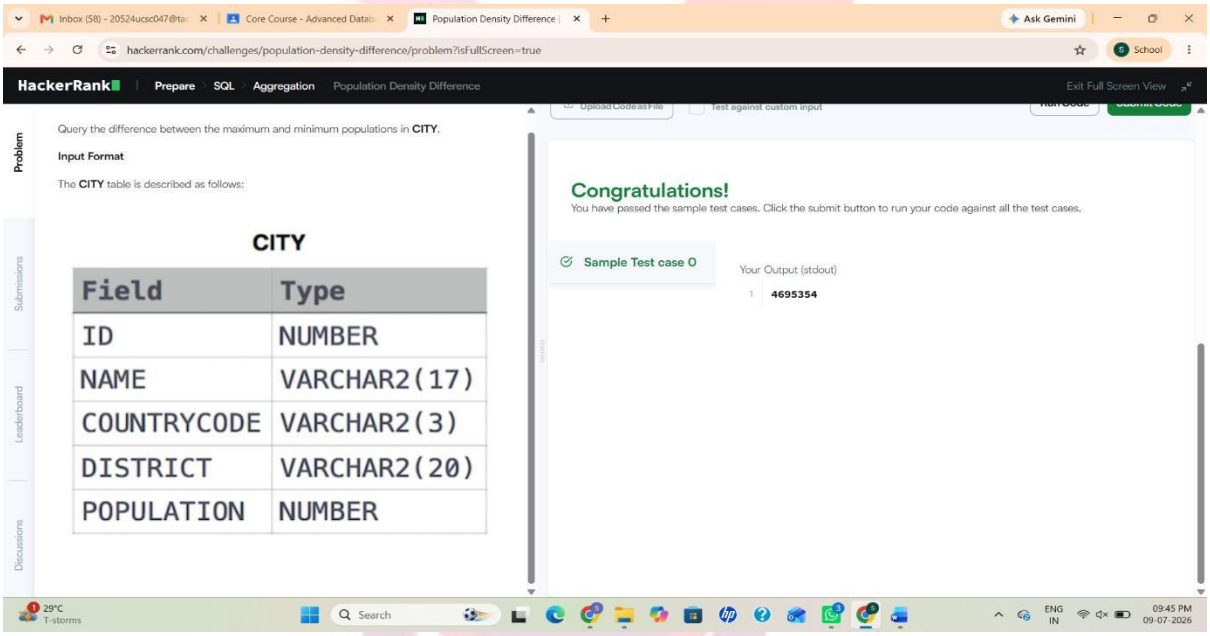
27.POPULATION DENSITY DIFFERENCE

PROGRAM

```
SELECT MAX(POPULATION) - MIN(POPULATION)

FROM CITY;
```

OUTPUT

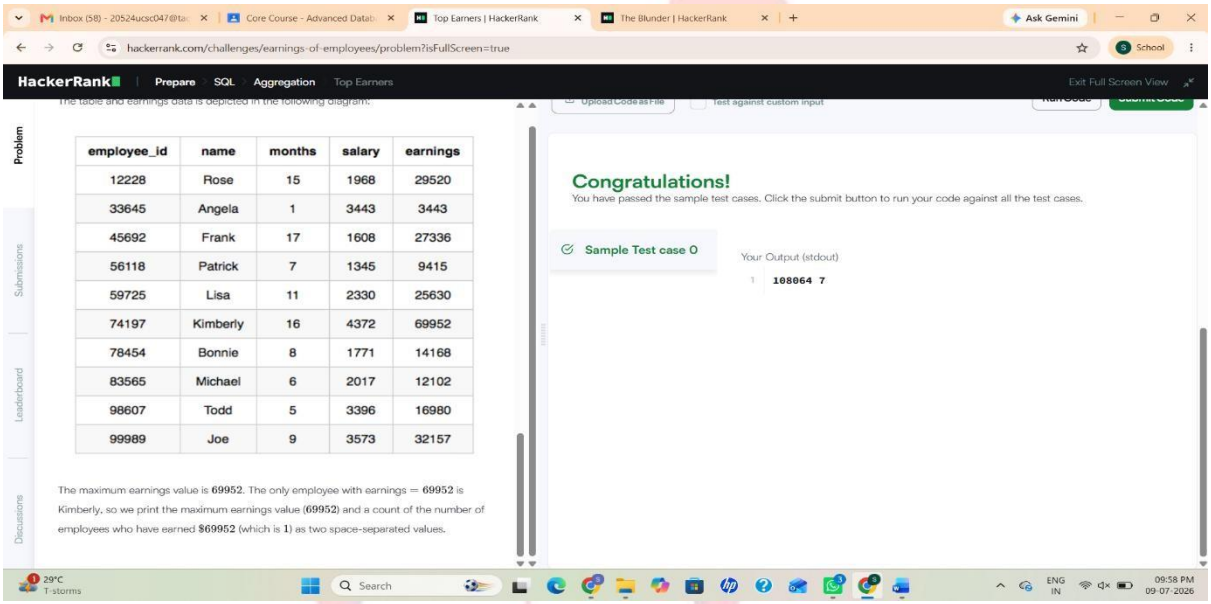


28.TOP EARNERS

PROGRAM

```
SELECT MAX(salary * months), COUNT(*) FROM Employee WHERE salary * months = (SELECT
MAX(salary * months) FROM Employee);
```

OUTPUT

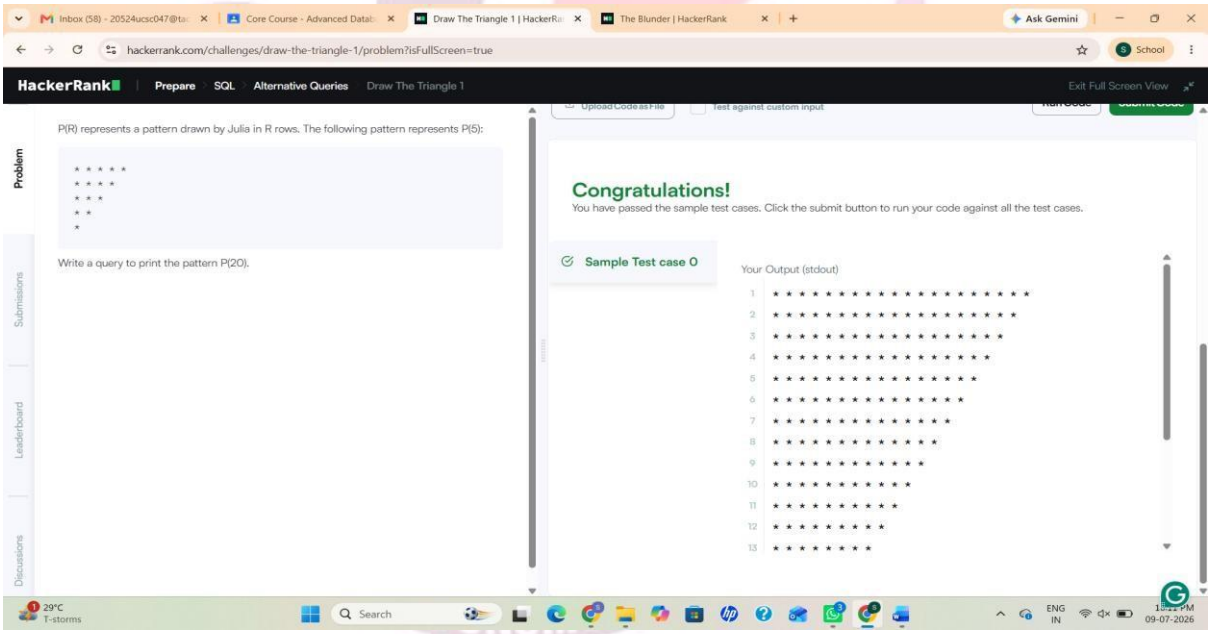


29.DRAW THE TRIANGLE 1

PROGRAM

```
SET @n = 21;
SELECT REPEAT('* ', @n := @n - 1)
FROM information_schema.tables
LIMIT 20;
```

OUTPUT

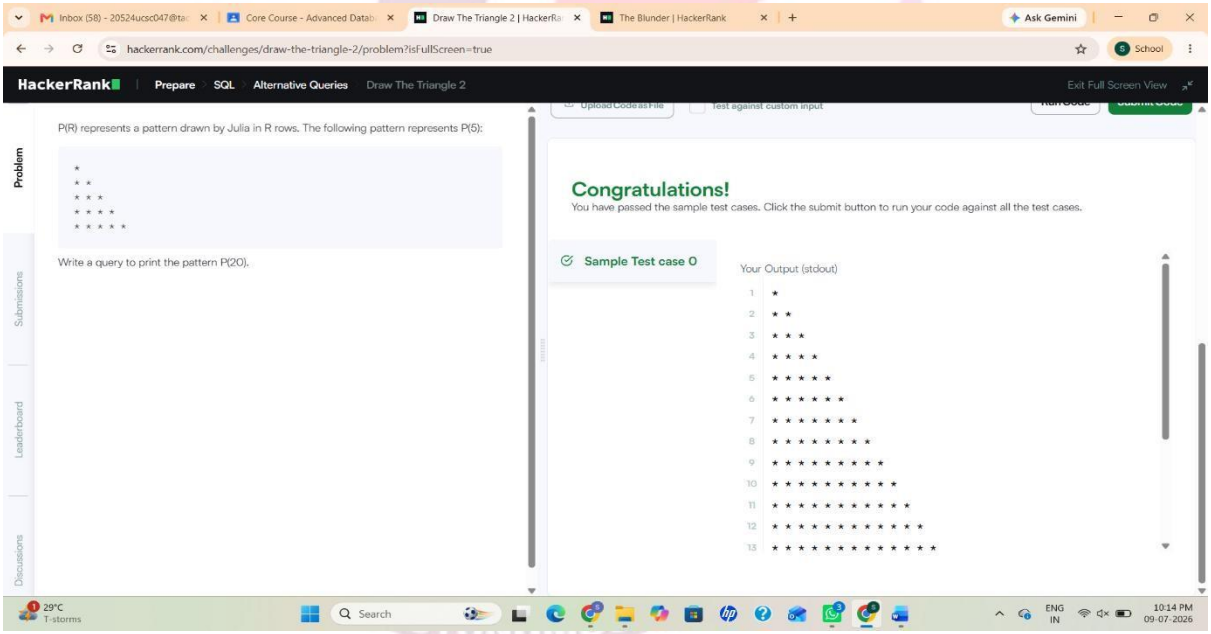


30. DRAW THE TRIANGLE 2

PROGRAM

```
SELECT REPEAT('* ', @n := @n + 1)
FROM information_schema.tables,
(SELECT @n := 0) t
LIMIT 20;
```

OUTPUT

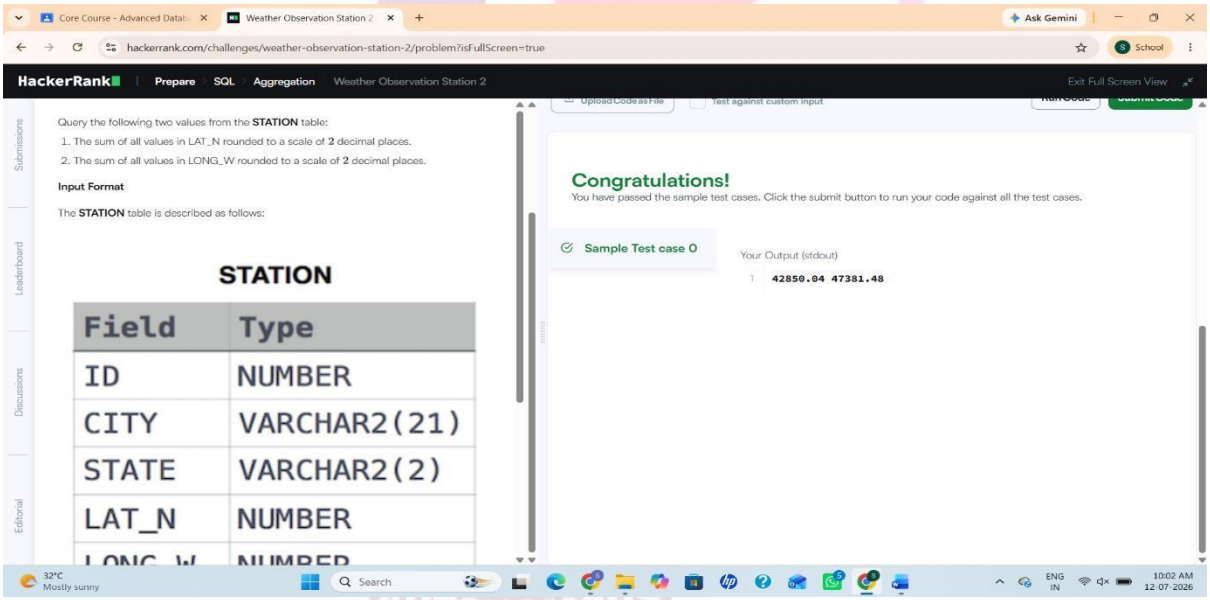


31.WEATHER OBSERVATION STATION 2

PROGRAM

```
SELECT
   ROUND(SUM(LAT_N), 2),
   ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT

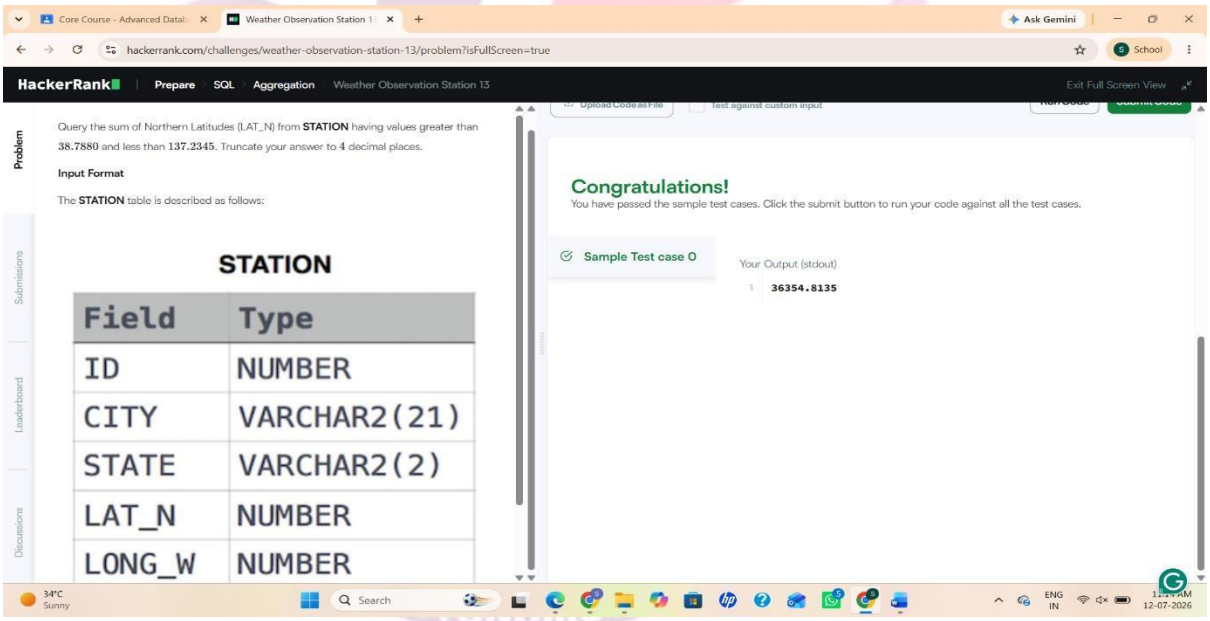


32.WEATHER OBSERVATION STATION 13

PROGRAM

```
SELECT TRUNCATE(SUM(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7880
  AND LAT_N < 137.2345;
```

OUTPUT



33.WEATHER OBSERVATION STATION 14

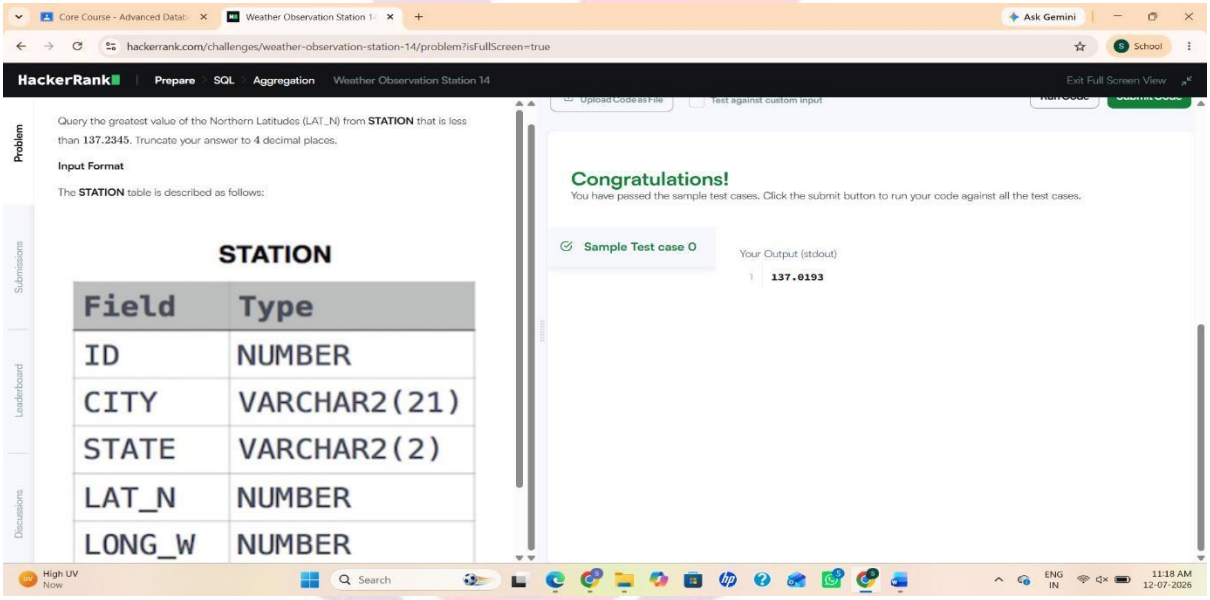
PROGRAM

```
SELECT TRUNCATE(MAX(LAT_N), 4)

FROM STATION

WHERE LAT_N < 137.2345;
```

OUTPUT

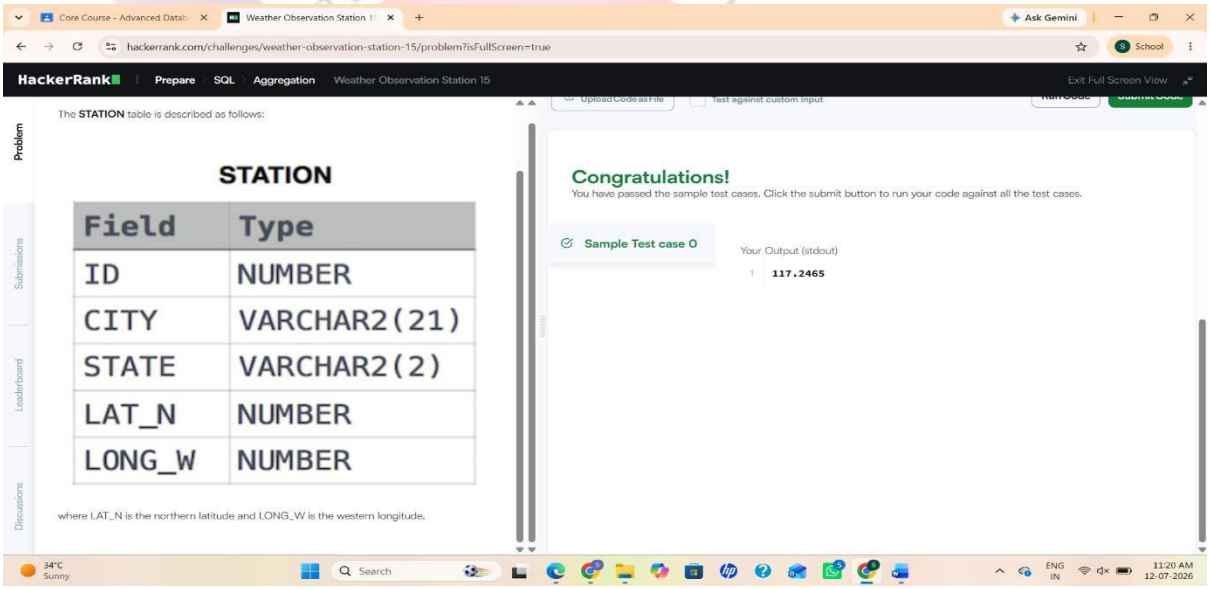


34.WEATHER OBSERVATION STATION 15

PROGRAM

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MAX(LAT_N)
   FROM STATION
   WHERE LAT_N < 137.2345
);
```

OUTPUT



35.WEATHER OBSERVATION STATION 16

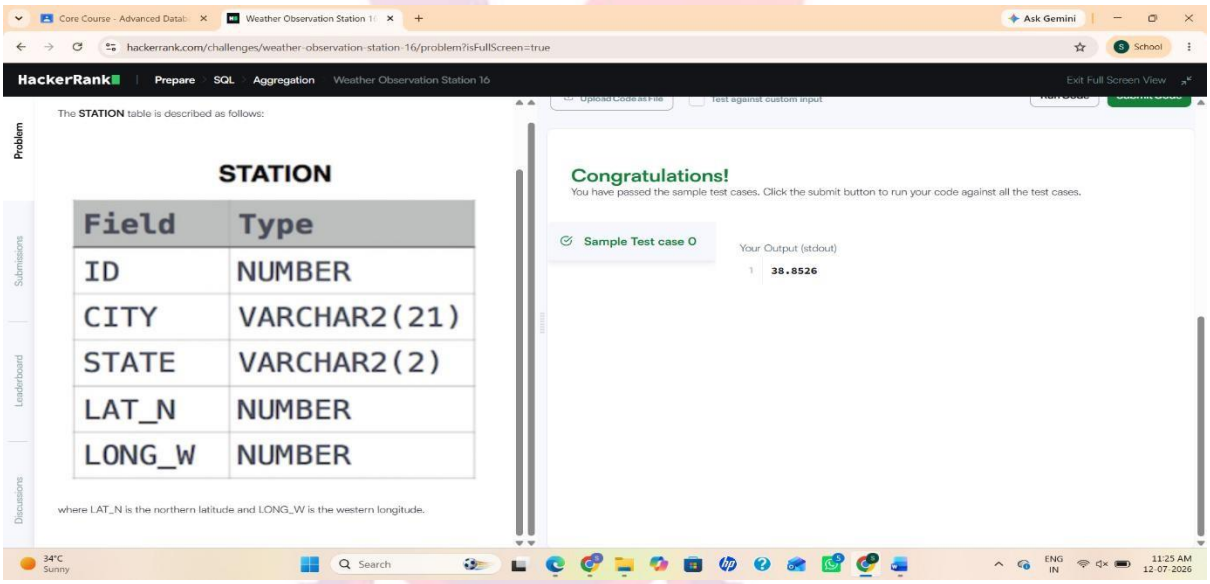
PROGRAM

```
SELECT ROUND(MIN(LAT_N), 4)

FROM STATION

WHERE LAT_N > 38.7780;
```

OUTPUT

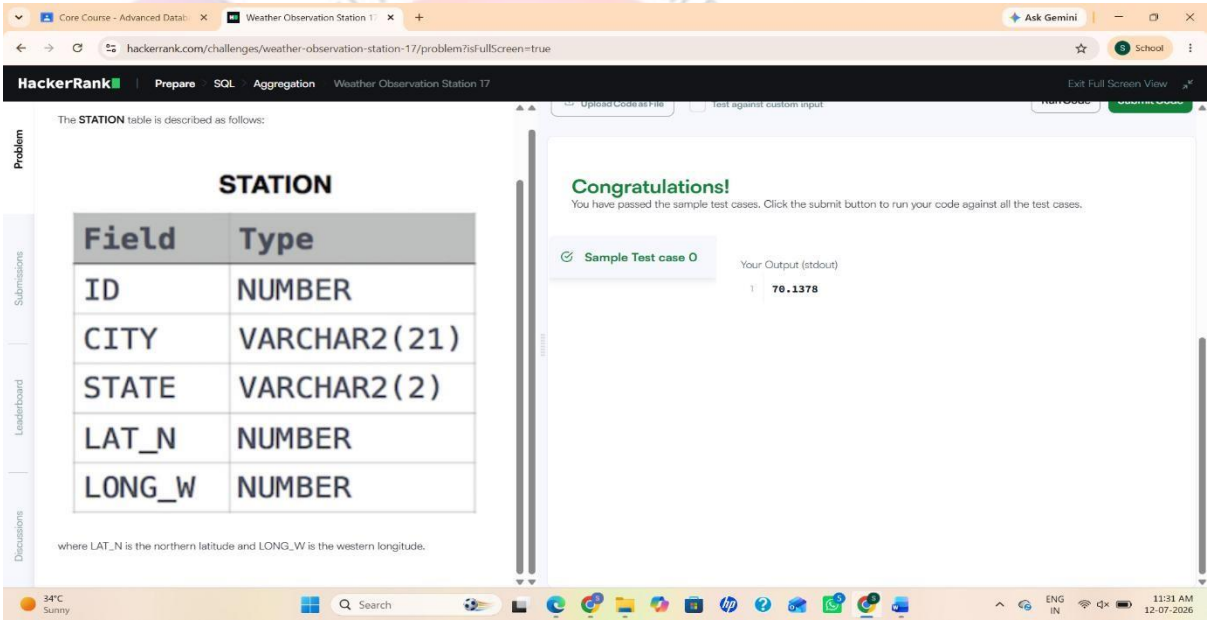


36.WEATHER OBSERVATION STATION 17

PROGRAM

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MIN(LAT_N)
   FROM STATION
   WHERE LAT_N > 38.7780
);
```

OUTPUT

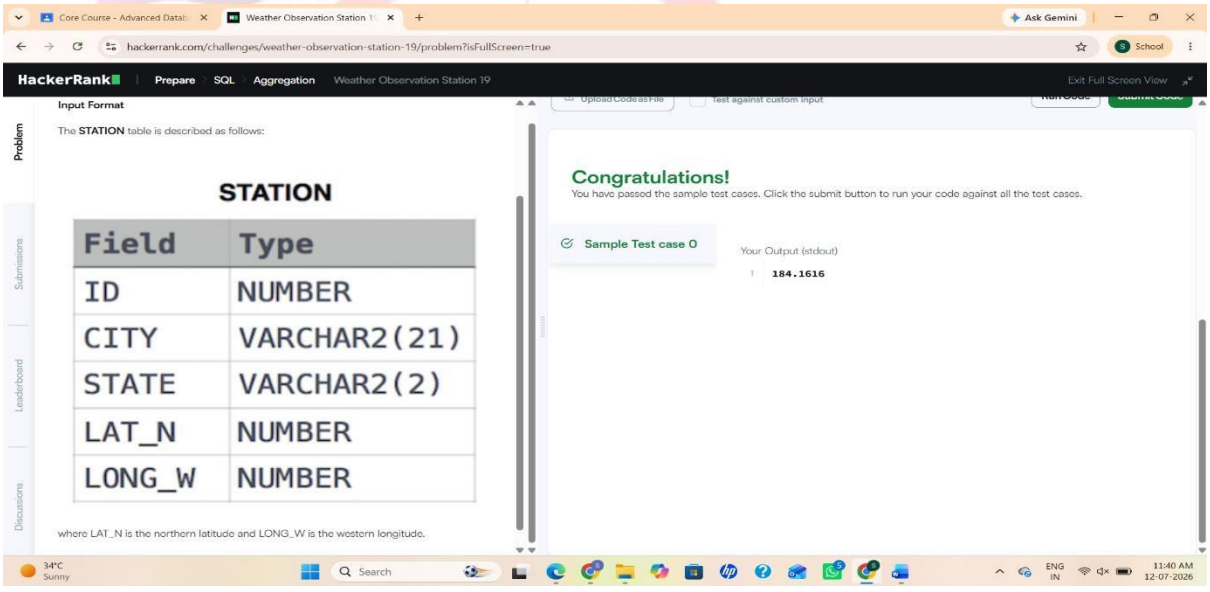


37.WEATHER OBSERVATION STATION 19

PROGRAM

```
SELECT ROUND(
   SQRT(
      POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
      POWER(MAX(LONG_W) - MIN(LONG_W), 2)
   ),
   4
)
FROM STATION;
```

OUTPUT

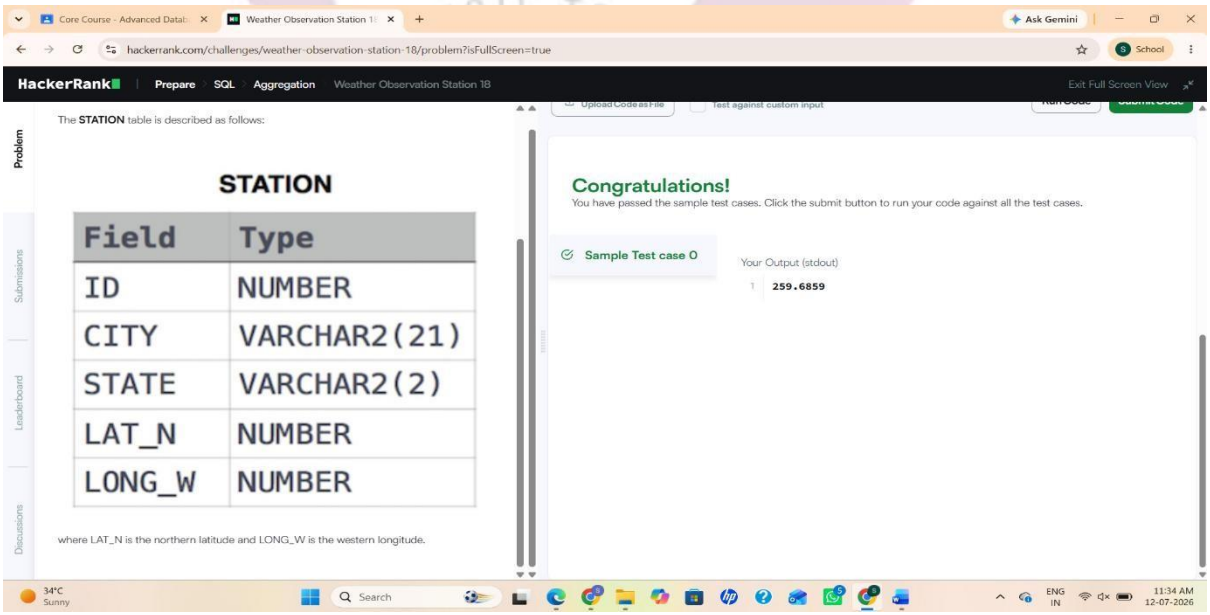


38.WEATHER OBSERVATION STATION 18

PROGRAM

```
SELECT ROUND(
    ABS(MAX(LAT_N) - MIN(LAT_N)) +
    ABS(MAX(LONG_W) - MIN(LONG_W)),
    4
)
FROM STATION;
```

OUTPUT



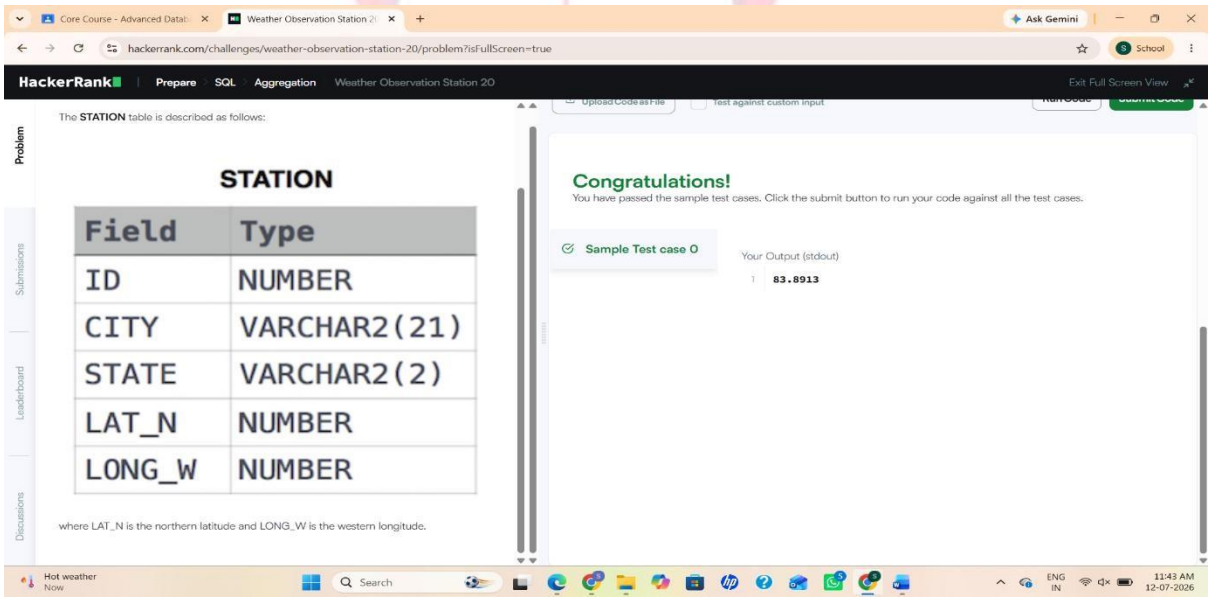
39.WEATHER OBSERVATION STATION 20

PROGRAM

```
SET @rownum := 0;


SELECT ROUND(AVG(LAT_N), 4)
FROM (
    SELECT LAT_N,
        @rownum := @rownum + 1 AS row_num,
        (SELECT COUNT(*) FROM STATION) AS total_rows
    FROM STATION
    ORDER BY LAT_N
) AS t
WHERE row_num IN (FLOOR((total_rows + 1) / 2), FLOOR((total_rows + 2) / 2));
```

OUTPUT

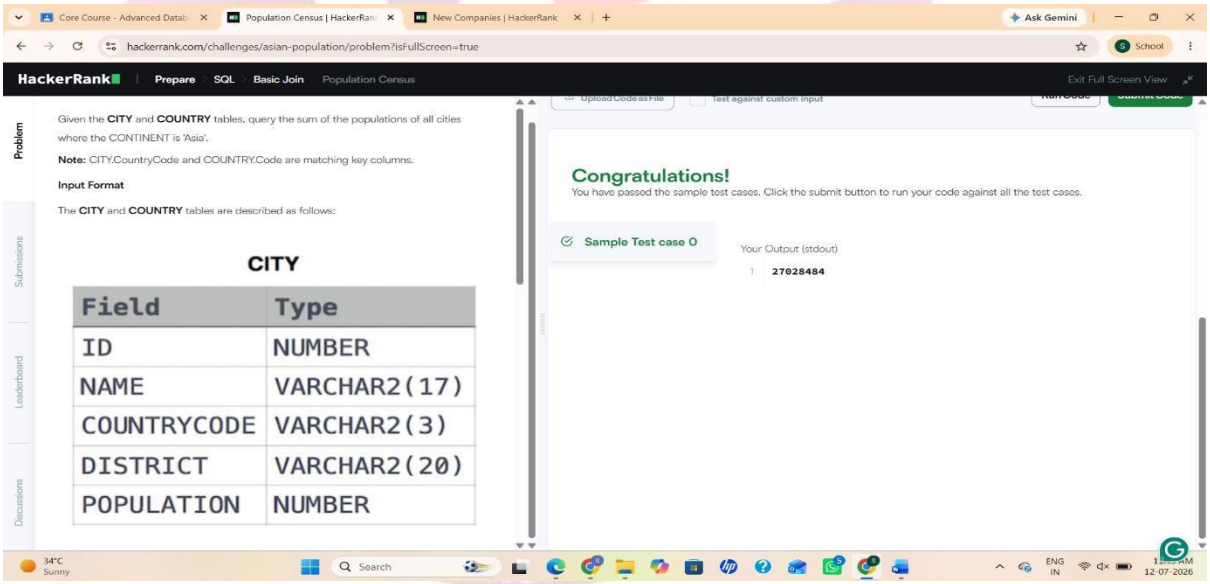


40.POPULATION CENSUS

PROGRAM

```
SELECT SUM(CITY.Population)
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Asia';
```

OUTPUT

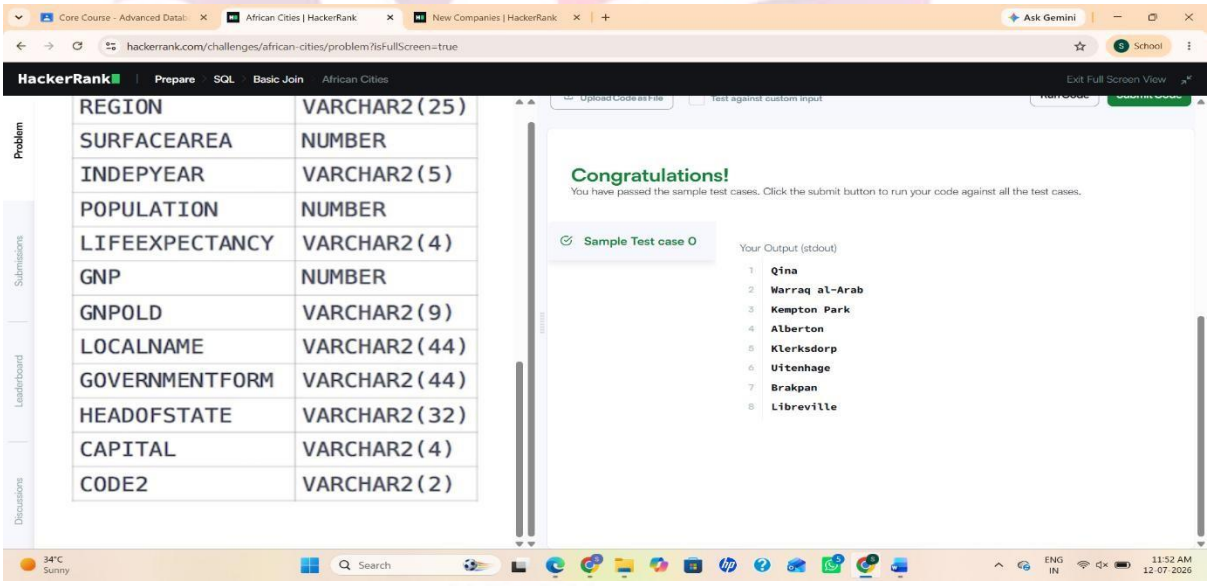


41.AFRICAN CITIES

PROGRAM

```
SELECT CITY.NAME
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Africa';
```

OUTPUT

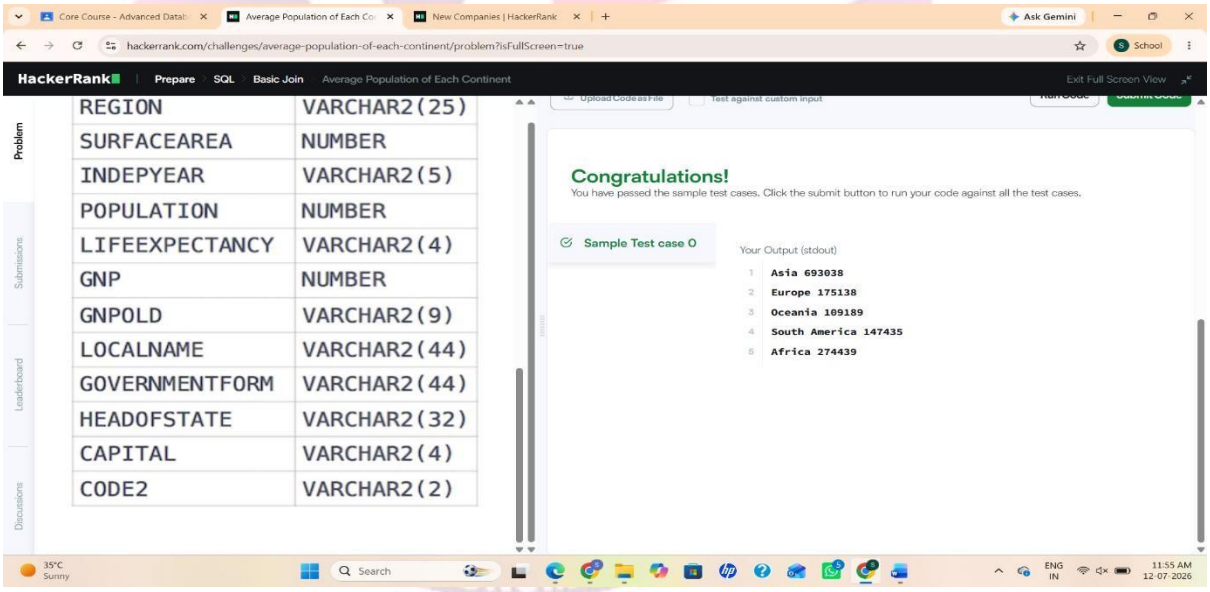


42. AVERAGE POPULATION OF EACH CONTINENT

PROGRAM

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

OUTPUT

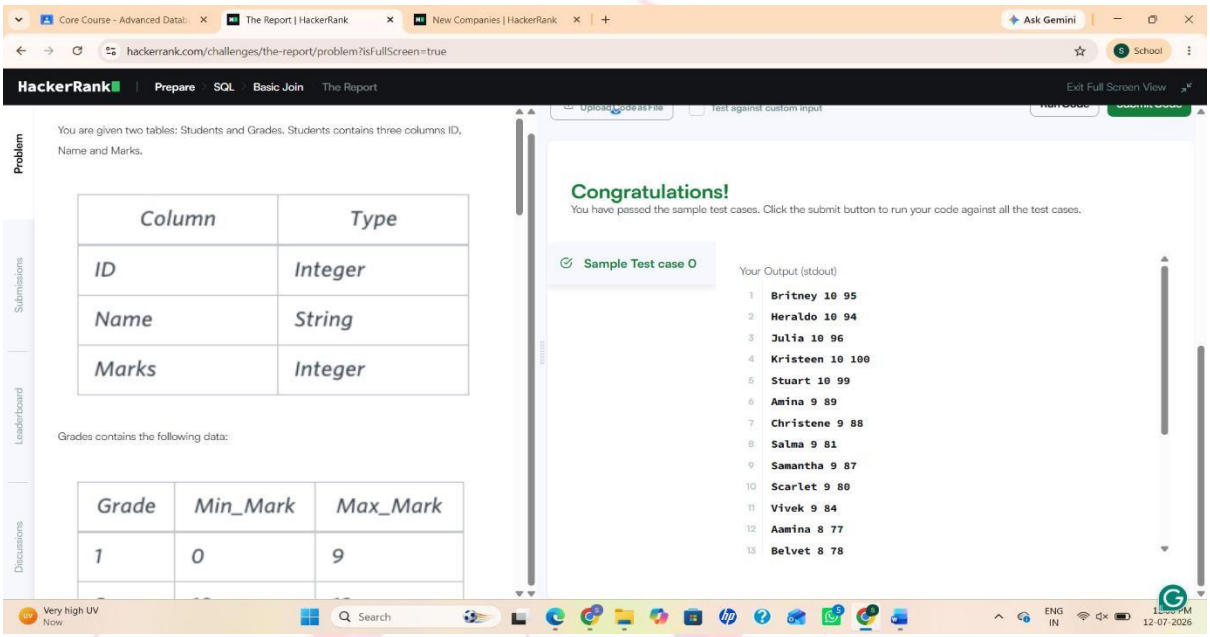


43.THE REPORT

PROGRAM

```
SELECT
   CASE
      WHEN G.Grade < 8 THEN 'NULL'
      ELSE S.Name
   END AS Name,
   G.Grade,
   S.Marks
FROM Students S
JOIN Grades G
ON S.Marks BETWEEN G.Min_Mark AND G.Max_Mark
ORDER BY
   G.Grade DESC,
   CASE
      WHEN G.Grade >= 8 THEN S.Name
   END ASC,
   CASE
      WHEN G.Grade < 8 THEN S.Marks
   END ASC;
```

OUTPUT



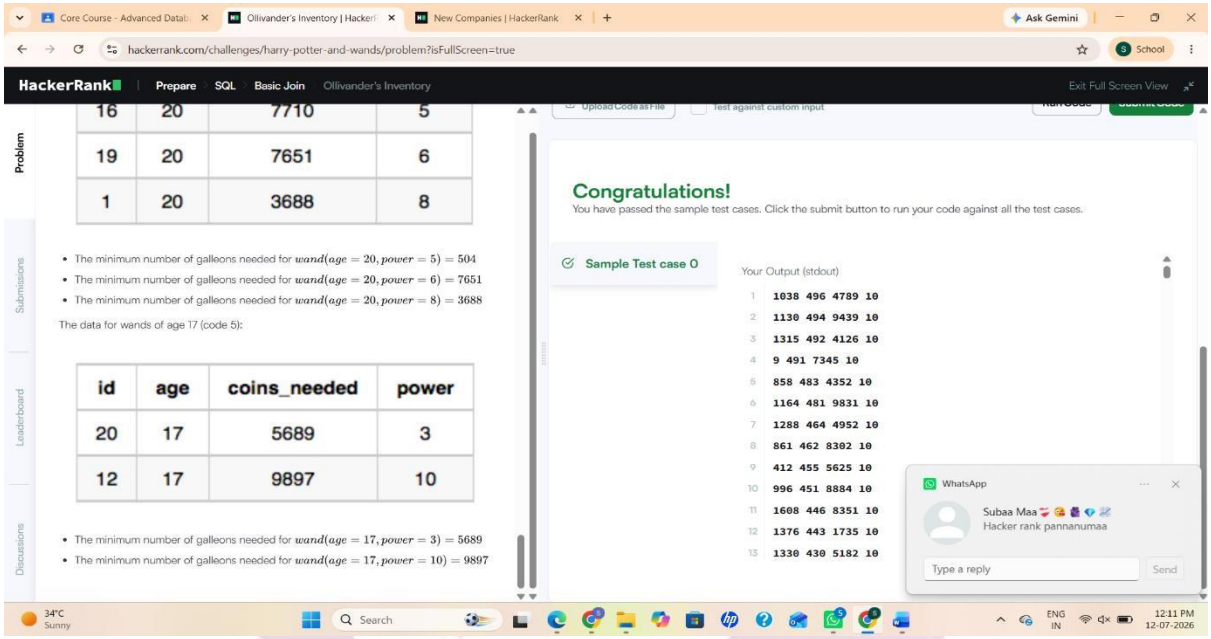
AUTONOMOUS

44. OLLIVANDER'S INVENTORY

PROGRAM

```
SELECT
   w.id,     wp.age,
   w.coins_needed,
   w.power
FROM Wands w
JOIN Wands_Property wp
   ON w.code = wp.code
WHERE wp.is_evil = 0
  AND w.coins_needed = (
     SELECT MIN(w2.coins_needed)
     FROM Wands w2
     JOIN Wands_Property wp2
ON w2.code = wp2.code
     WHERE wp2.is_evil = 0
       AND w2.power = w.power
       AND wp2.age = wp.age
  )
ORDER BY
  w.power DESC,    wp.age
DESC;
```

OUTPUT



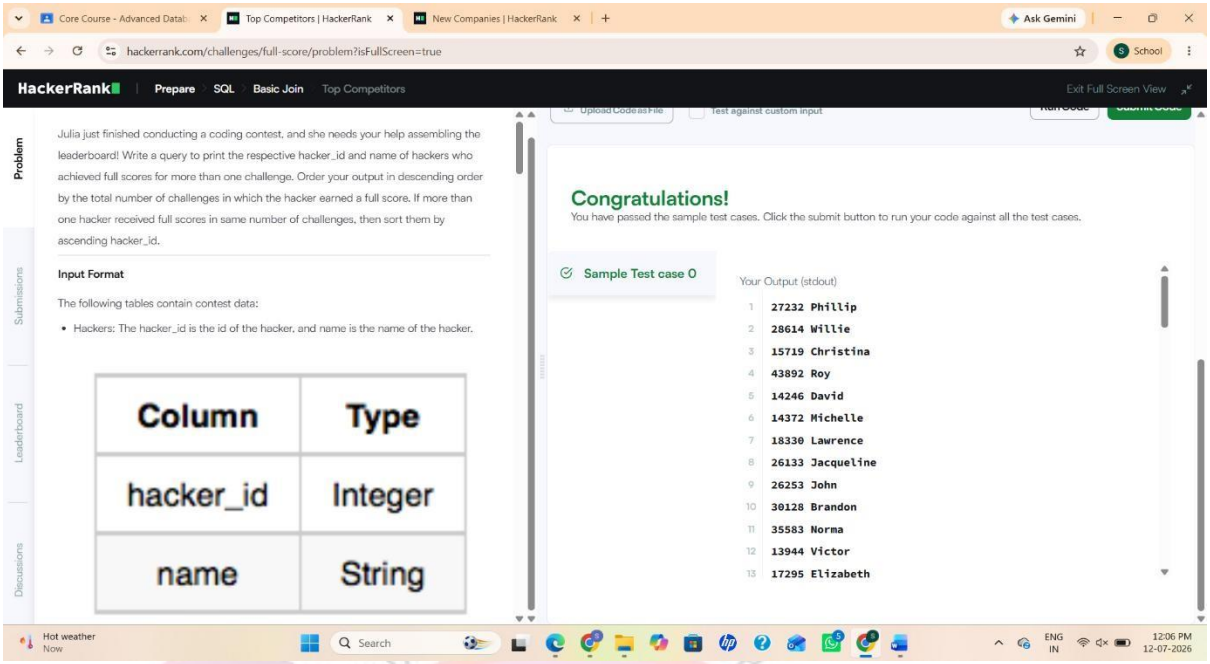
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

45. TOP COMPETITORS

PROGRAM

```
SELECT
   h.hacker_id,
   h.name
FROM Hackers h
JOIN Submissions s
   ON h.hacker_id = s.hacker_id
JOIN Challenges c
   ON s.challenge_id = c.challenge_id
JOIN Difficulty d
   ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1 ORDER BY
COUNT(DISTINCT s.challenge_id) DESC,
      h.hacker_id ASC;
```

OUTPUT



AUTONOMOUS

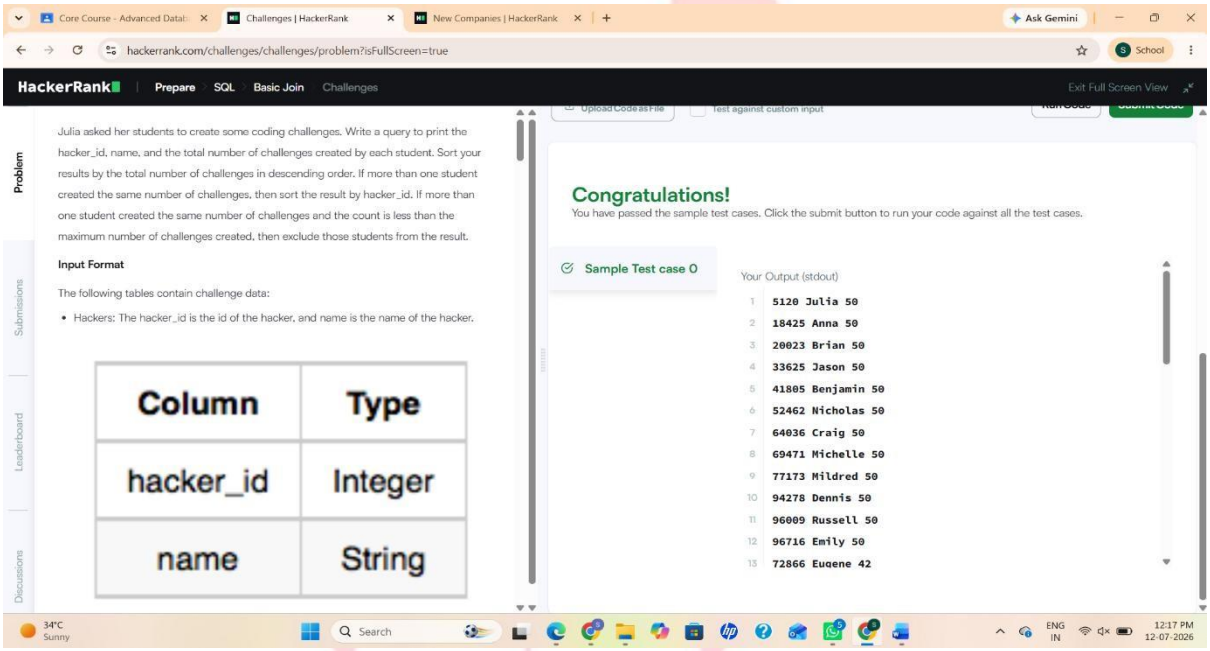
46.CHALLENGES

PROGRAM

```
SELECT h.hacker_id,
    h.name,
    COUNT(c.challenge_id) AS total_challenges
FROM Hackers h
JOIN Challenges c
ON h.hacker_id = c.hacker_id
GROUP BY h.hacker_id, h.name
HAVING COUNT(c.challenge_id) = (
        SELECT MAX(cnt)
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
            GROUP BY hacker_id
        ) t
    )
    OR COUNT(c.challenge_id) IN (
        SELECT cnt
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
           GROUP BY hacker_id
        ) t
        GROUP BY cnt
```

```
    HAVING COUNT(*) = 1
)
ORDER BY total_challenges DESC,
    h.hacker_id ASC;
```

OUTPUT

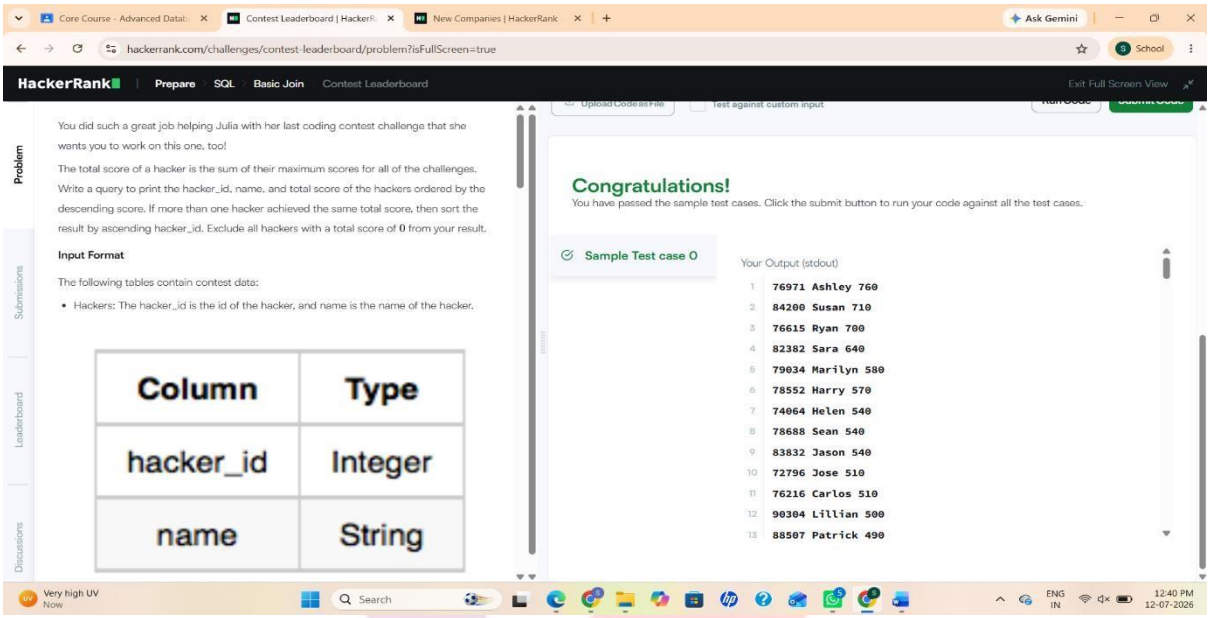


47.CONTEST LEADERBOARD

PROGRAM

```
SELECT
    h.hacker_id,
    h.name,
    SUM(ms.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT
        hacker_id,        challenge_id,
        MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) ms
ON h.hacker_id = ms.hacker_id
GROUP BY h.hacker_id, h.name
HAVING SUM(ms.max_score) > 0 ORDER BY
total_score DESC,
        h.hacker_id ASC;
```

OUTPUT

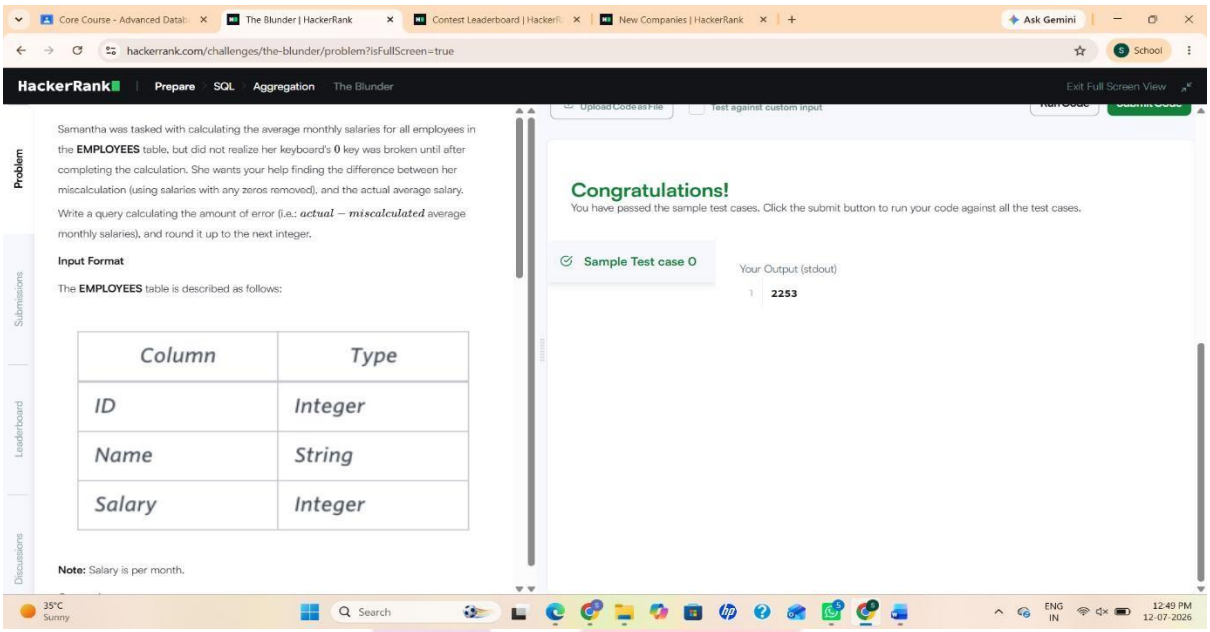


48.THE BLUNDER

PROGRAM

```
SELECT CEIL(
    AVG(Salary) - AVG(REPLACE(Salary, '0', ''))
)
FROM EMPLOYEES;
```

OUTPUT



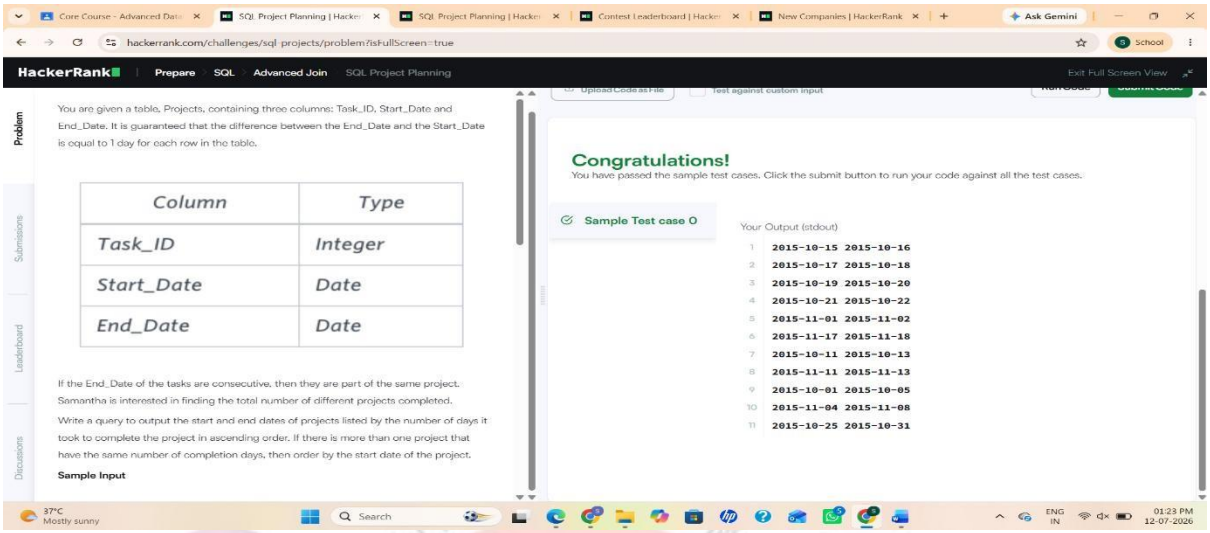
49.SQL PROJECT PLANNING

PROGRAM

```
SELECT
    MIN(Start_Date) AS Start_Date,
    MAX(End_Date) AS End_Date
FROM (
    SELECT
        Start_Date,
        End_Date,
        DATE_SUB(Start_Date, INTERVAL ROW_NUMBER() OVER (ORDER BY
Start_Date) DAY) AS grp
    FROM Projects
) p
```

```
GROUP BY grp
ORDER BY DATEDIFF(MAX(End_Date), MIN(Start_Date)),
        MIN(Start_Date);
```

OUTPUT



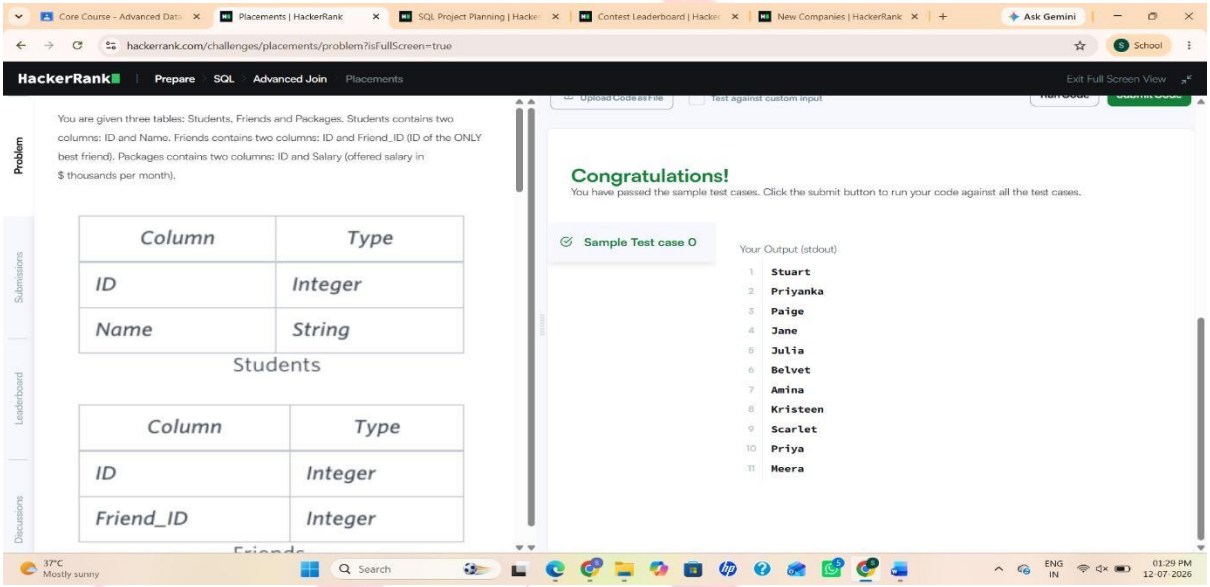
50.PLACEMENTS

PROGRAM

```
SELECT s.Name
FROM Students s
JOIN Friends f
    ON s.ID = f.ID
JOIN Packages p1
    ON s.ID = p1.ID
JOIN Packages p2
```

```
    ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```

OUTPUT



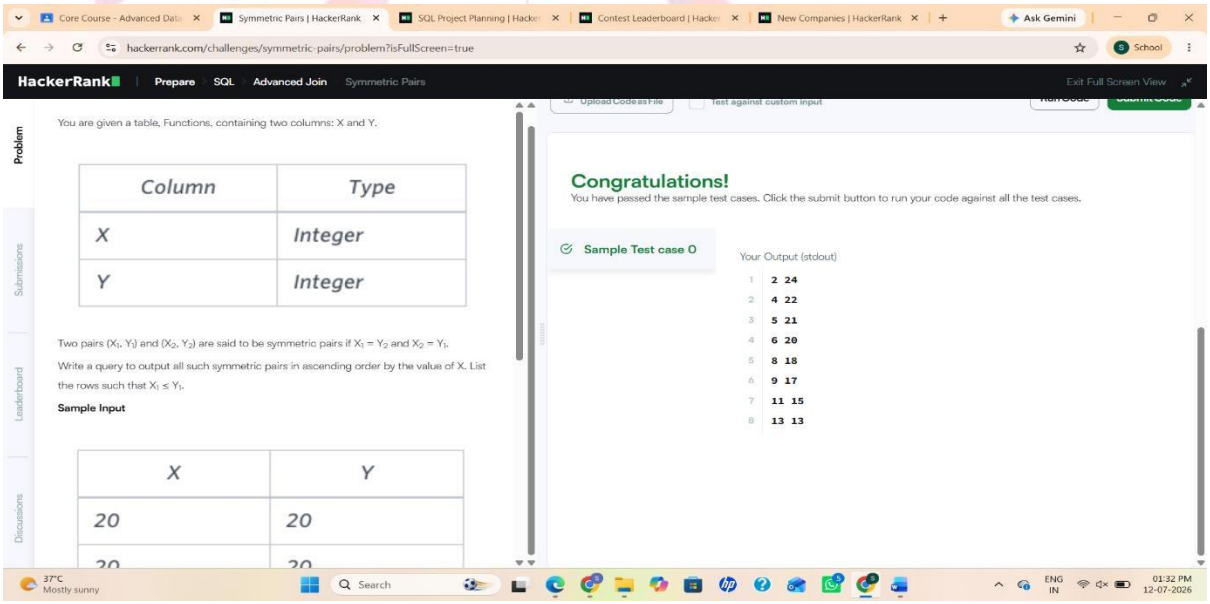
51.SYMMETRIC PAIRS

PROGRAM

```
SELECT DISTINCT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2
ON f1.X = f2.Y
AND f1.Y = f2.X
```

```
WHERE f1.X < f1.Y
  OR (       f1.X =
f1.Y
      AND (
          SELECT COUNT(*)
FROM Functions f3
          WHERE f3.X = f1.X
            AND f3.Y = f1.Y
      ) > 1
  )
ORDER BY f1.X, f1.Y;
```

OUTPUT



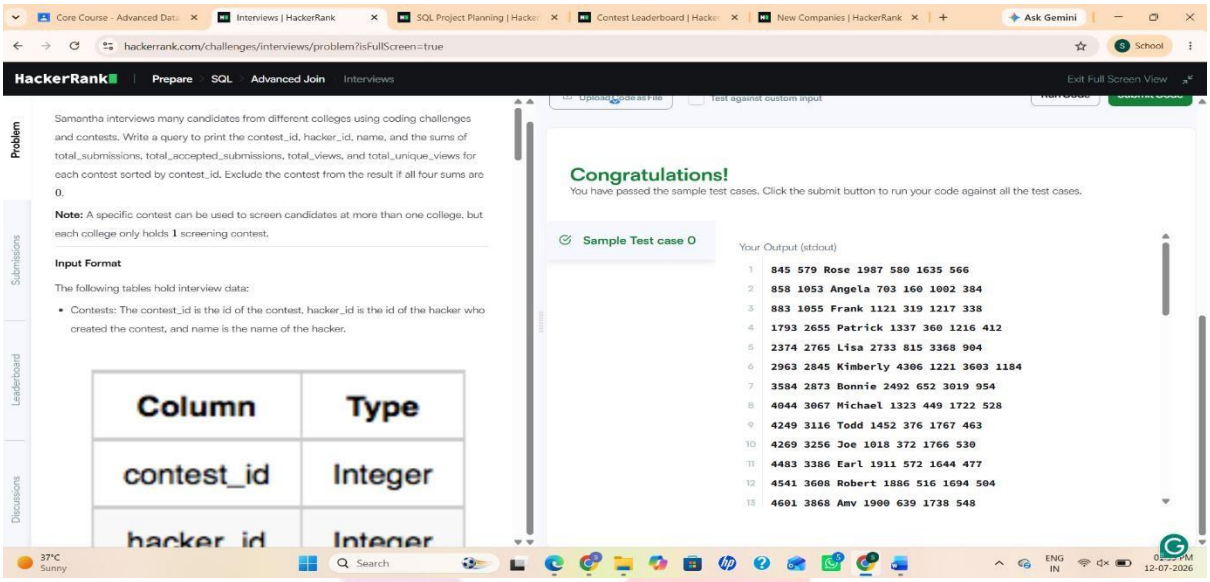
52.INTERVIEWS

PROGRAM

```
SELECT
    c.contest_id,
    c.hacker_id,
    c.name,
    SUM(IFNULL(s.total_submissions, 0)) AS total_submissions,
    SUM(IFNULL(s.total_accepted_submissions, 0)) AS total_accepted_submissions,
    SUM(IFNULL(v.total_views, 0)) AS total_views,
    SUM(IFNULL(v.total_unique_views, 0)) AS total_unique_views
FROM Contests c
JOIN Colleges co
    ON c.contest_id = co.contest_id
JOIN Challenges ch
    ON co.college_id = ch.college_id
LEFT JOIN (     SELECT
        challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) s
ON ch.challenge_id = s.challenge_id
LEFT JOIN (    SELECT
```

```
    challenge_id,
    SUM(total_views) AS total_views,
     SUM(total_unique_views) AS total_unique_views
  FROM View_Stats
  GROUP BY challenge_id
) v
ON ch.challenge_id = v.challenge_id
GROUP BY
  c.contest_id,
  c.hacker_id,
  c.name
HAVING
  total_submissions +
total_accepted_submissions +
total_views +    total_unique_views > 0
ORDER BY
  c.contest_id;
```

OUTPUT



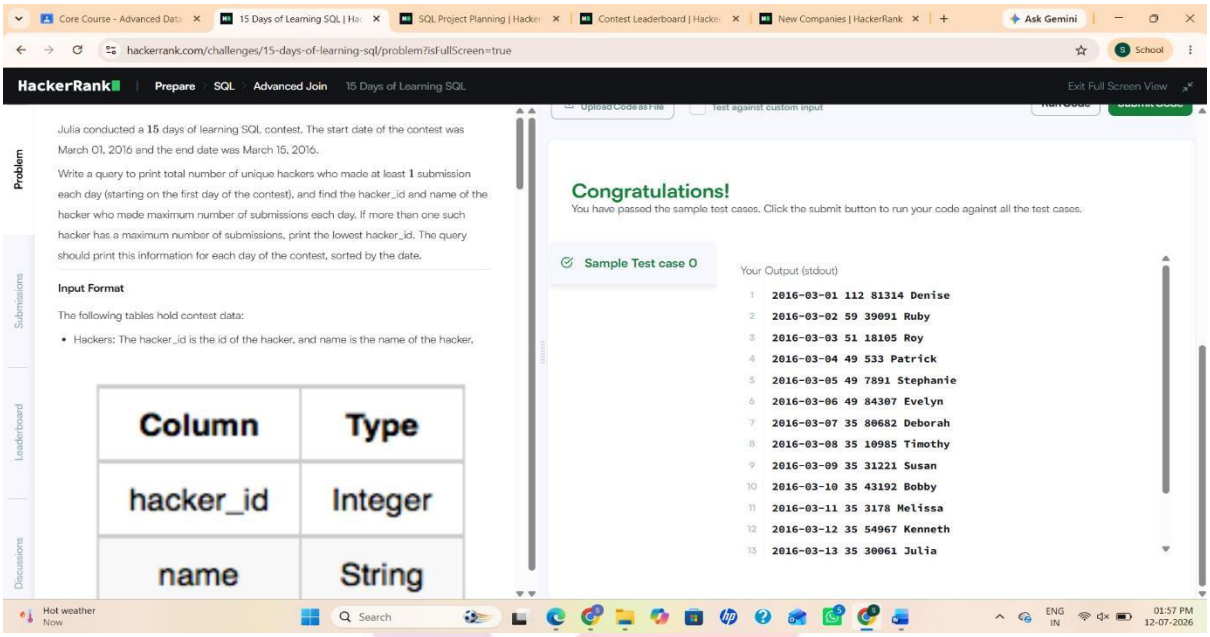
53. 15 DAYS OF LEARNING SQL

PROGRAM

```
SELECT
   d.submission_date,
   (
      SELECT COUNT(*)
      FROM (
         SELECT hacker_id
         FROM Submissions s
         WHERE s.submission_date <= d.submission_date
         GROUP BY hacker_id
         HAVING COUNT(DISTINCT s.submission_date)
               = DATEDIFF(d.submission_date,'2016-03-01') + 1
      ) x
```

```
    ) AS total_hackers,
    h.hacker_id,
    h.name
FROM
(
    SELECT submission_date,
        hacker_id,
        ROW_NUMBER() OVER
        (
            PARTITION BY submission_date
            ORDER BY COUNT(*) DESC, hacker_id
        ) rn
    FROM Submissions
    GROUP BY submission_date,hacker_id
) d
JOIN Hackers h
ON d.hacker_id = h.hacker_id
WHERE rn = 1
ORDER BY submission_date;
```

OUTPUT



54.PRINT PRIME NUMBERS

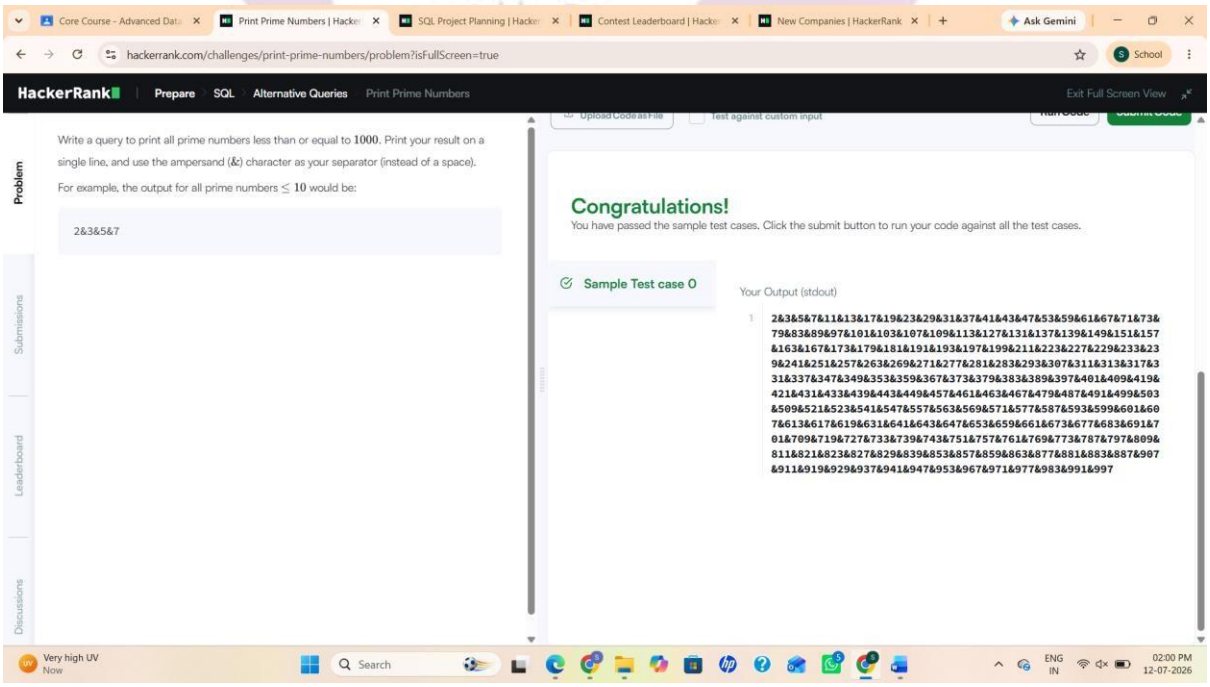
PROGRAM

```
SET @num = 1;


SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM (
   SELECT @num := @num + 1 AS n
   FROM information_schema.tables t1,        information_schema.tables t2
   LIMIT 998
) numbers
WHERE NOT EXISTS (
   SELECT 1
   FROM (
```

```
    SELECT @div := @div + 1 AS d
    FROM information_schema.tables,
        (SELECT @div := 1) init
    LIMIT 998
) divisors
WHERE d < n
  AND d > 1
  AND MOD(n, d) = 0
);
```

OUTPUT



55. OCCUPATIONS

PROGRAM

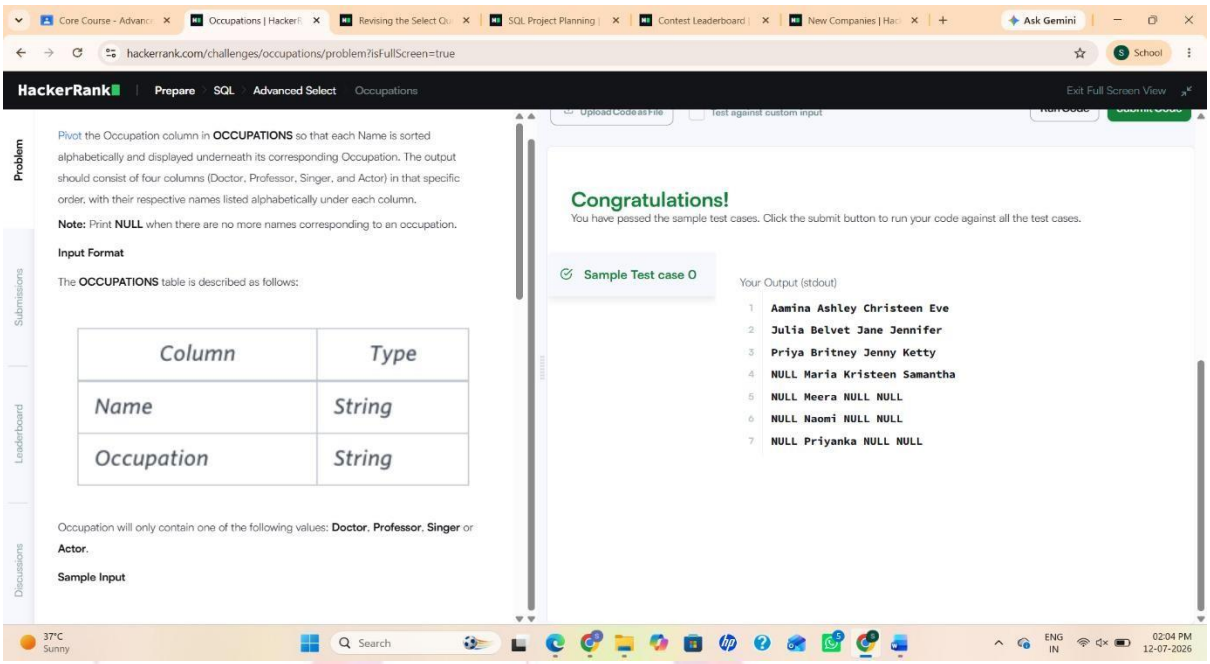
```
SET @d = 0, @p = 0, @s = 0, @a = 0;


SELECT
   MAX(Doctor),
   MAX(Professor),
   MAX(Singer),
   MAX(Actor)
FROM
(
   SELECT
      CASE
         WHEN Occupation = 'Doctor' THEN (@d := @d + 1)
         WHEN Occupation = 'Professor' THEN (@p := @p + 1)
         WHEN Occupation = 'Singer' THEN (@s := @s + 1)
         WHEN Occupation = 'Actor' THEN (@a := @a + 1)
      END AS rn,
      CASE WHEN Occupation = 'Doctor' THEN Name END AS Doctor,
      CASE WHEN Occupation = 'Professor' THEN Name END AS Professor,
      CASE WHEN Occupation = 'Singer' THEN Name END AS Singer,
      CASE WHEN Occupation = 'Actor' THEN Name END AS Actor
   FROM OCCUPATIONS
   ORDER BY Occupation, Name
) t
```

GROUP BY rn

ORDER BY rn;

OUTPUT

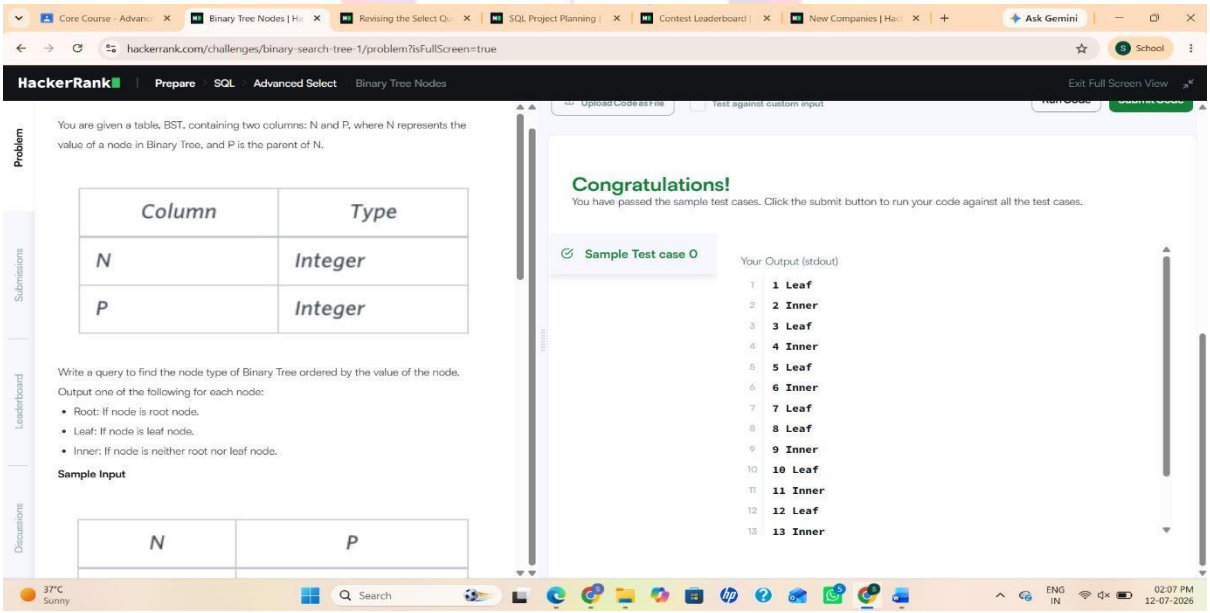


56.BINARY TREE NODES

PROGRAM

```
SELECT
   N,
   CASE
      WHEN P IS NULL THEN 'Root'
      WHEN N NOT IN (SELECT DISTINCT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'
      ELSE 'Inner'
   END AS NodeType
FROM BST
ORDER BY N;
```

OUTPUT



KHAN ACADEMY

1. BOOK LIST DATABASE

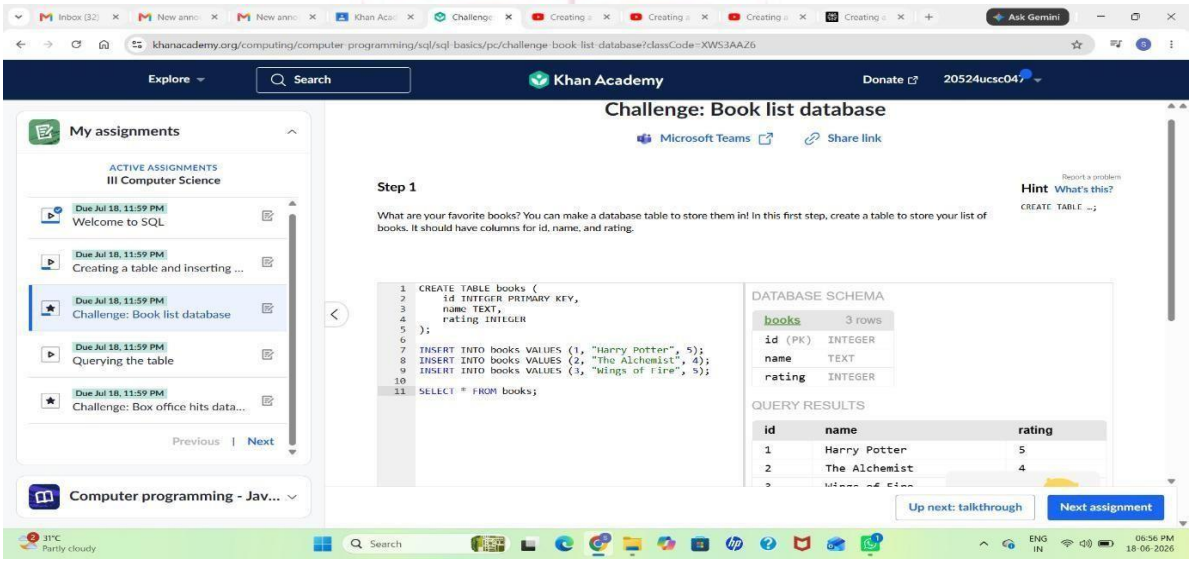
PROGRAM

```
CREATE TABLE books ( id
    INTEGER PRIMARY KEY,
    name TEXT, rating INTEGER
);

INSERT INTO books VALUES (1, "Harry Potter", 5); INSERT INTO
books VALUES (2, "The Alchemist", 4); INSERT INTO books
VALUES (3, "Wings of Fire", 5);


SELECT * FROM books;
```

OUTPUT



2. QUERYING THE TABLE

PROGRAM

```
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle
INTEGER);

INSERT INTO groceries VALUES (1, "Bananas", 4, 7); INSERT

INTO groceries VALUES(2, "Peanut Butter", 1, 2);

INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2); INSERT

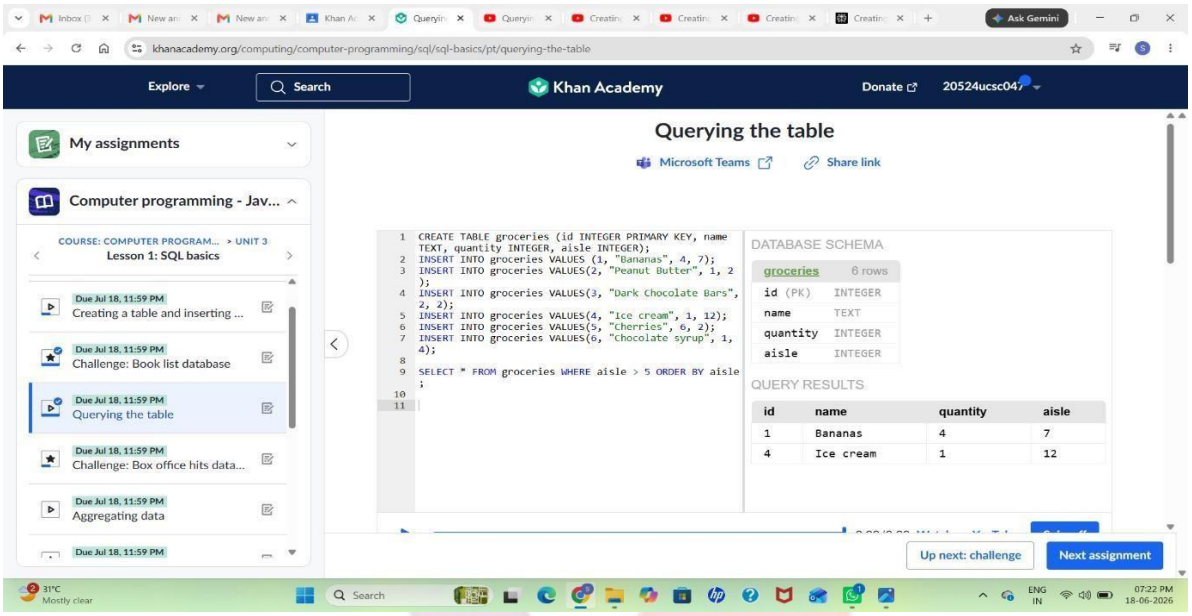
INTO groceries VALUES(4, "Ice cream", 1, 12);

INSERT INTO groceries VALUES(5, "Cherries", 6, 2); INSERT INTO groceries

VALUES(6, "Chocolate syrup", 1, 4);


SELECT * FROM groceries WHERE aisle > 5 ORDER BY aisle;
```

OUTPUT

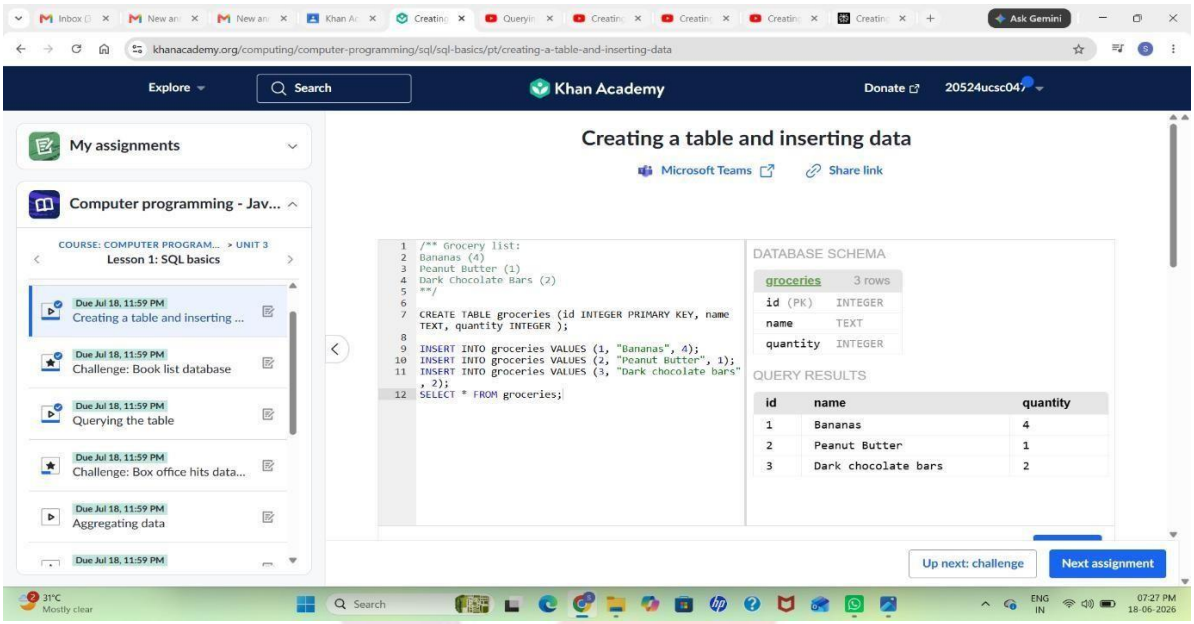


3. CREATING A TABLE AND INSERTING DATA

PROGRAM

```
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER );
INSERT INTO groceries VALUES (1, "Bananas", 4);
INSERT INTO groceries VALUES (2, "Peanut Butter", 1);
INSERT INTO groceries VALUES (3, "Dark chocolate bars",
2);
SELECT * FROM groceries;
```

OUTPUT



4. AGGREGATING DATA

PROGRAM

CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);

INSERT INTO groceries VALUES (1, "Bananas", 56, 7); INSERT

INTO groceries VALUES(2, "Peanut Butter", 1, 2);

INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2); INSERT

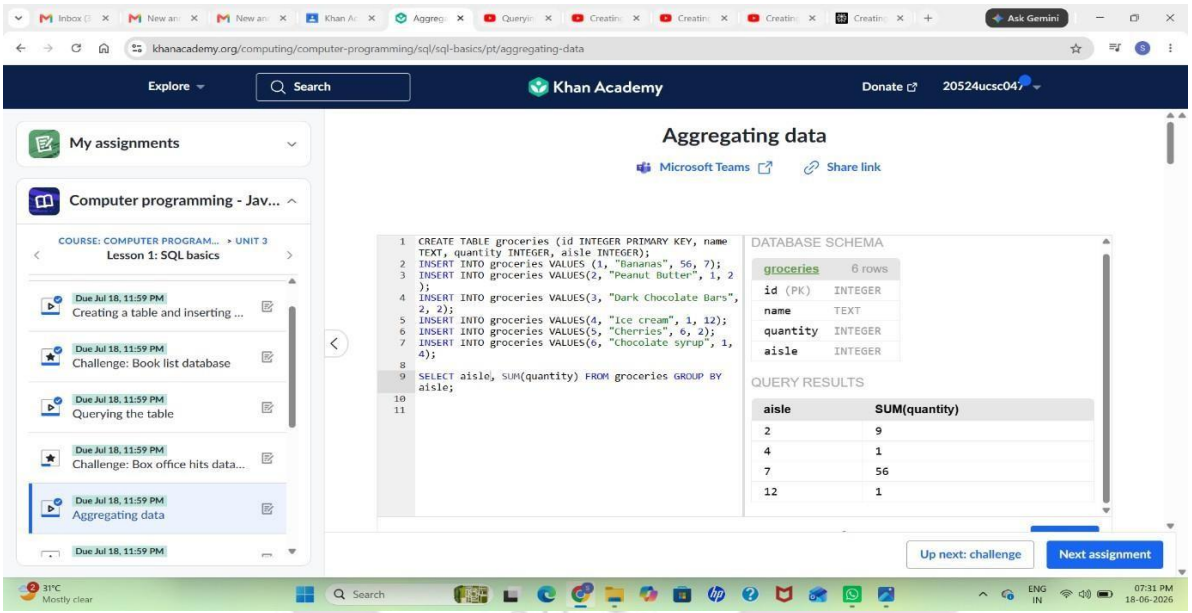
INTO groceries VALUES(4, "Ice cream", 1, 12);

INSERT INTO groceries VALUES(5, "Cherries", 6, 2); INSERT INTO groceries

VALUES(6, "Chocolate syrup", 1, 4);

SELECT aisle, SUM(quantity) FROM groceries GROUP BY aisle;

OUTPUT



5. BOX OFFICE HITS DATABASE

PROGRAM

CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009); INSERT

INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977); INSERT INTO movies

VALUES (4, "Shrek 2", 2004);

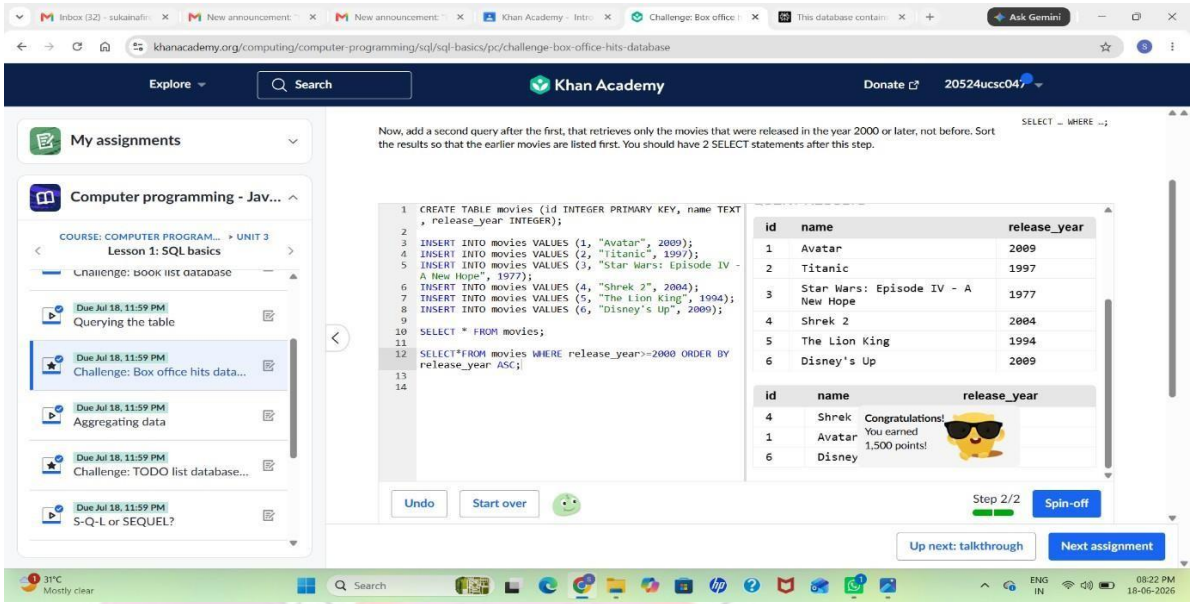
INSERT INTO movies VALUES (5, "The Lion King", 1994); INSERT

INTO movies VALUES (6, "Disney's Up", 2009);

SELECT * FROM movies;

SELECT*FROM movies WHERE release_year>=2000 ORDER BY release_year ASC;

OUTPUT



6. TODO LIST DATABASE STATS

PROGRAM

CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER); INSERT

INTO todo_list VALUES (1, "Wash the dishes", 15);

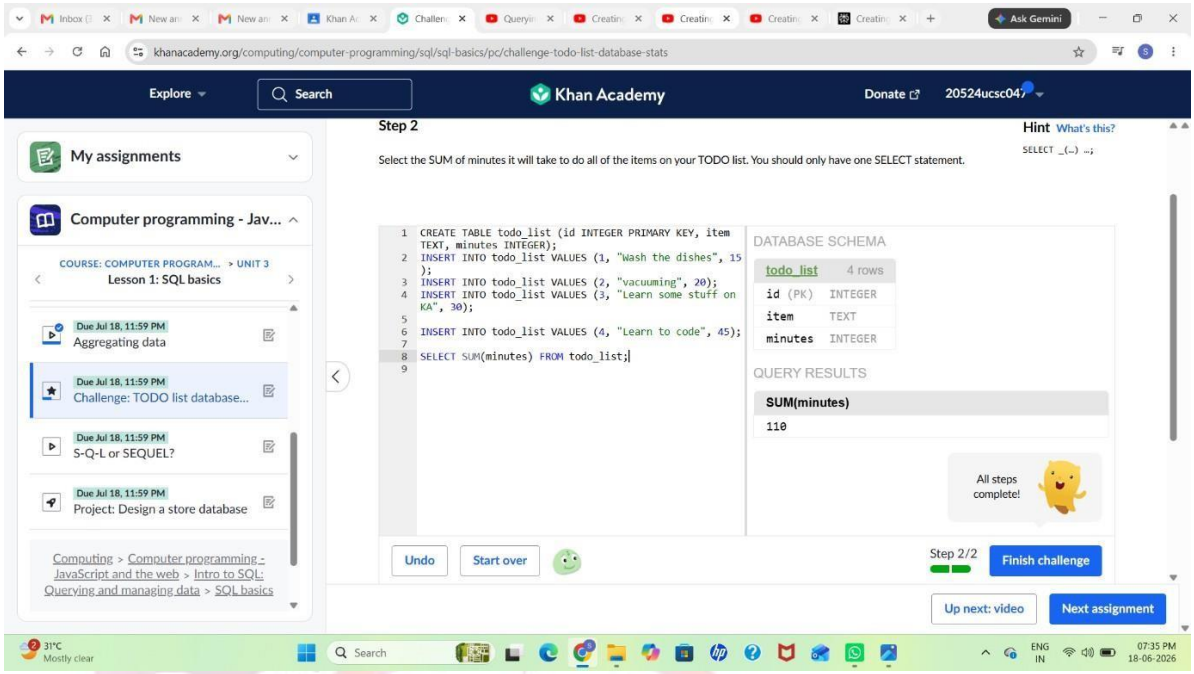
INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Learn to code", 45);

SELECT SUM(minutes) FROM todo_list;

OUTPUT



7. PROJECT: DESIGN A STORE DATABASE

PROGRAM

```
CREATE TABLE books ( id
    INTEGER PRIMARY KEY,
    name TEXT, author TEXT,
    genre TEXT, price REAL,
    stock
    INTEGER
);
```

INSERT INTO books VALUES (1, "Harry Potter 1", "J.K. Rowling", "Fantasy", 12.99, 50); INSERT INTO books VALUES (2, "Harry Potter 2", "J.K. Rowling", "Fantasy", 13.99, 45);

INSERT INTO books VALUES (3, "Harry Potter 3", "J.K. Rowling", "Fantasy", 14.99, 40);

INSERT INTO books VALUES (4, "The Hobbit", "J.R.R. Tolkien", "Fantasy", 10.99, 30);

INSERT INTO books VALUES (5, "Lord of the Rings", "J.R.R. Tolkien", "Fantasy", 19.99, 25);

INSERT INTO books VALUES (6, "The Alchemist", "Paulo Coelho", "Fiction", 9.99, 60); INSERT INTO books VALUES (7, "Atomic Habits", "James Clear", "Self-Help", 15.99, 55);

INSERT INTO books VALUES (8, "Rich Dad Poor Dad", "Robert Kiyosaki", "Finance", 11.99, 35);

INSERT INTO books VALUES (9, "Think and Grow Rich", "Napoleon Hill", "Finance", 8.99, 40);

INSERT INTO books VALUES (10, "Deep Work", "Cal Newport", "Self-Help", 13.99, 20);

INSERT INTO books VALUES (11, "1984", "George Orwell", "Dystopian", 7.99, 70);

INSERT INTO books VALUES (12, "Animal Farm", "George Orwell", "Dystopian", 6.99, 65);

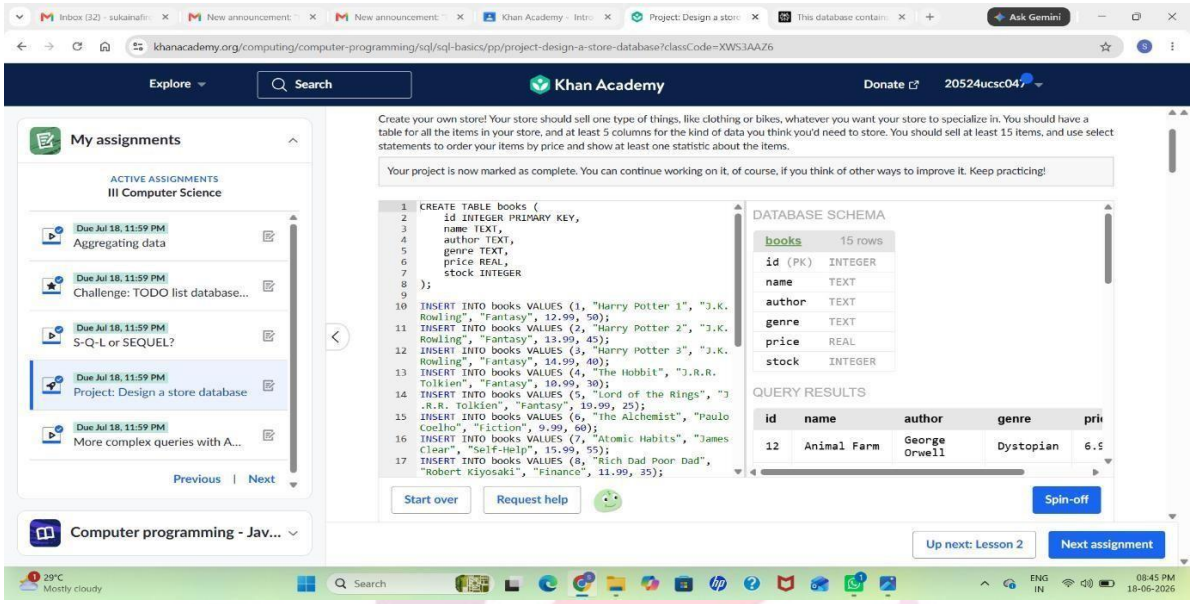
INSERT INTO books VALUES (13, "The Great Gatsby", "F. Scott Fitzgerald", "Classic", 8.99, 30);
INSERT INTO books VALUES (14, "To Kill a Mockingbird", "Harper Lee", "Classic", 9.99, 25);

INSERT INTO books VALUES (15, "The Diary of a Young Girl", "Anne Frank", "Biography", 10.99, 50);

SELECT * FROM books ORDER BY price ASC;

SELECT MAX(price) FROM books; SELECT MIN(price) FROM books; SELECT AVG(price) FROM books;

OUTPUT



8. MORE COMPLEX QUERIES WITH AND/OR

PROGRAM

```
CREATE TABLE exercise_logs
    (id INTEGER PRIMARY KEY AUTOINCREMENT, type TEXT,
    minutes
    INTEGER, calories INTEGER,
    heart_rate INTEGER);
```

```
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);


SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;

/* AND */

SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;

/* OR */

SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
```

OUTPUT

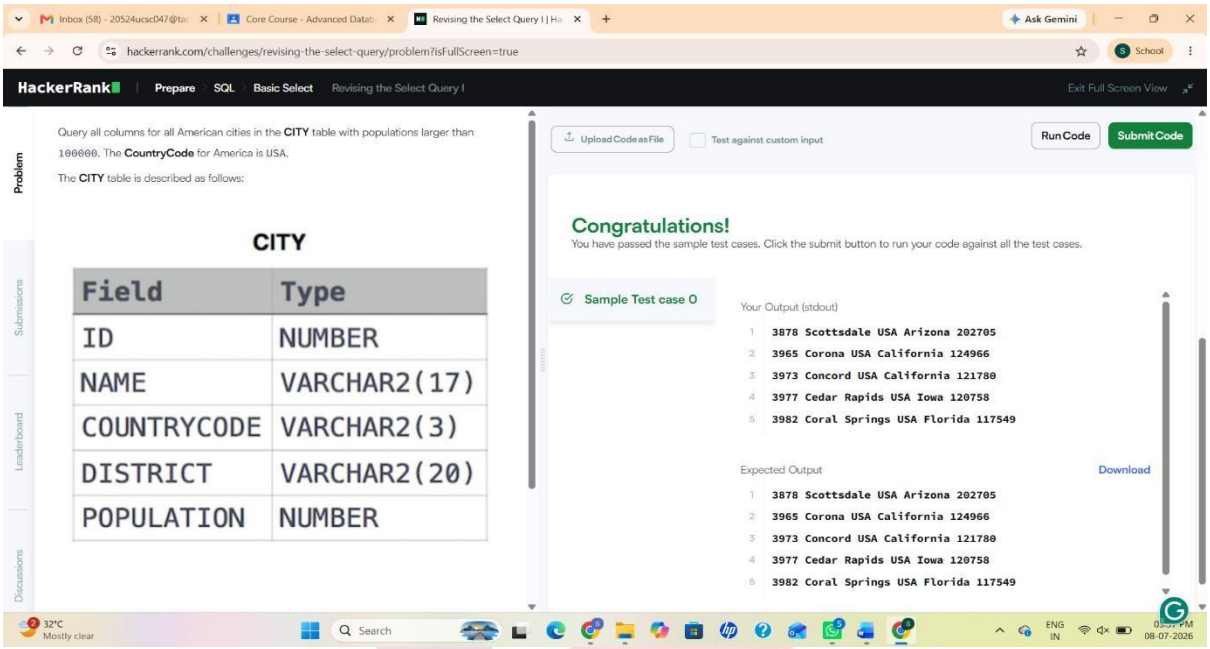
HACKRANK

1.REVISING THE SELECT QUERY I

PROGRAM

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION >
100000;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

2.SELECT BY ID

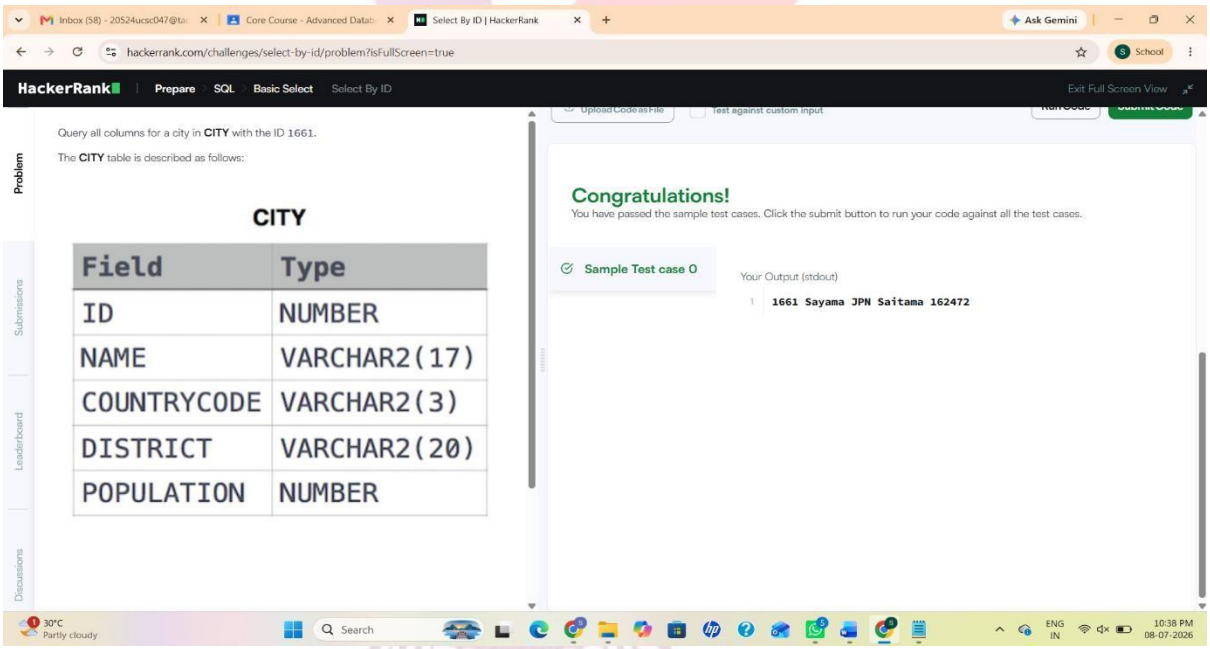
PROGRAM

```
SELECT t.*

FROM CITY t

WHERE t.ID=1661;
```

OUTPUT

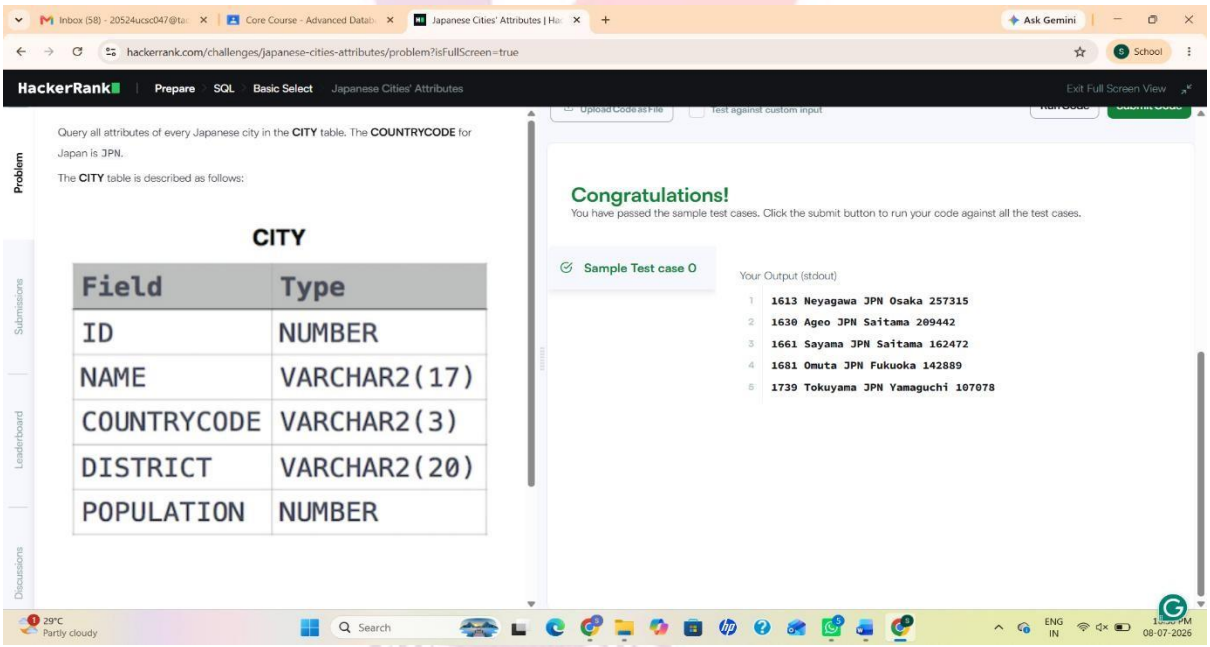


3.JAPANESE CITIES' ATTRIBUTES

PROGRAM

```
SELECT t.*
FROM CITY t
WHERE t.COUNTRYCODE = 'JPN';
```

OUTPUT



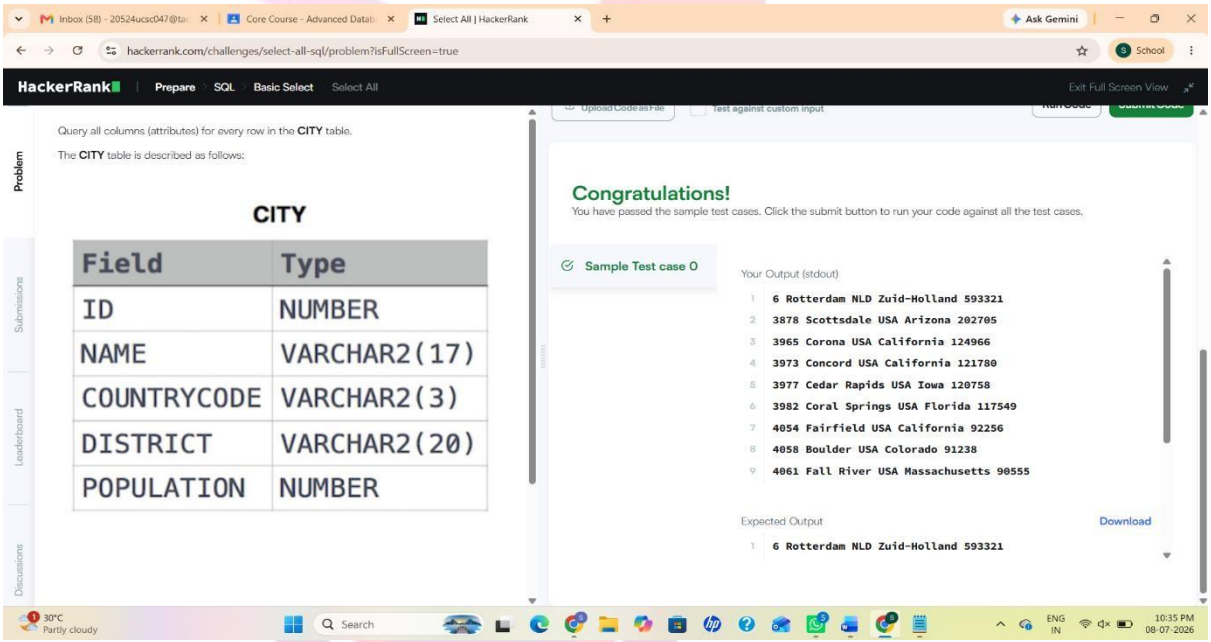
4.SELECT ALL

PROGRAM

```
SELECT t.*

FROM CITY t;
```

OUTPUT



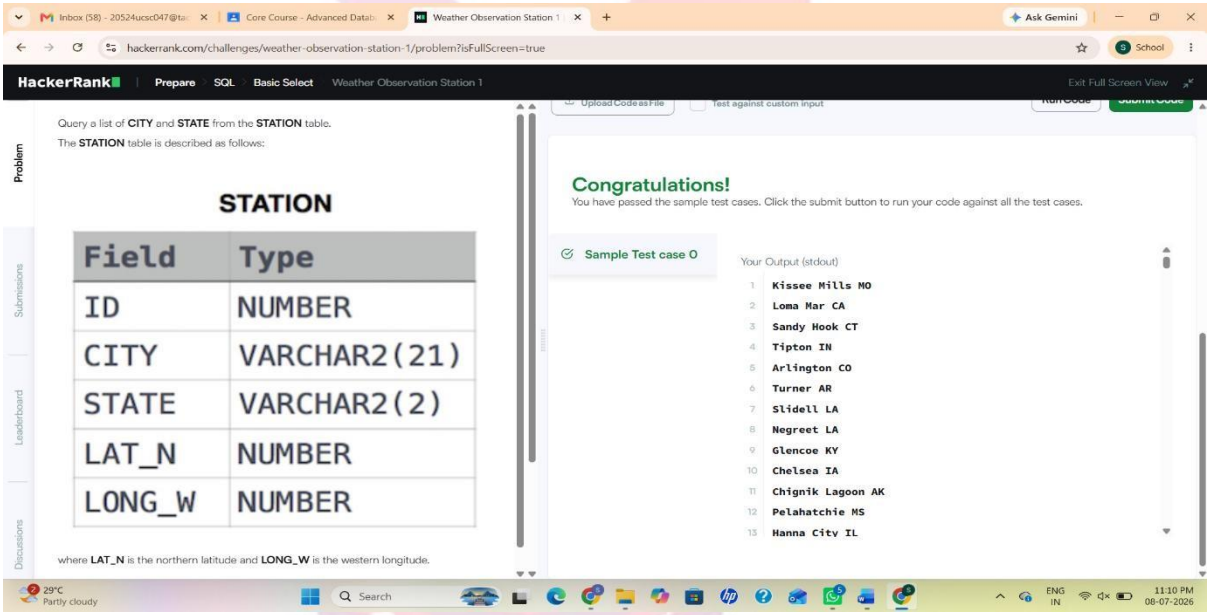
5. WEATHER OBSERVATION STATION 1

PROGRAM

```
SELECT t.CITY,t.STATE

FROM STATION t;
```

OUTPUT

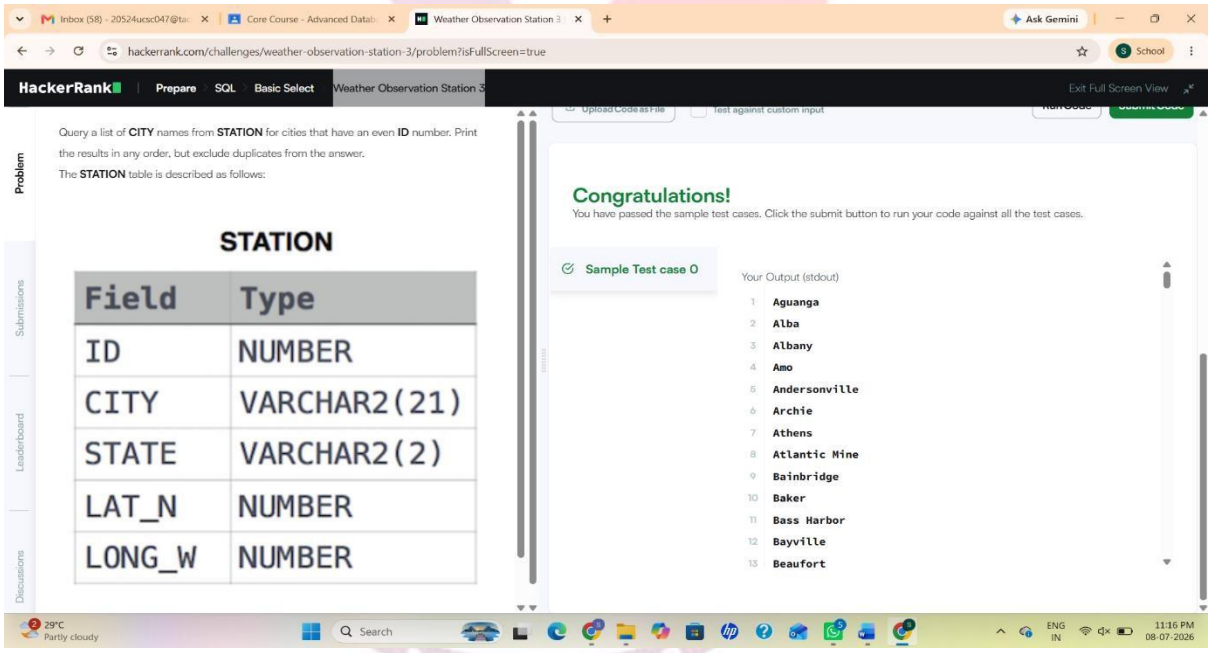


6. WEATHER OBSERVATION STATION 3

PROGRAM

```
SELECT DISTINCT t.CITY
FROM STATION t
WHERE MOD(t.ID,2) = 0;
```

OUTPUT



7.JAPANESE CITIES' NAMES

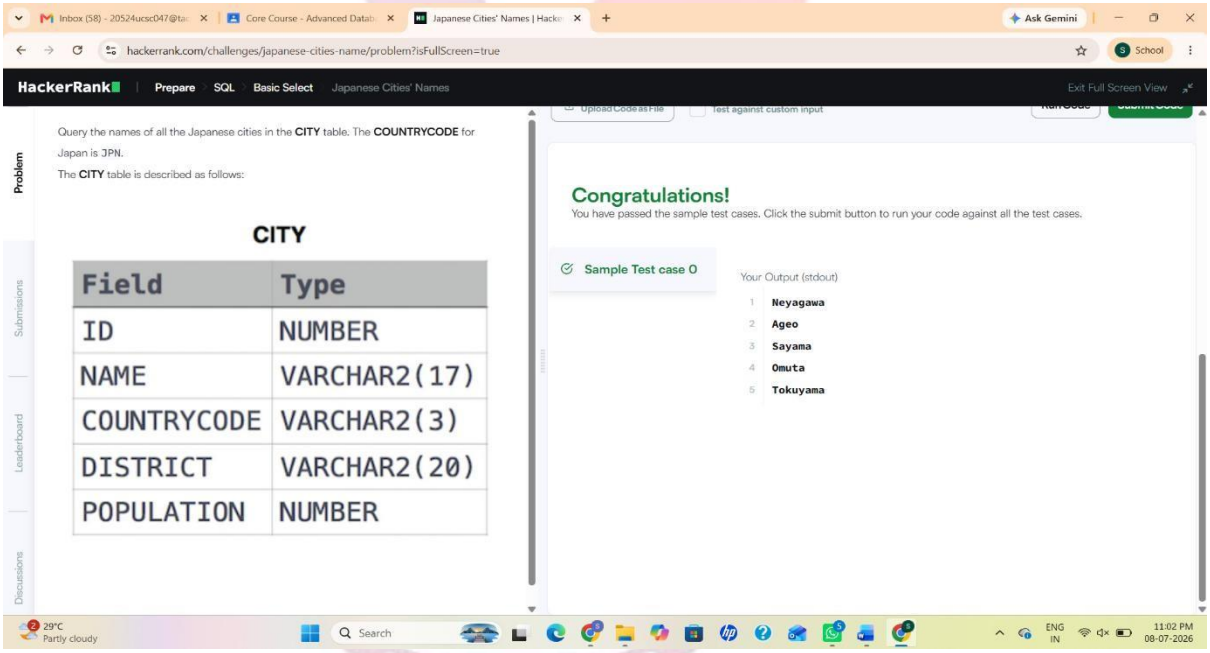
PROGRAM

```
SELECT t.NAME

FROM CITY t

WHERE t.COUNTRYCODE = 'JPN';
```

OUTPUT



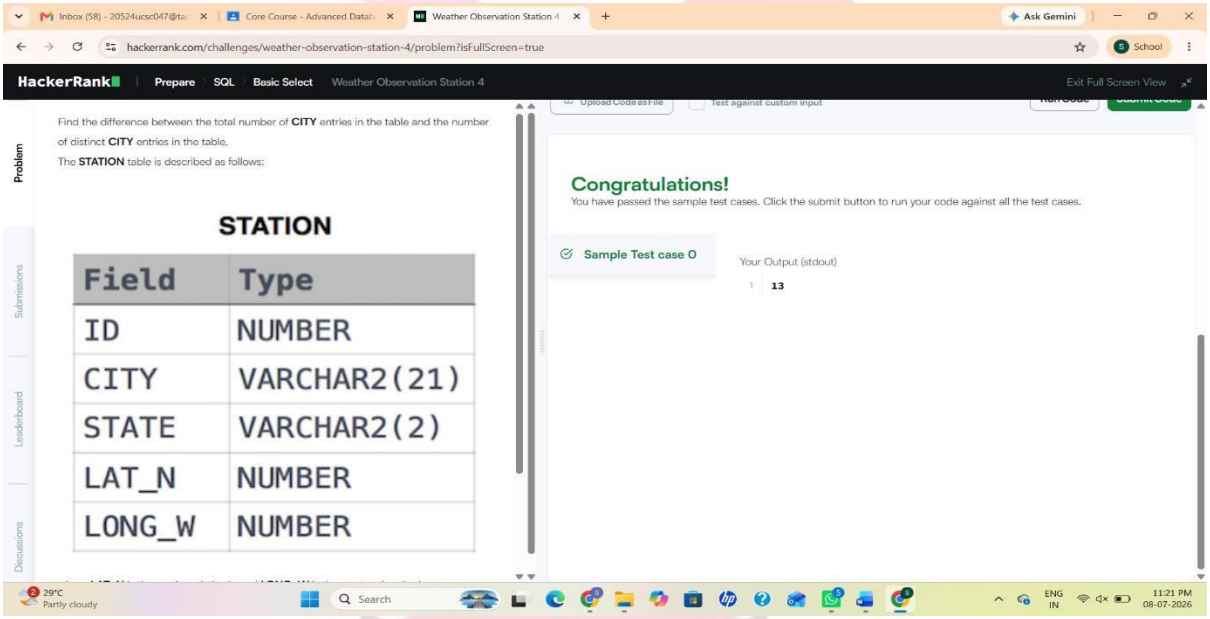
8.WEATHER OBSERVATION STATION 4

PROGRAM

```
SELECT COUNT(t.CITY) - COUNT(DISTINCT t.CITY)

FROM STATION t;
```

OUTPUT



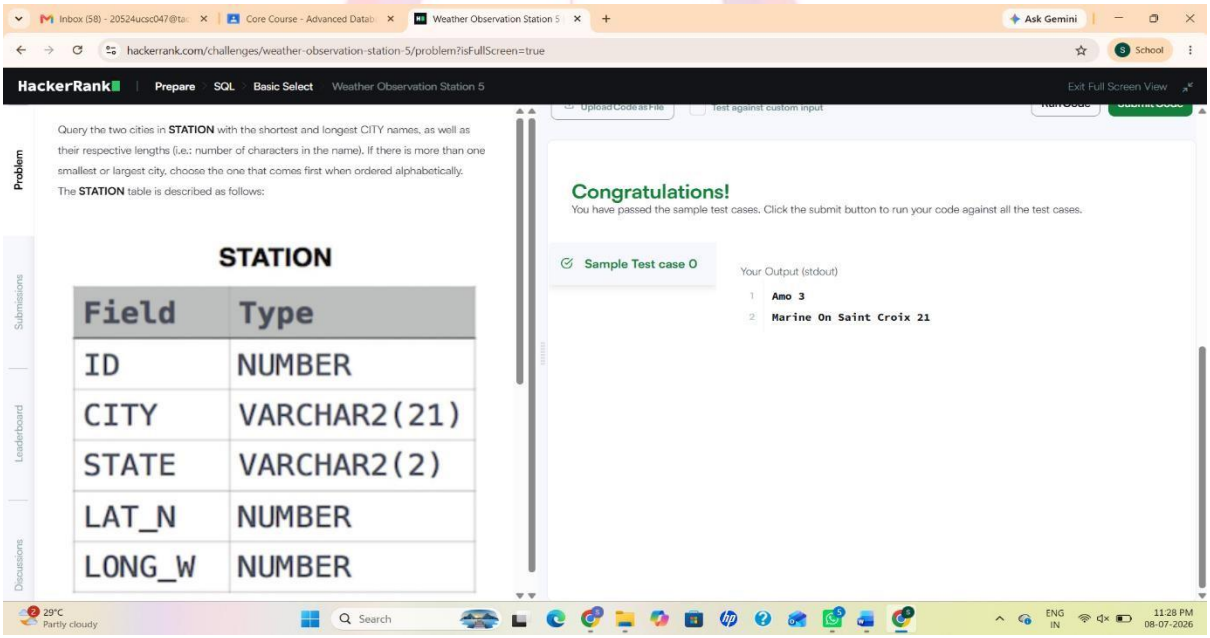
9. WEATHER OBSERVATION STATION 5

PROGRAM

```
SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY), CITY
FETCH FIRST 1 ROW ONLY;

SELECT CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY
FETCH FIRST 1 ROW ONLY;
```

OUTPUT

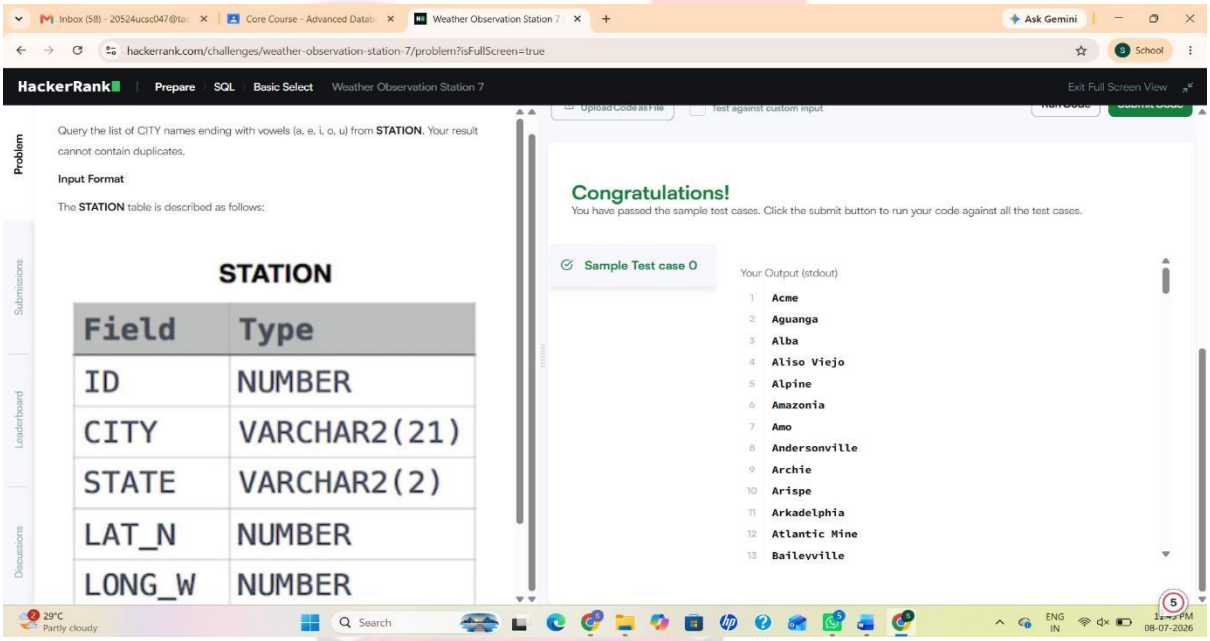


10. WEATHER OBSERVATION STATION 7

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE
UPPER(SUBSTR(CITY,LENGTH(CITY),1)) IN ('A','E','I','O','U');
```

OUTPUT

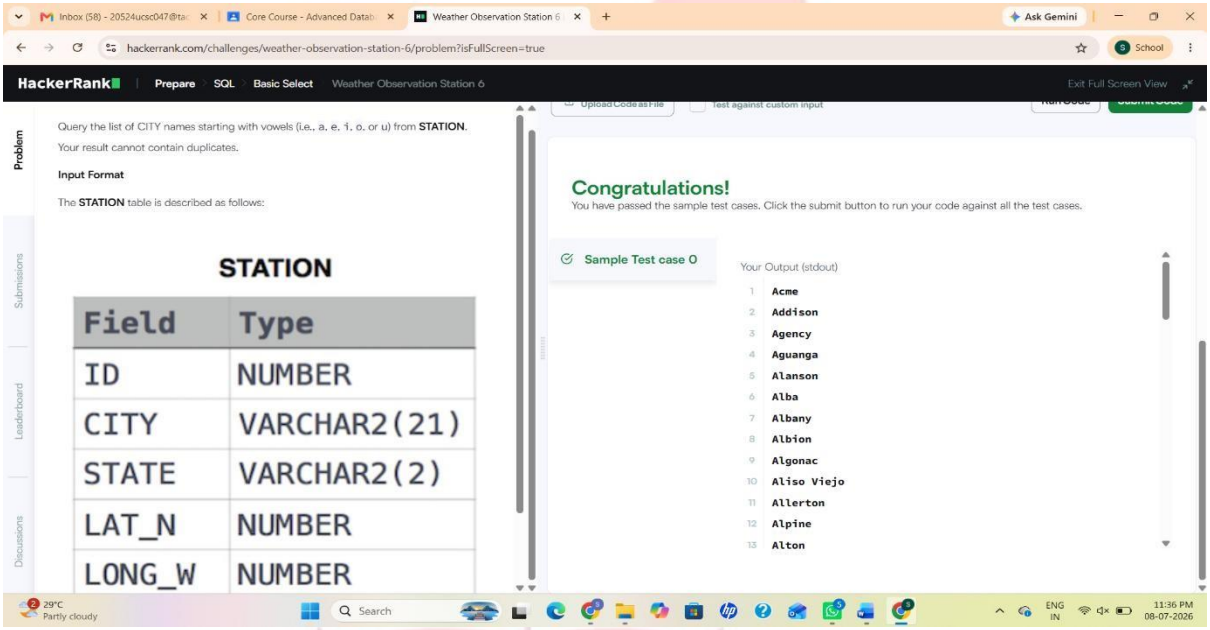


11.WEATHER OBSERVATION STATION 6

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) IN
('A','E','I','O','U');
```

OUTPUT

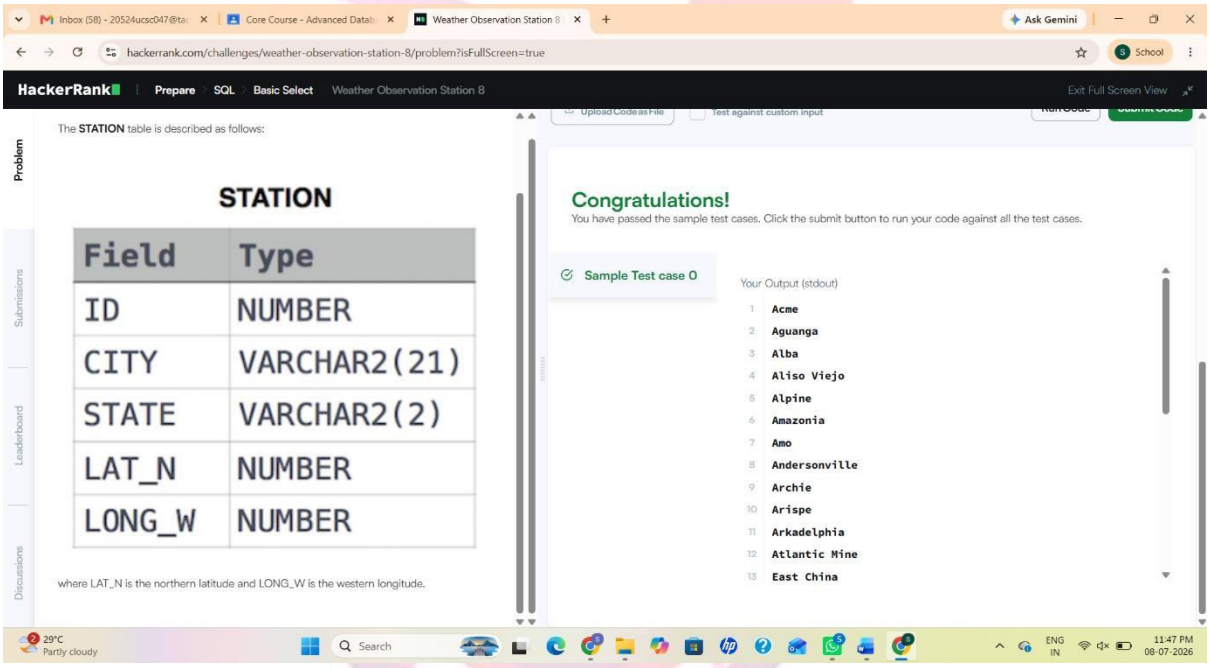


12.WHETHER OBSERVATION STATION

PROGRAM

('A','E','I','O','U') AND UPPER(SUBSTR(CITY,LENGTH(CITY),1)) IN ('A','E','I','O','U');
OUTPUT SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) IN

OUTPUT

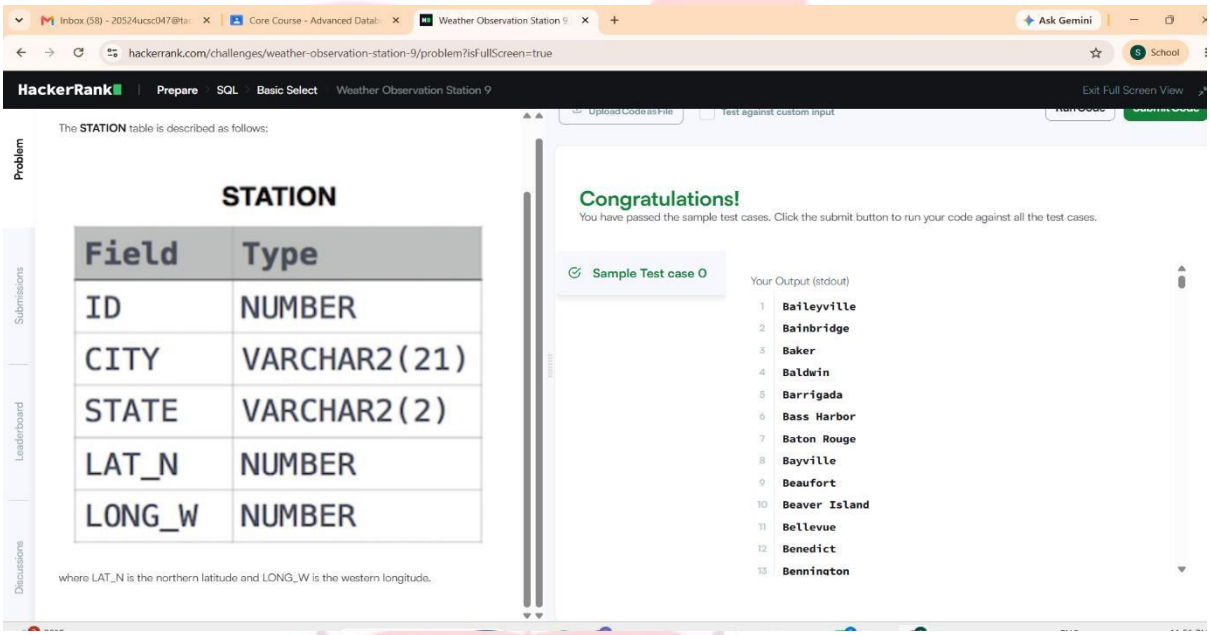


13.WHETHER OBSERVATION STATION

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT
IN ('A','E','I','O','U');
```

OUTPUT

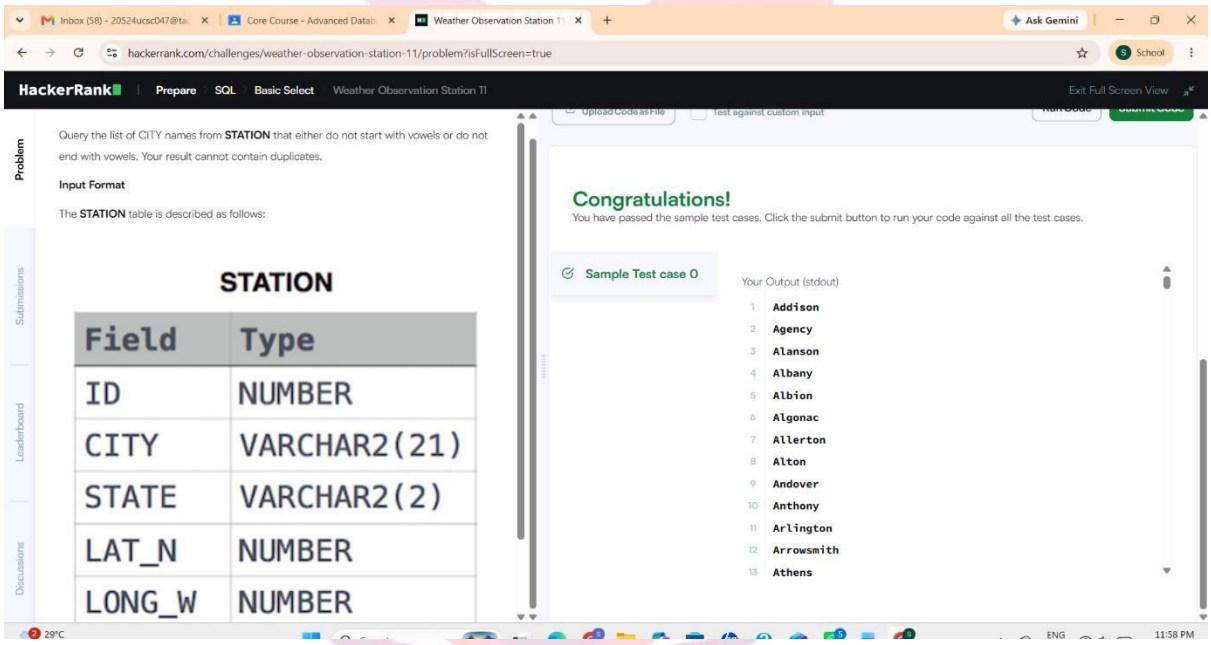


14.WEATHER OBSERVATION STATION 11

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT
IN ('A','E','I','O','U') OR UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN
('A','E','I','O','U');
```

OUTPUT

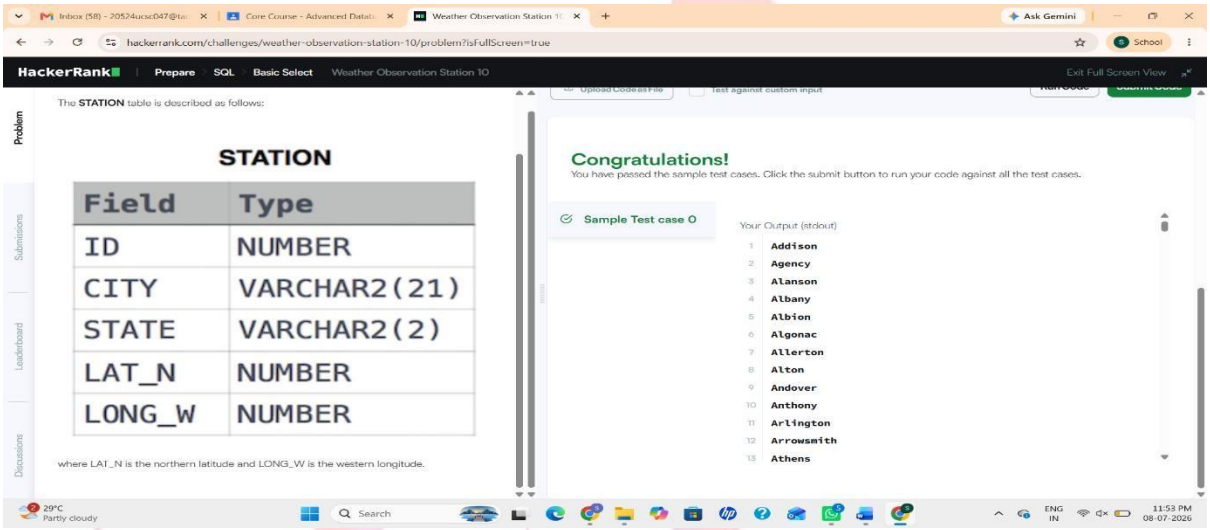


15.WHETHER OBSERVATION STATION

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE
UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN ('A','E','I','O','U');
```

OUTPUT

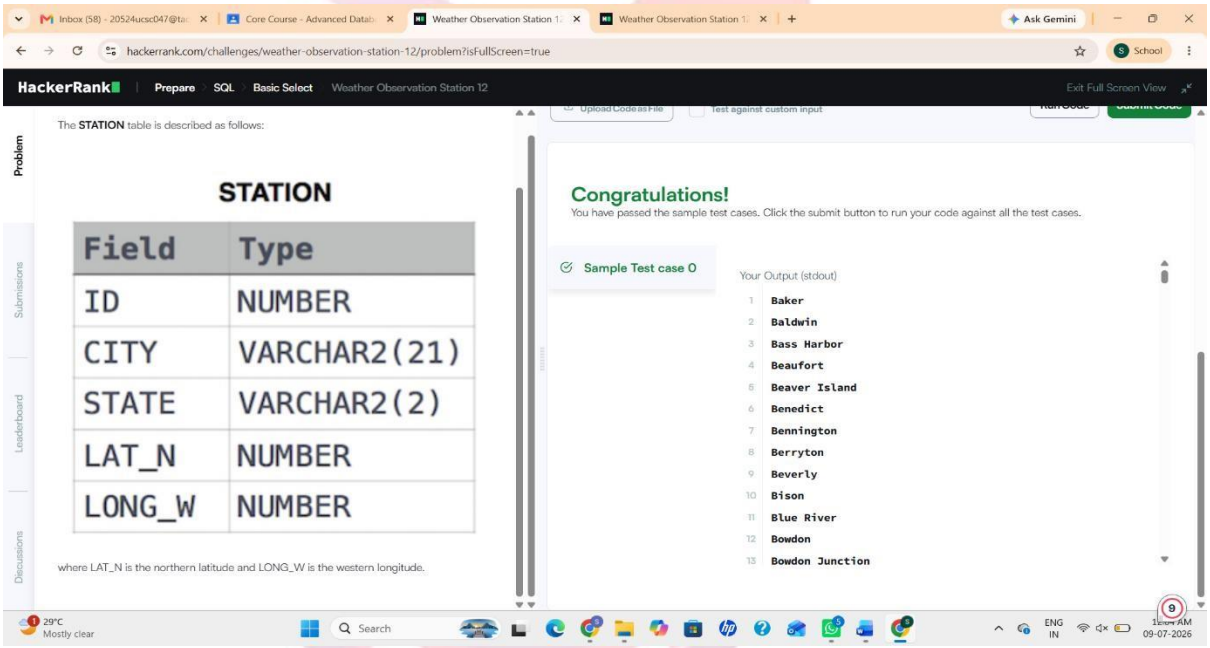


16.WEATHER OBSERVATION STATION 12

PROGRAM

```
SELECT DISTINCT CITY FROM STATION WHERE UPPER(SUBSTR(CITY,1,1)) NOT
IN ('A','E','I','O','U') AND UPPER(SUBSTR(CITY,LENGTH(CITY),1)) NOT IN
('A','E','I','O','U');
```

OUTPUT

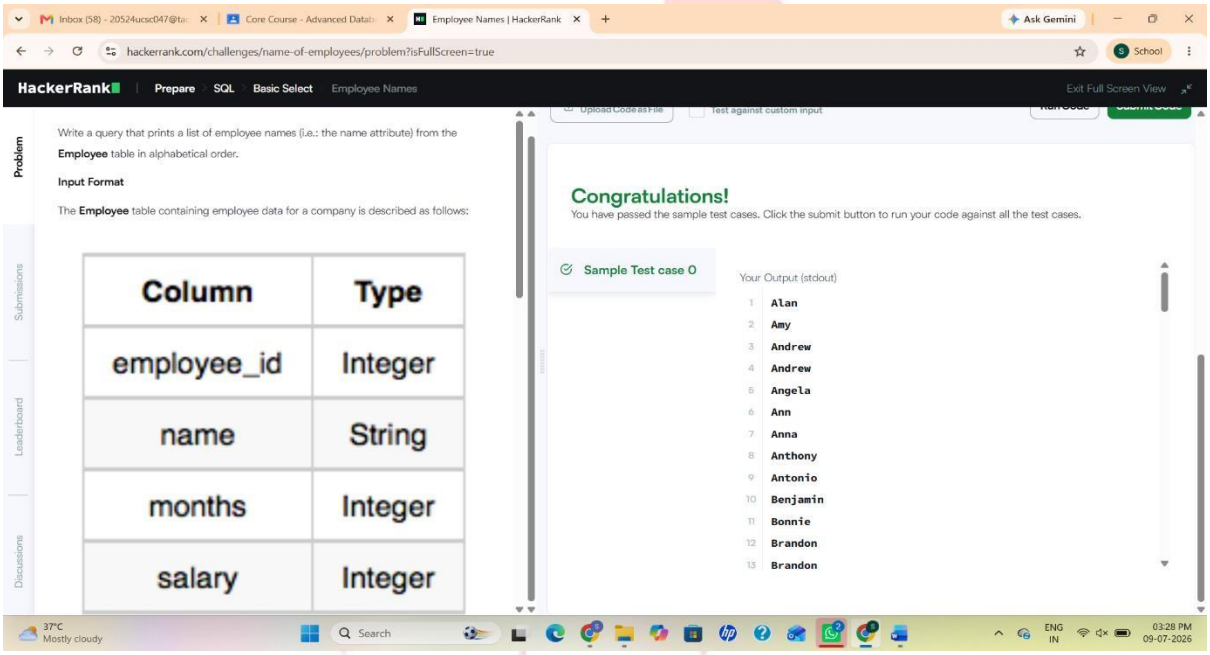


17.EMPLOYEE NAMES

PROGRAM

```
SELECT name FROM Employee ORDER BY name;
```

OUTPUT

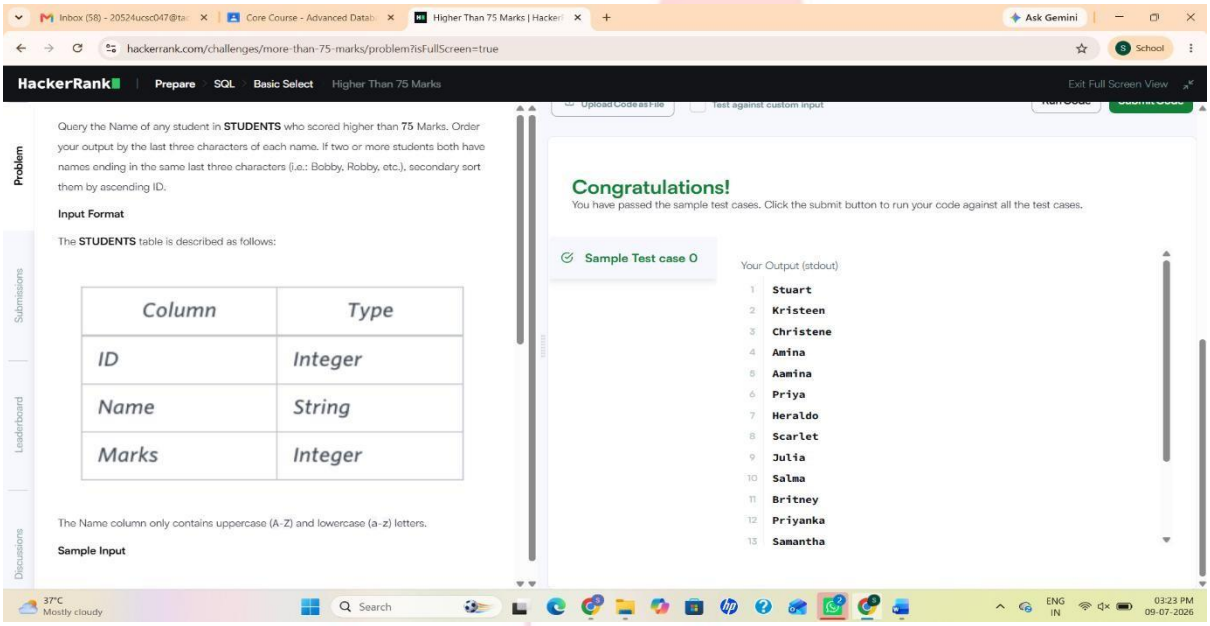


18. HIGHER THAN 75 MARKS

PROGRAM

```
SELECT Name FROM STUDENTS WHERE Marks > 75 ORDER BY SUBSTR(Name, LENGTH(Name)-2, 3), ID;
```

OUTPUT

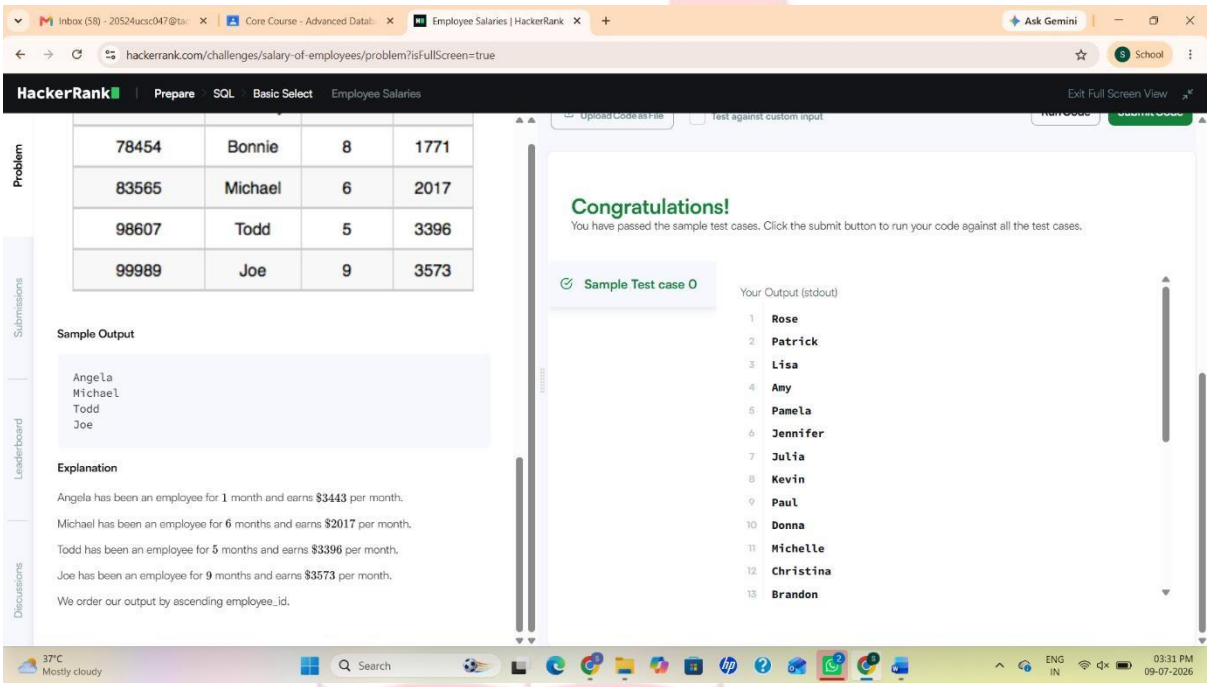


19.EMPLOYEE SALARIES

PROGRAM

```
SELECT name FROM Employee WHERE salary > 2000 AND months < 10 ORDER BY employee_id;
```

OUTPUT

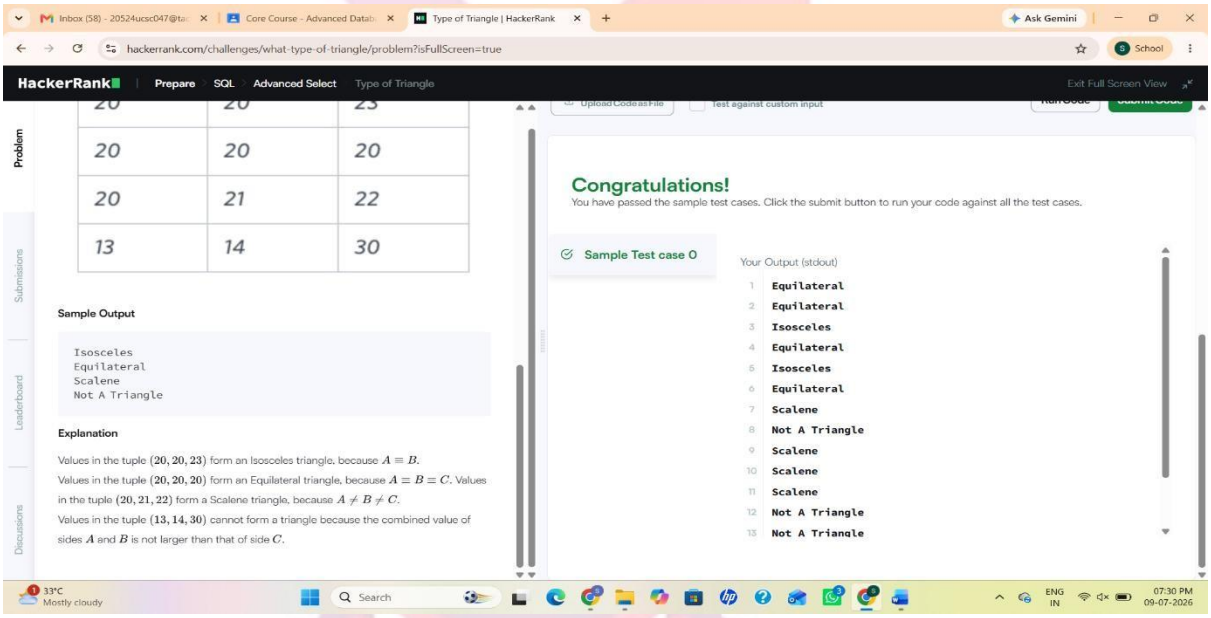


20.TYPES OF TRIANGLE

PROGRAM

```
SELECT CASE WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
WHEN A = B AND B = C THEN 'Equilateral' WHEN A = B OR B = C OR A = C THEN 'Isosceles'
ELSE 'Scalene' END FROM TRIANGLES;
```

OUTPUT



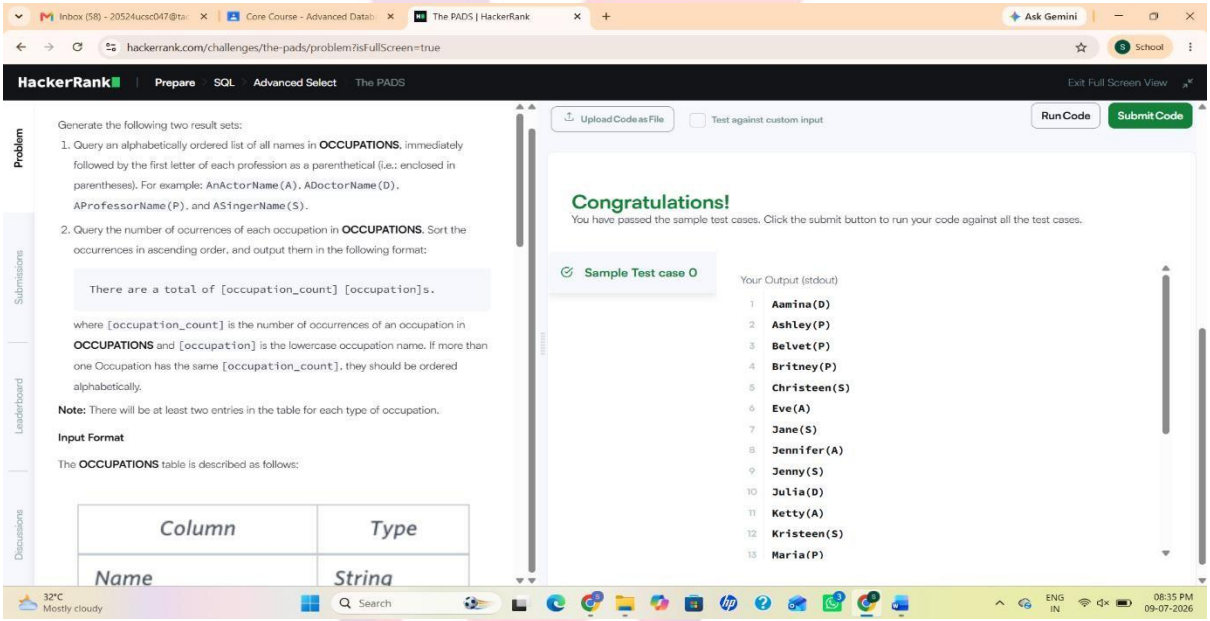
21.THE PADS

PROGRAM

```
SELECT NAME || '(' || SUBSTR(OCCUPATION,1,1) || ')' FROM OCCUPATIONS ORDER BY NAME;

SELECT 'There are a total of ' || COUNT(*) || ' ' || LOWER(OCCUPATION) || 's.' FROM OCCUPATIONS GROUP BY OCCUPATION ORDER BY COUNT(*), OCCUPATION;
```

OUTPUT



22.REVISING AGGREGATIONS – THE COUNT FUNCTION

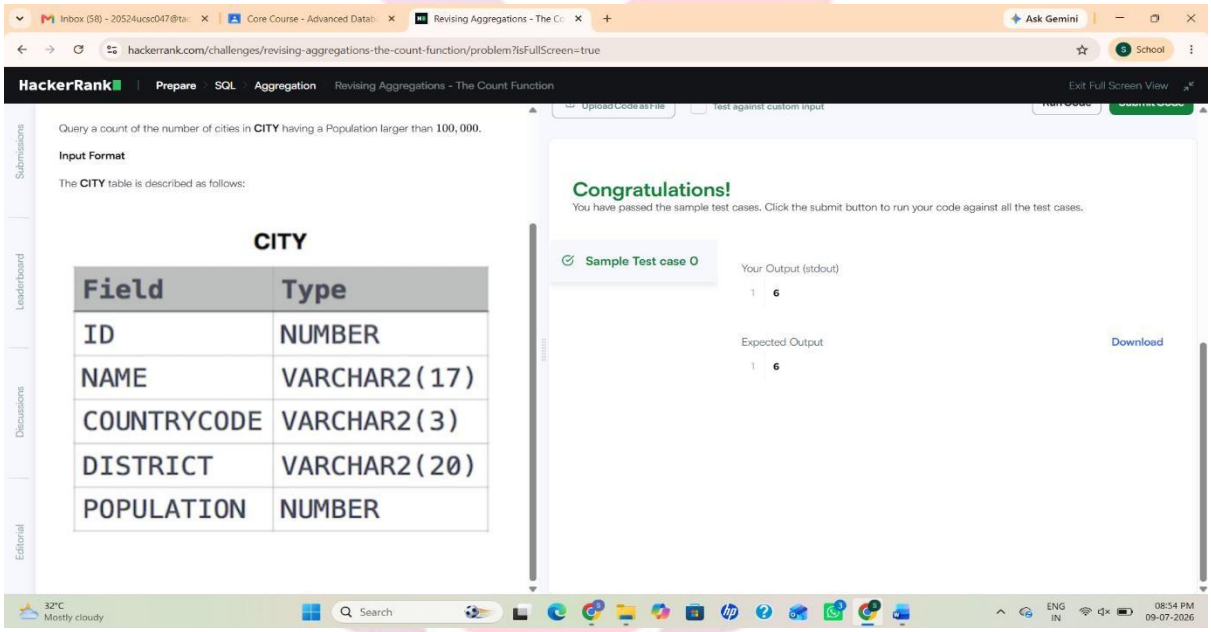
PROGRAM

```
SELECT COUNT(*)

FROM CITY

WHERE POPULATION > 100000;
```

OUTPUT



23. REVISING AGGREGATIONS – THE SUM FUNCTION

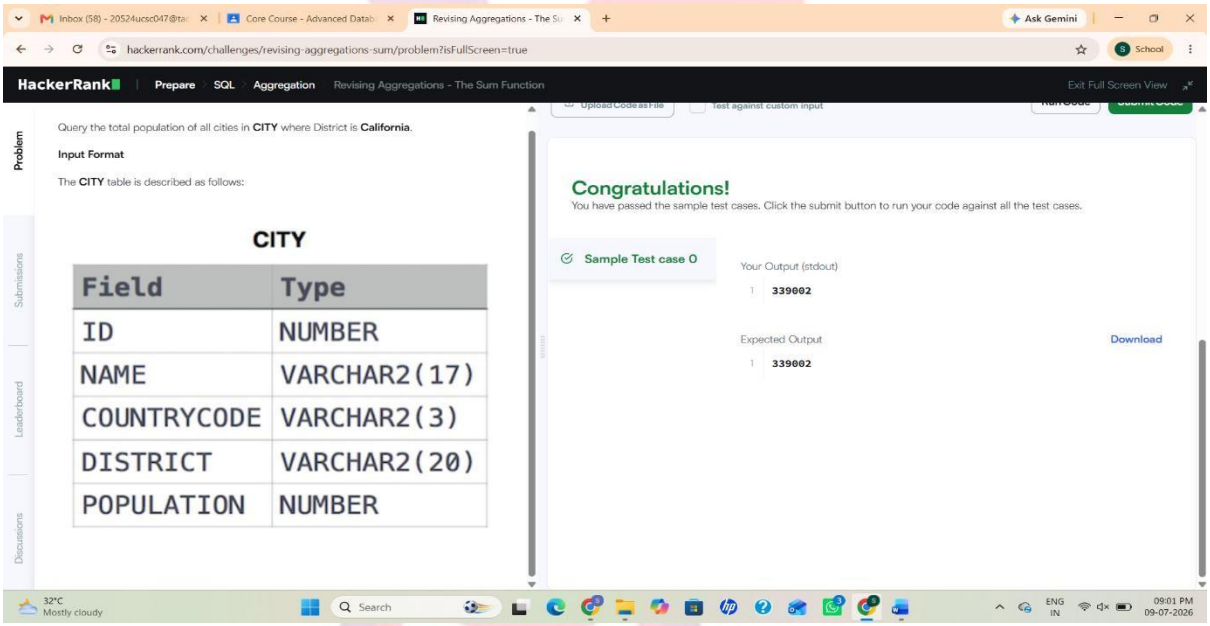
PROGRAM

```
SELECT SUM(POPULATION)

FROM CITY

WHERE DISTRICT = 'California';
```

OUTPUT

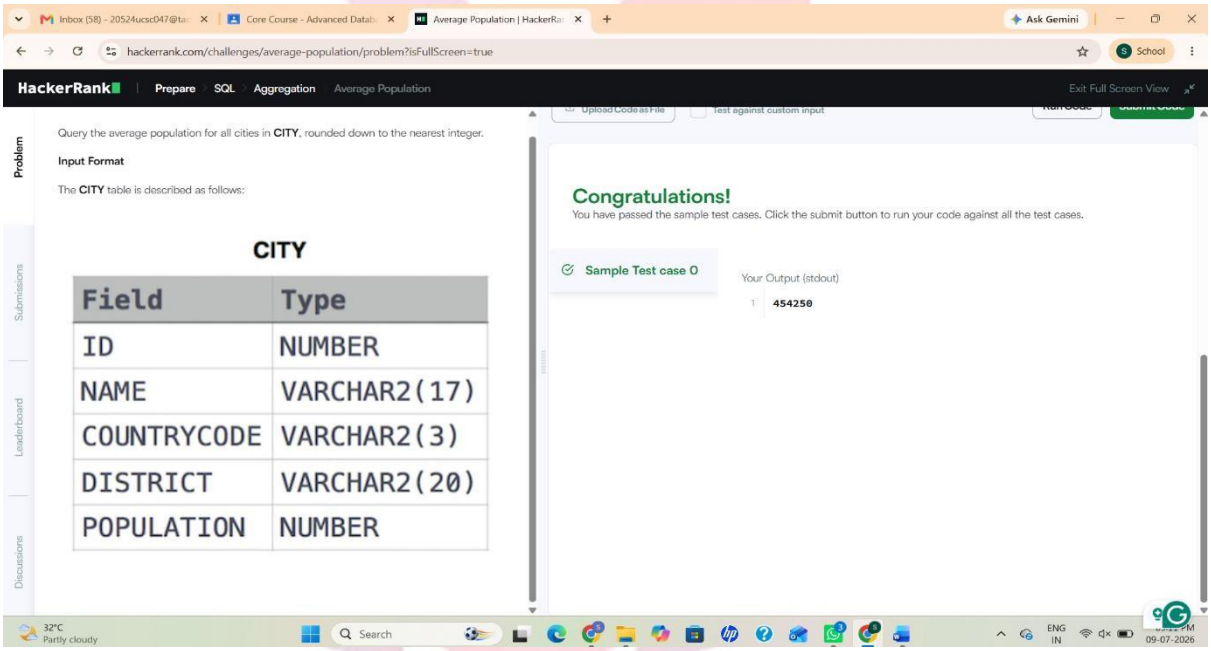


24.AVERGE POPULATION

PROGRAM

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT



25. REVISING AGGREGATIONS – AVERAGES

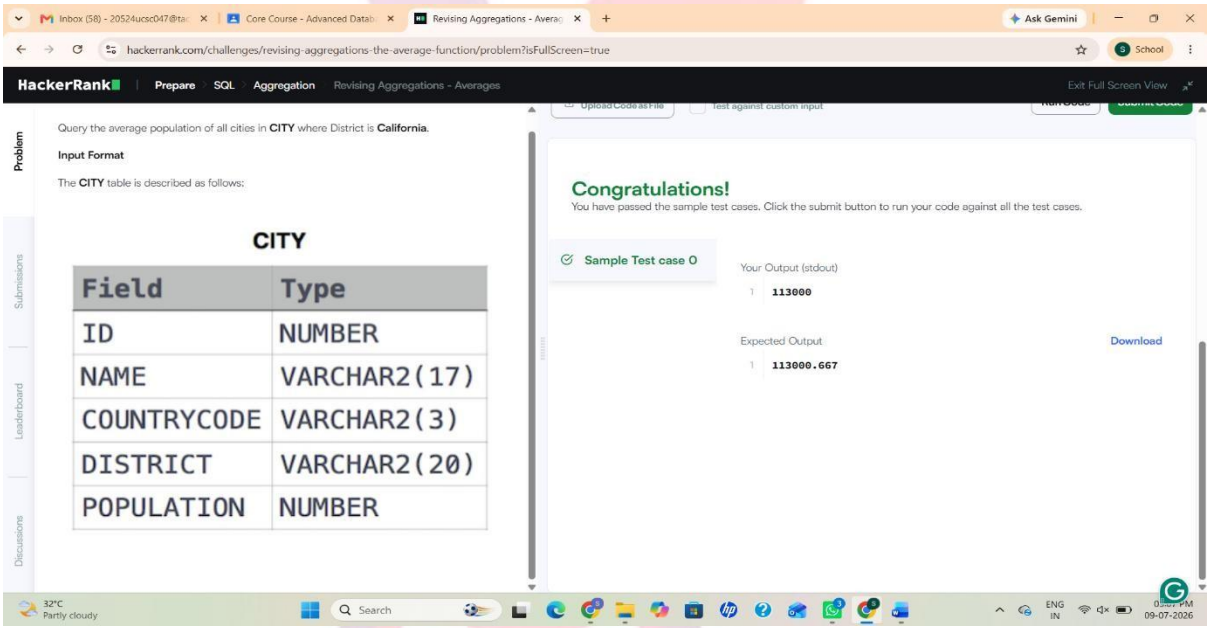
PROGRAM

```
SELECT AVG(POPULATION)

FROM CITY

WHERE DISTRICT = 'California';
```

OUTPUT



26.JAPAN POPULATION

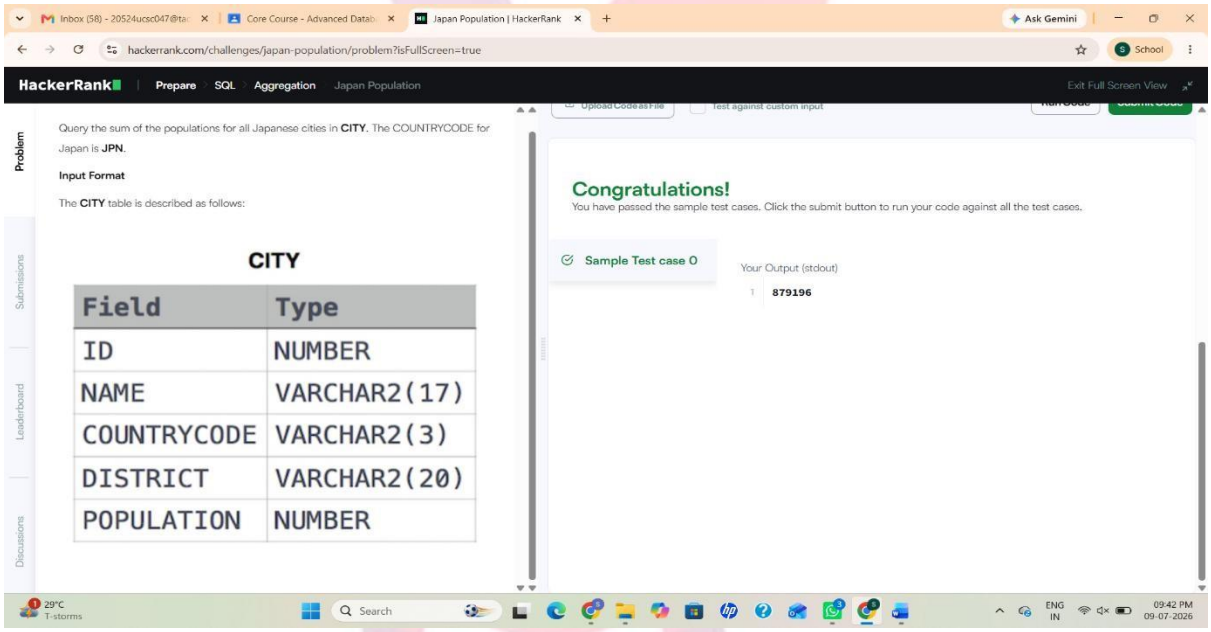
PROGRAM

```
SELECT SUM(POPULATION)

FROM CITY

WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



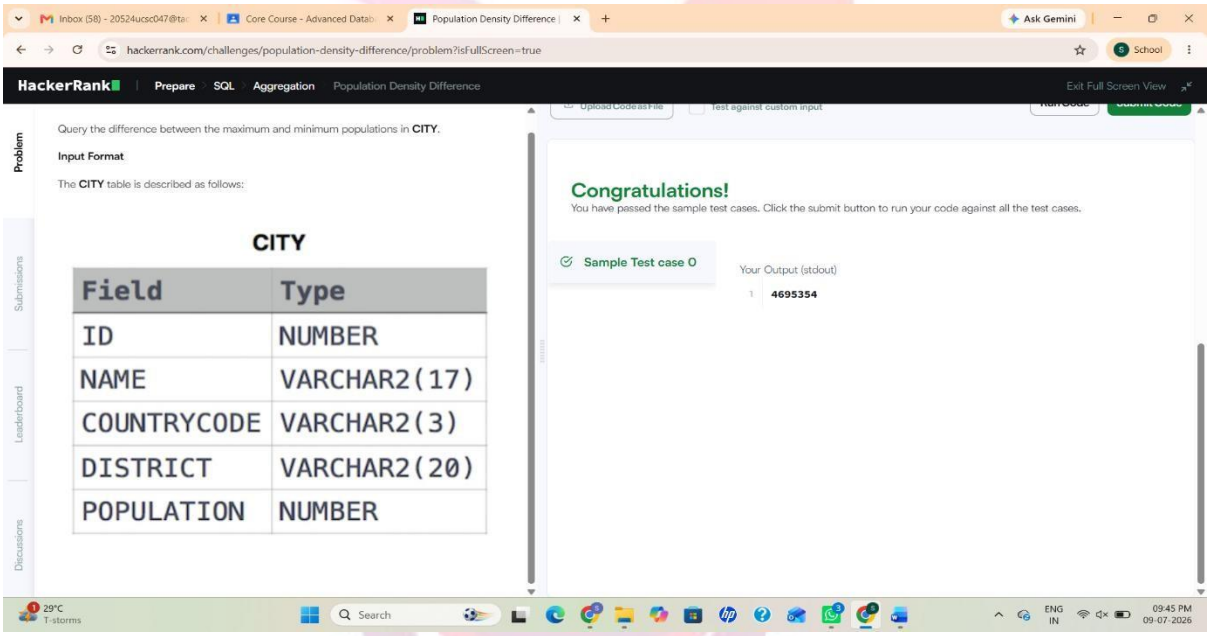
27.POPULATION DENSITY DIFFERENCE

PROGRAM

```
SELECT MAX(POPULATION) - MIN(POPULATION)

FROM CITY;
```

OUTPUT

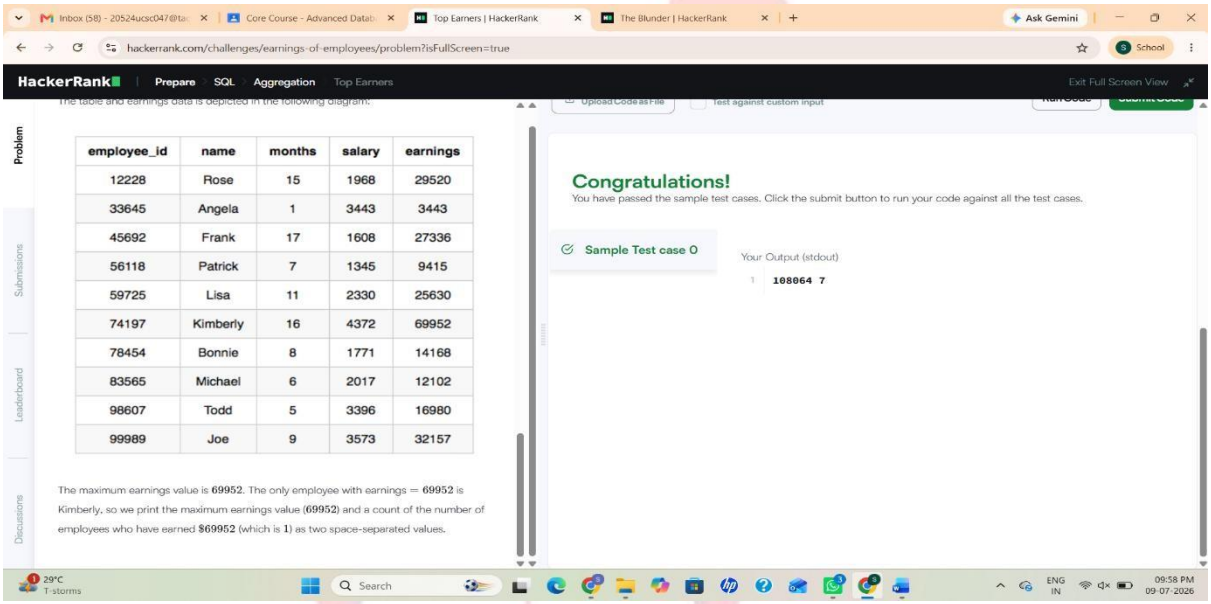


28.TOP EARNERS

PROGRAM

```
SELECT MAX(salary * months), COUNT(*) FROM Employee WHERE salary * months = (SELECT
MAX(salary * months) FROM Employee);
```

OUTPUT

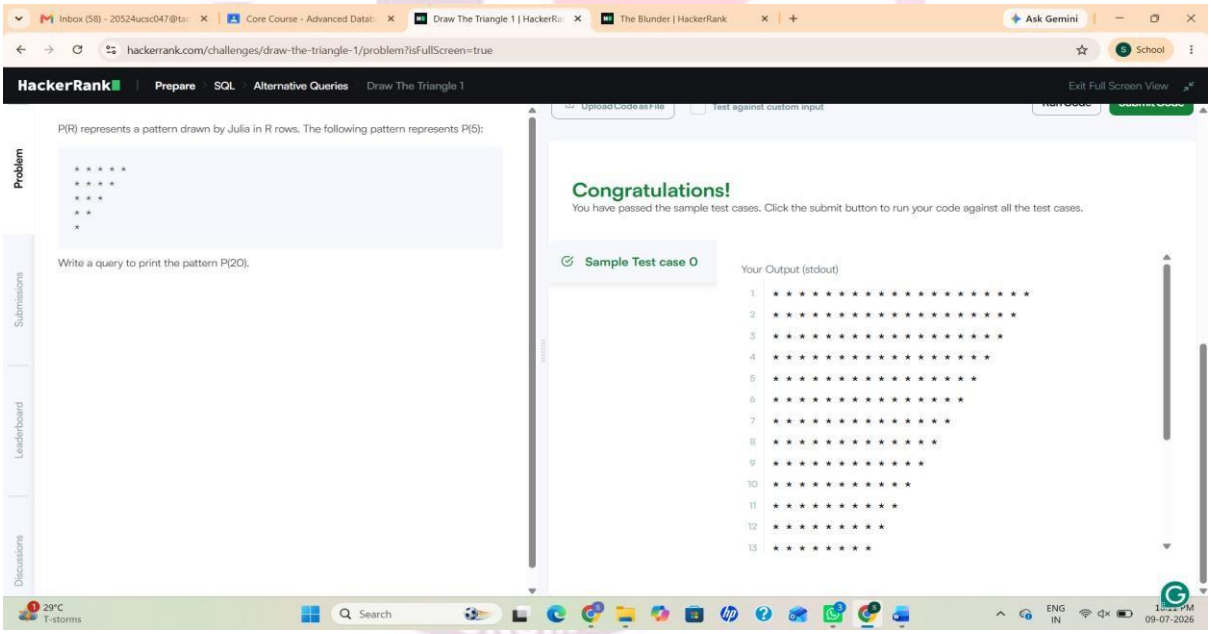


29.DRAW THE TRIANGLE 1

PROGRAM

```
SET @n = 21;
SELECT REPEAT('* ', @n := @n - 1)
FROM information_schema.tables
LIMIT 20;
```

OUTPUT

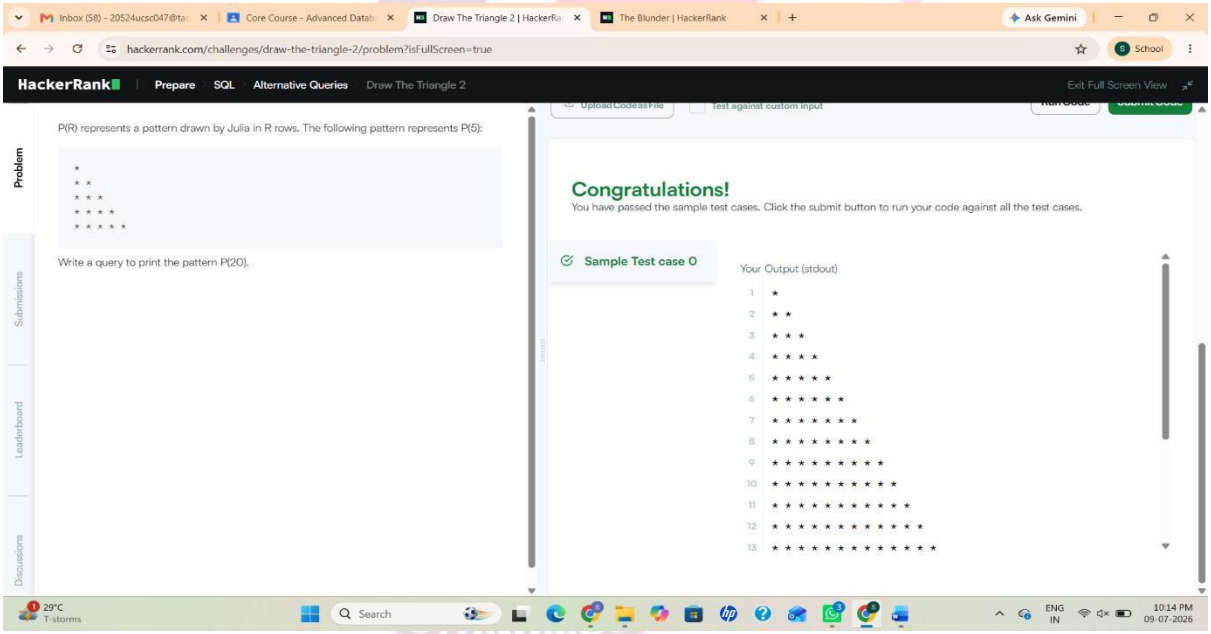


30. DRAW THE TRIANGLE 2

PROGRAM

```
SELECT REPEAT('* ', @n := @n + 1)
FROM information_schema.tables,
(SELECT @n := 0) t
LIMIT 20;
```

OUTPUT

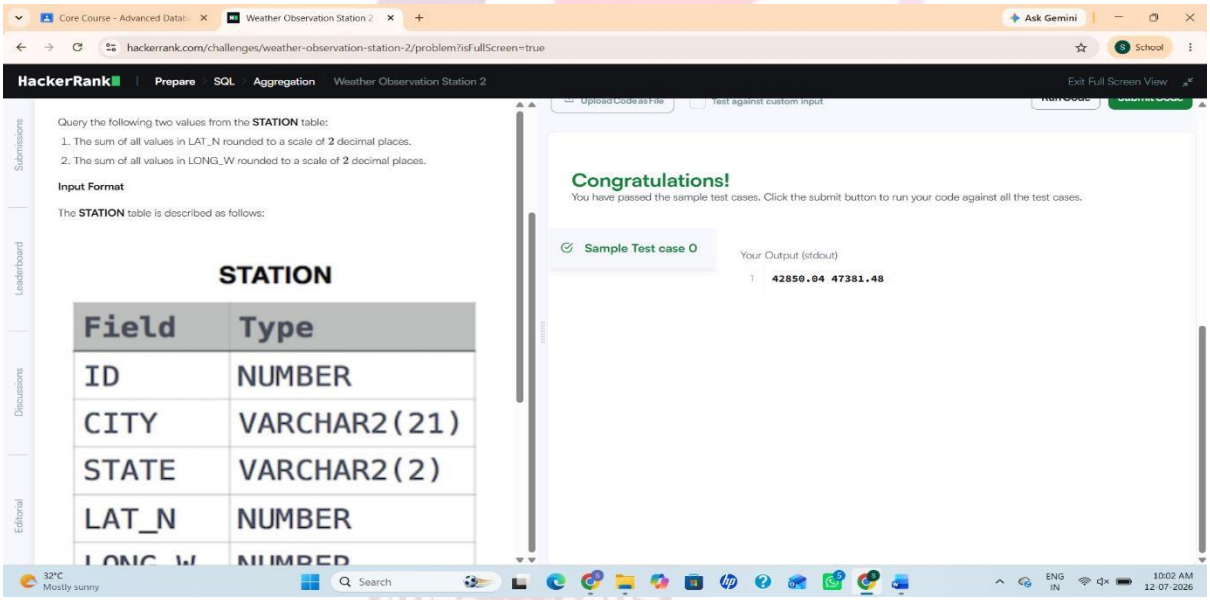


31.WEATHER OBSERVATION STATION 2

PROGRAM

```
SELECT
   ROUND(SUM(LAT_N), 2),
   ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT

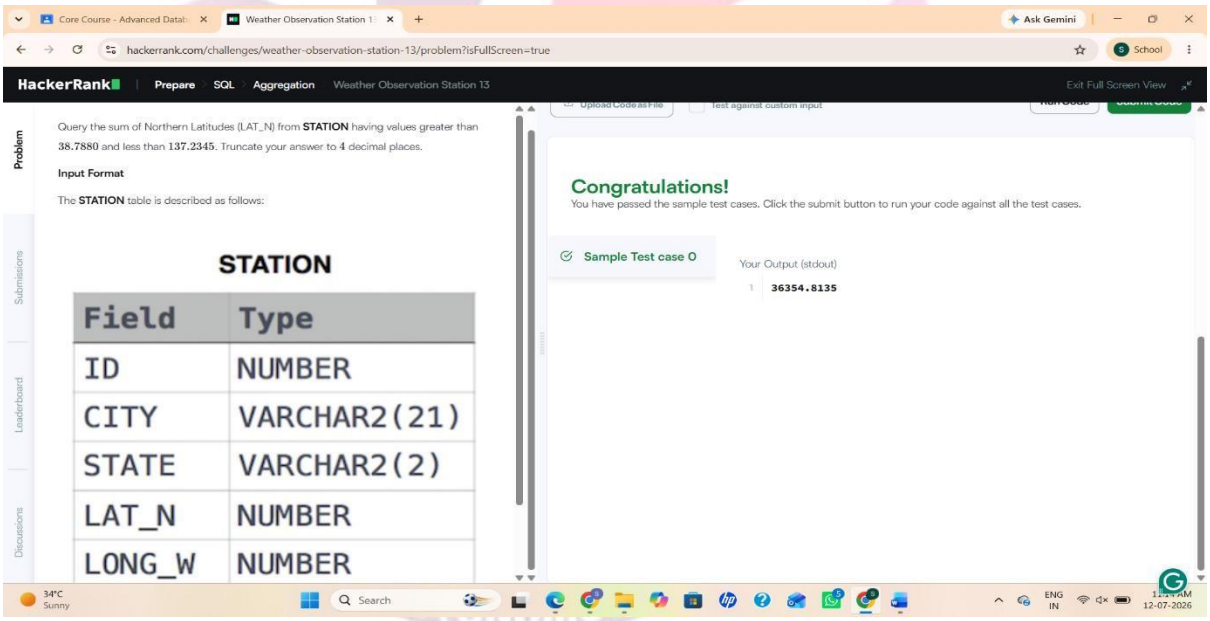


32.WEATHER OBSERVATION STATION 13

PROGRAM

```
SELECT TRUNCATE(SUM(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7880
  AND LAT_N < 137.2345;
```

OUTPUT



33.WEATHER OBSERVATION STATION 14

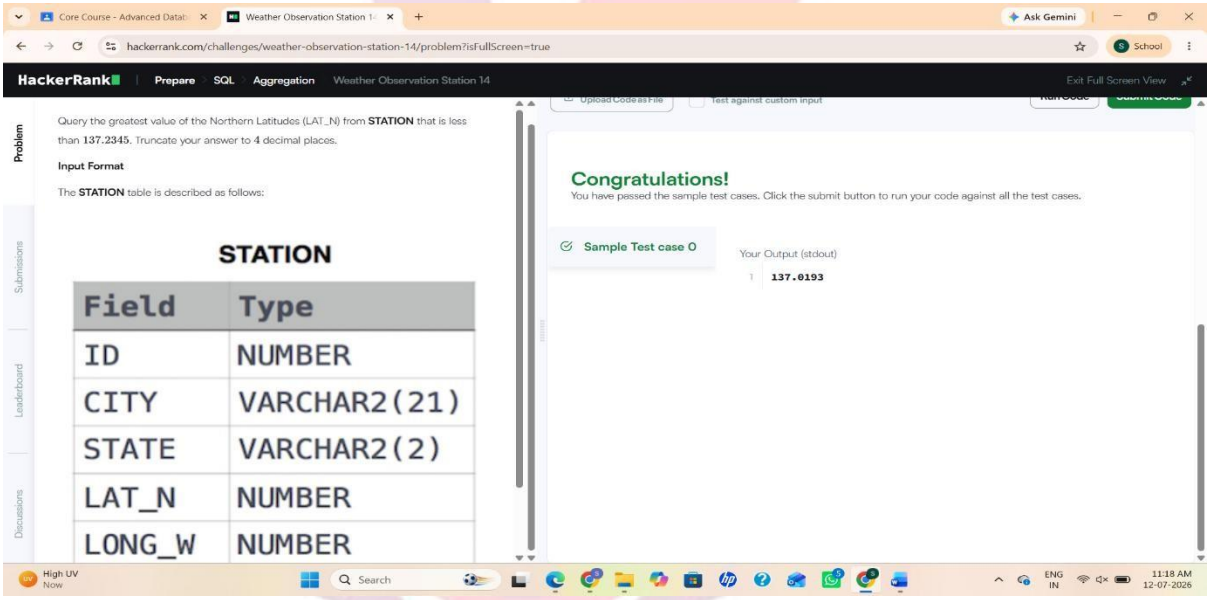
PROGRAM

```
SELECT TRUNCATE(MAX(LAT_N), 4)

FROM STATION

WHERE LAT_N < 137.2345;
```

OUTPUT

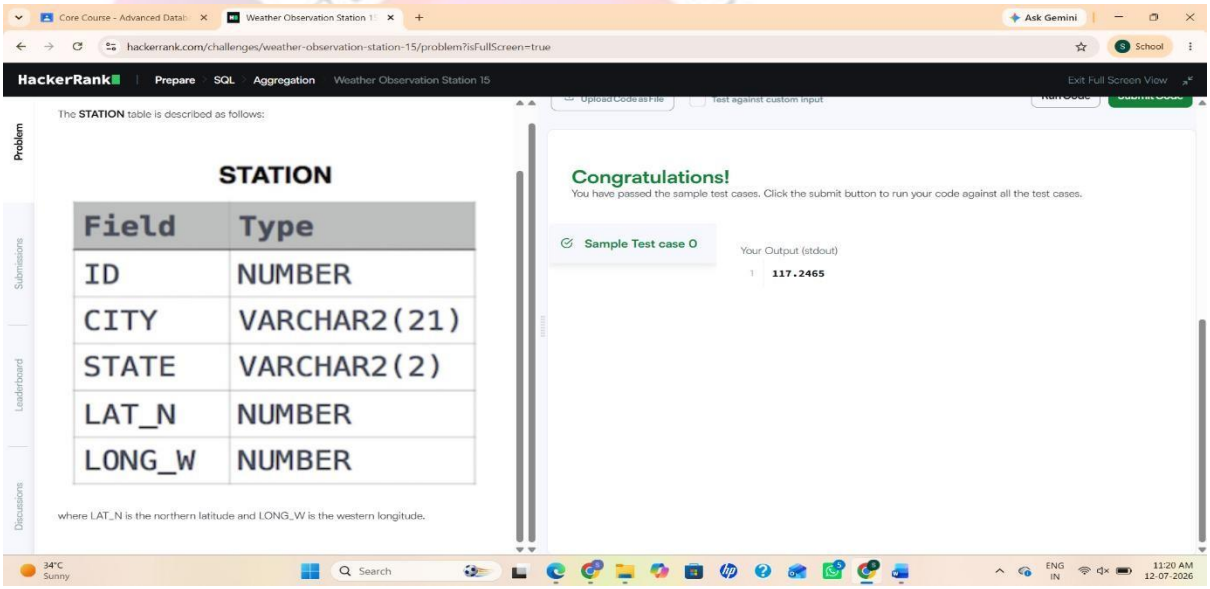


34.WEATHER OBSERVATION STATION 15

PROGRAM

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MAX(LAT_N)
   FROM STATION
   WHERE LAT_N < 137.2345
);
```

OUTPUT



35.WEATHER OBSERVATION STATION 16

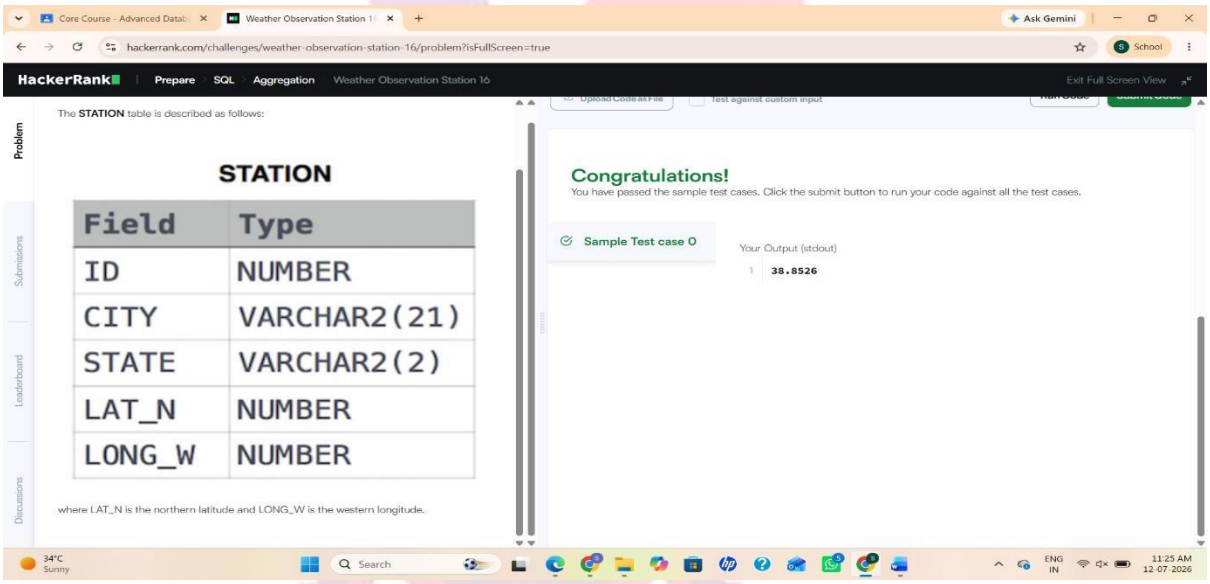
PROGRAM

```
SELECT ROUND(MIN(LAT_N), 4)

FROM STATION

WHERE LAT_N > 38.7780;
```

OUTPUT

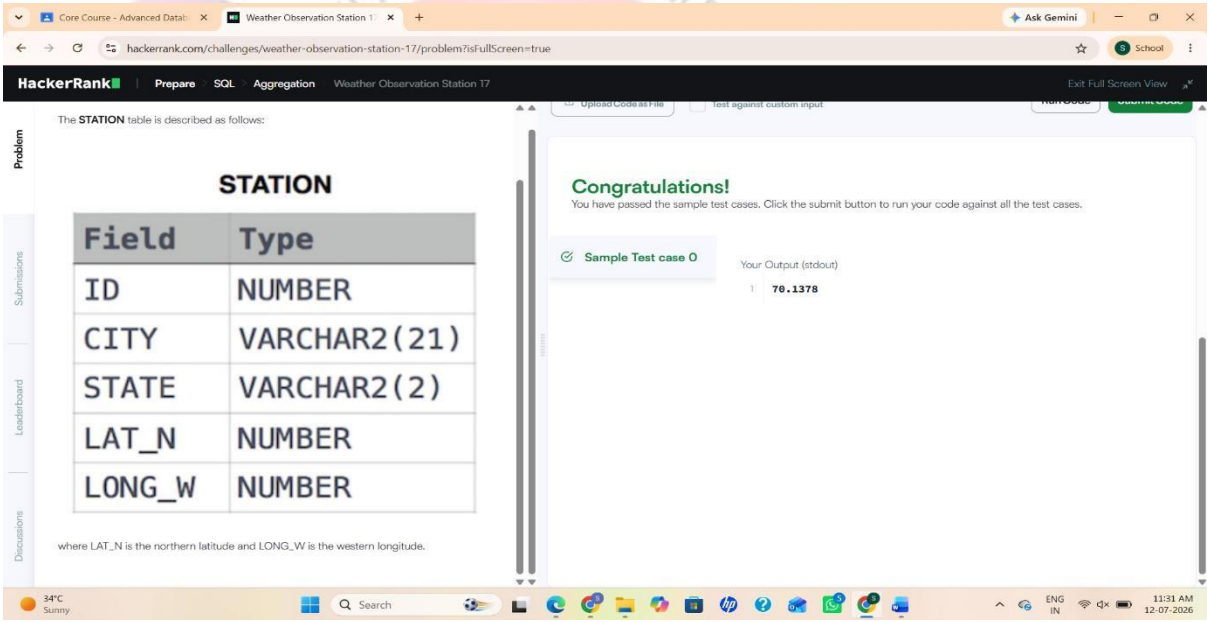


36.WEATHER OBSERVATION STATION 17

PROGRAM

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MIN(LAT_N)
   FROM STATION
   WHERE LAT_N > 38.7780
);
```

OUTPUT

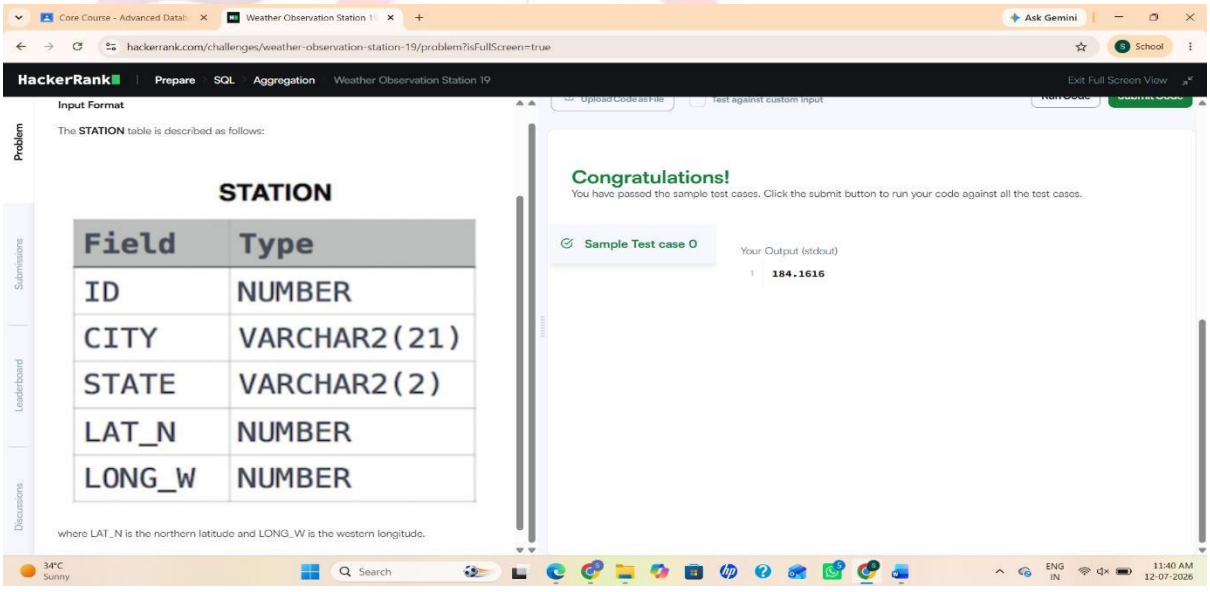


37.WEATHER OBSERVATION STATION 19

PROGRAM

```
SELECT ROUND(
   SQRT(
      POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
      POWER(MAX(LONG_W) - MIN(LONG_W), 2)
   ),
   4
)
FROM STATION;
```

OUTPUT

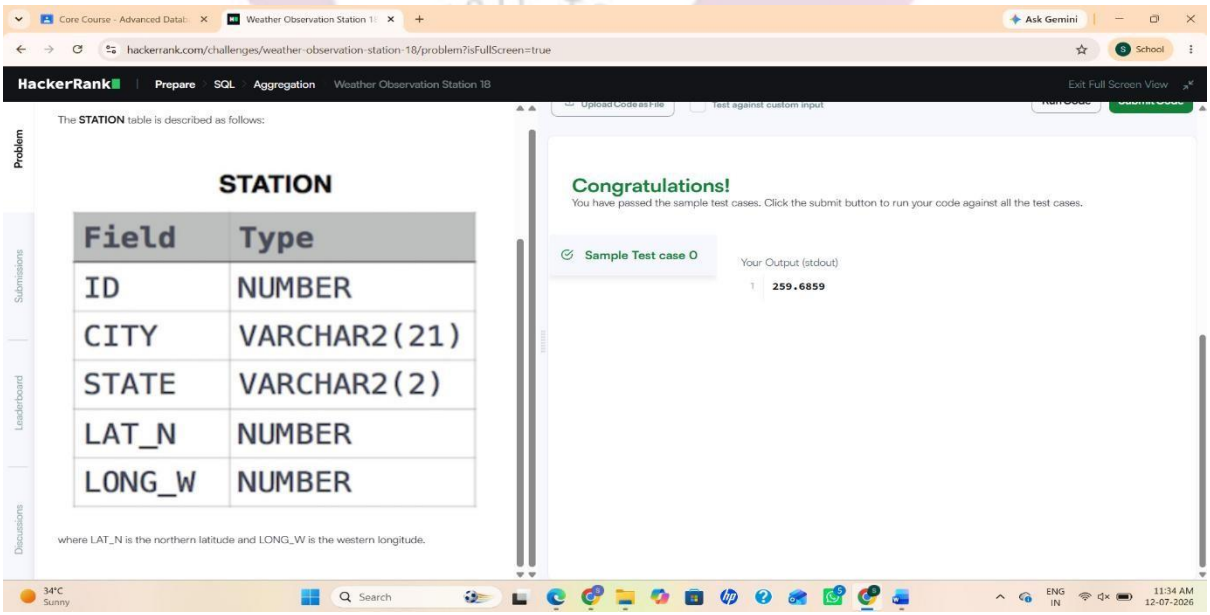


38.WEATHER OBSERVATION STATION 18

PROGRAM

```
SELECT ROUND(
    ABS(MAX(LAT_N) - MIN(LAT_N)) +
    ABS(MAX(LONG_W) - MIN(LONG_W)),
    4
)
FROM STATION;
```

OUTPUT



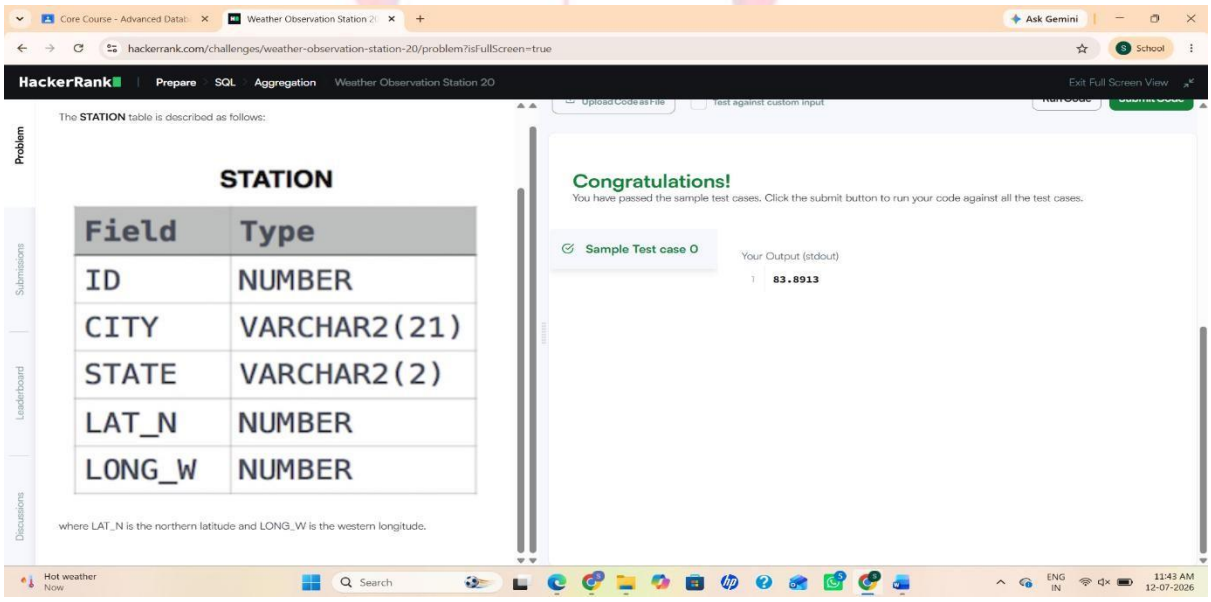
39.WEATHER OBSERVATION STATION 20

PROGRAM

```
SET @rownum := 0;


SELECT ROUND(AVG(LAT_N), 4)
FROM (
    SELECT LAT_N,
        @rownum := @rownum + 1 AS row_num,
        (SELECT COUNT(*) FROM STATION) AS total_rows
    FROM STATION
    ORDER BY LAT_N
) AS t
WHERE row_num IN (FLOOR((total_rows + 1) / 2), FLOOR((total_rows + 2) / 2));
```

OUTPUT

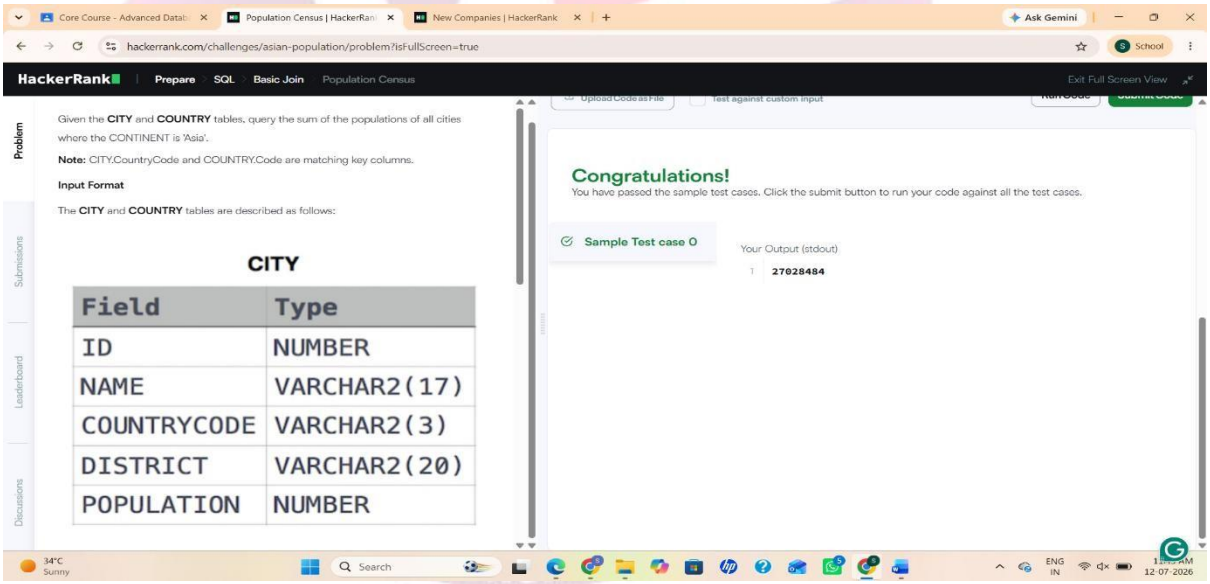


40.POPULATION CENSUS

PROGRAM

```
SELECT SUM(CITY.Population)
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Asia';
```

OUTPUT

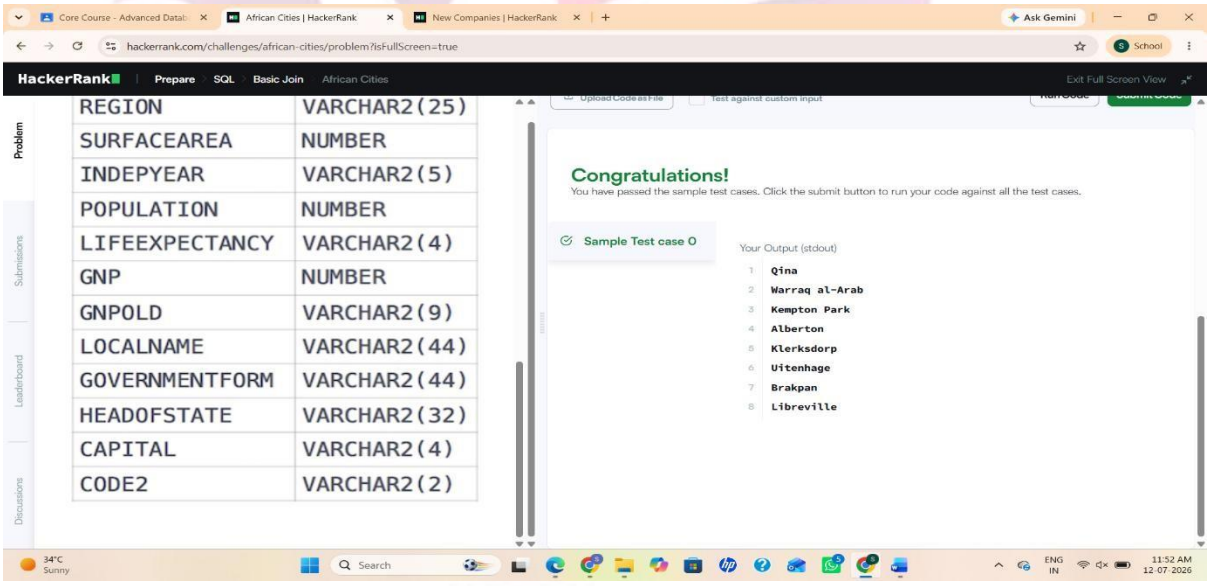


41.AFRICAN CITIES

PROGRAM

```
SELECT CITY.NAME
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
WHERE COUNTRY.Continent = 'Africa';
```

OUTPUT

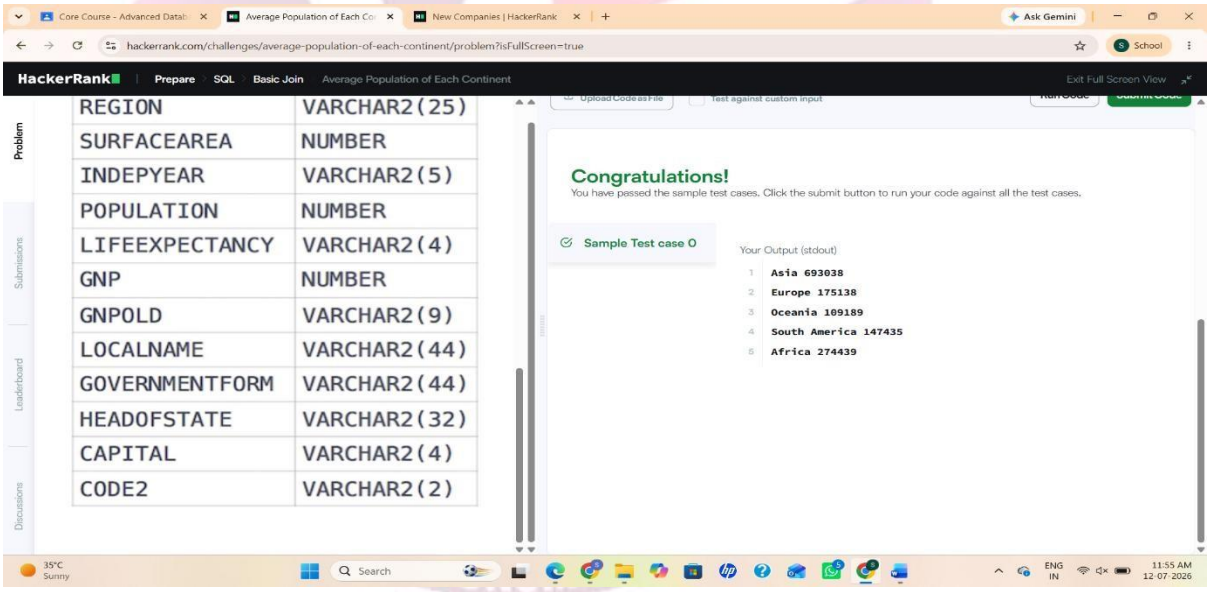


42. AVERAGE POPULATION OF EACH CONTINENT

PROGRAM

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY
ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

OUTPUT

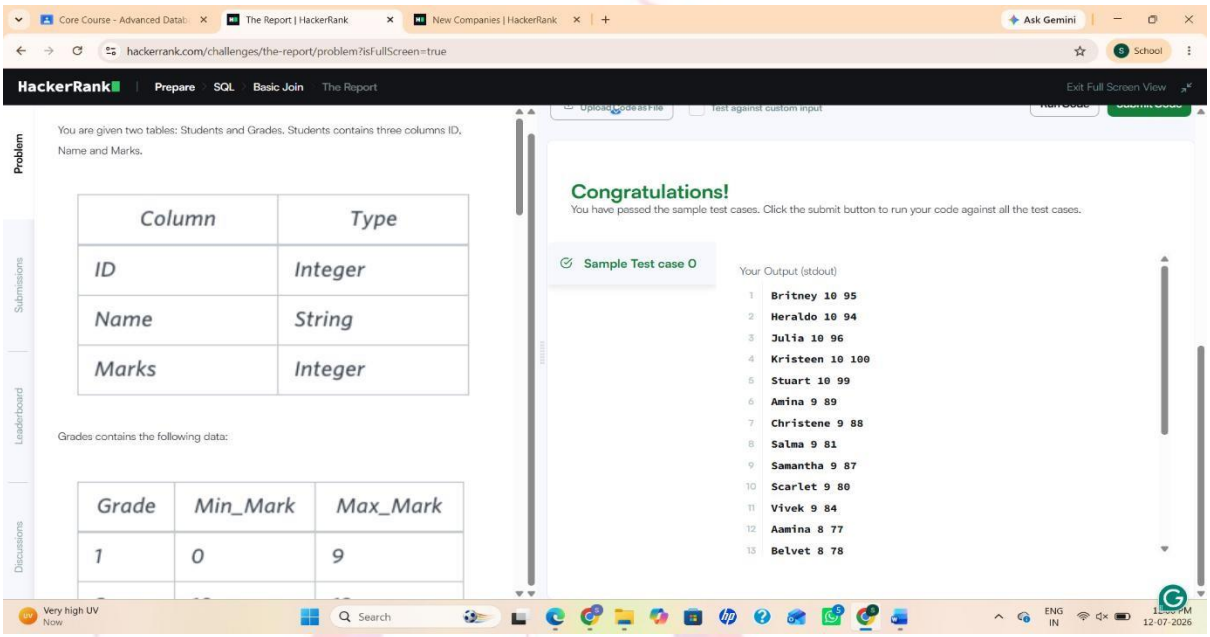


43.THE REPORT

PROGRAM

```
SELECT
   CASE
      WHEN G.Grade < 8 THEN 'NULL'
      ELSE S.Name
   END AS Name,
   G.Grade,
   S.Marks
FROM Students S
JOIN Grades G
ON S.Marks BETWEEN G.Min_Mark AND G.Max_Mark
ORDER BY
   G.Grade DESC,
   CASE
      WHEN G.Grade >= 8 THEN S.Name
   END ASC,
   CASE
      WHEN G.Grade < 8 THEN S.Marks
   END ASC;
```

OUTPUT



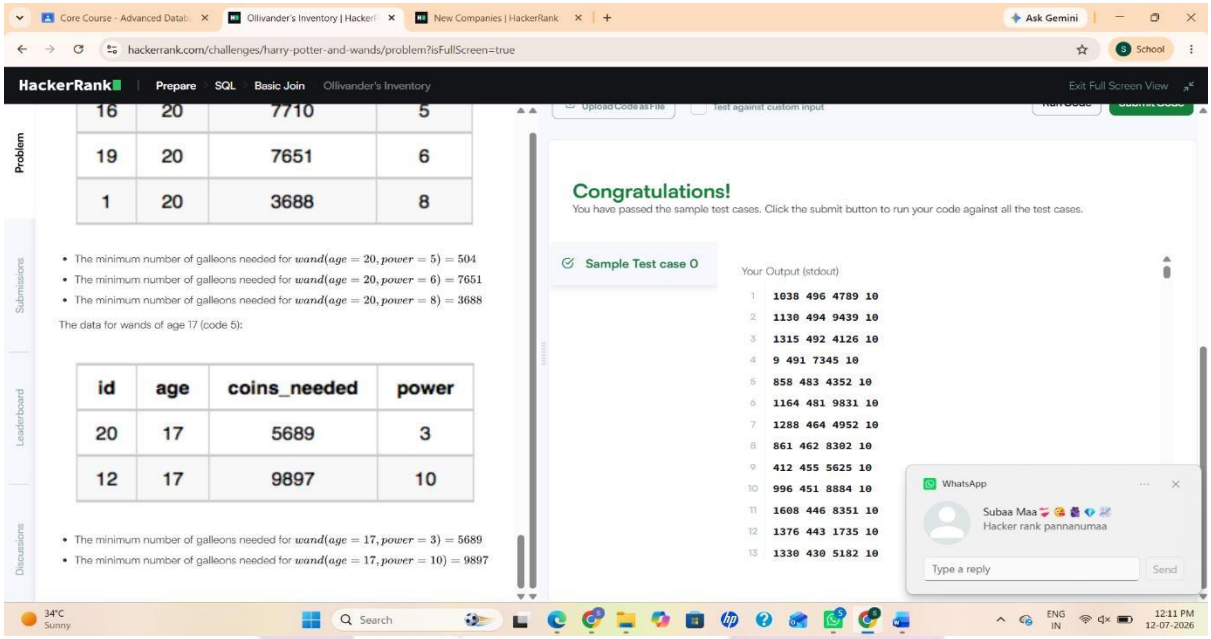
AUTONOMOUS

44. OLLIVANDER'S INVENTORY

PROGRAM

```
SELECT
   w.id,     wp.age,
   w.coins_needed,
   w.power
FROM Wands w
JOIN Wands_Property wp
   ON w.code = wp.code
WHERE wp.is_evil = 0
  AND w.coins_needed = (
     SELECT MIN(w2.coins_needed)
     FROM Wands w2
     JOIN Wands_Property wp2
ON w2.code = wp2.code
     WHERE wp2.is_evil = 0
       AND w2.power = w.power
       AND wp2.age = wp.age
  )
ORDER BY
  w.power DESC,    wp.age
DESC;
```

OUTPUT



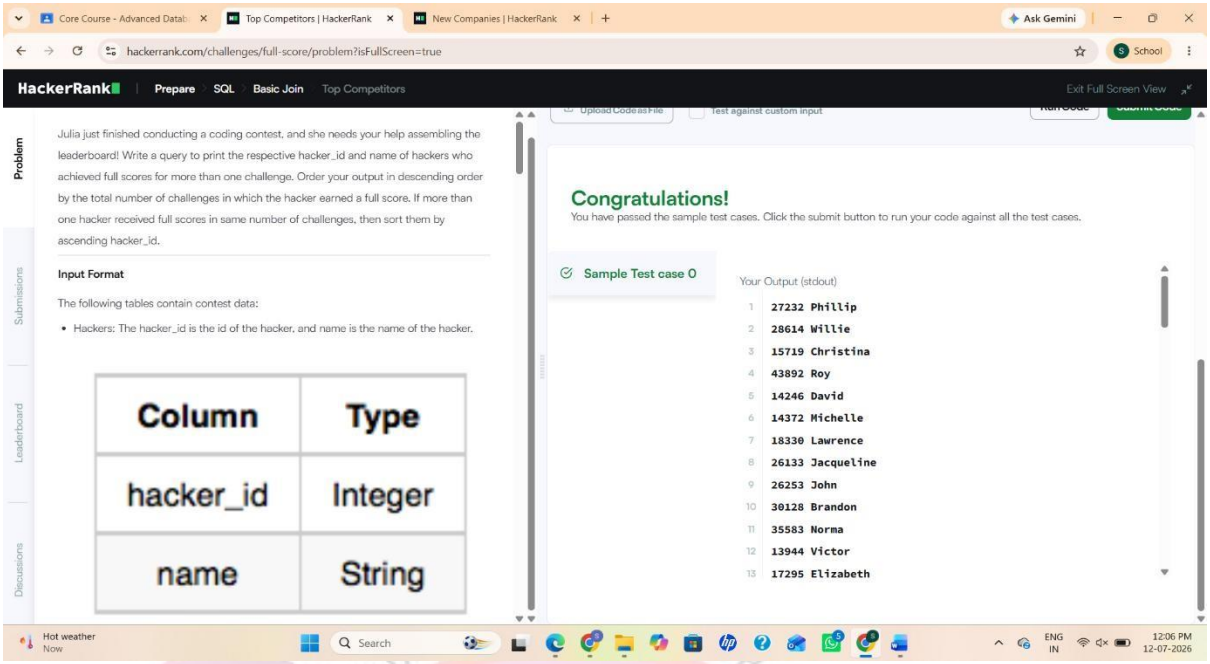
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

45. TOP COMPETITORS

PROGRAM

```
SELECT
   h.hacker_id,
   h.name
FROM Hackers h
JOIN Submissions s
   ON h.hacker_id = s.hacker_id
JOIN Challenges c
   ON s.challenge_id = c.challenge_id
JOIN Difficulty d
   ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1 ORDER BY
COUNT(DISTINCT s.challenge_id) DESC,
      h.hacker_id ASC;
```

OUTPUT



AUTONOMOUS

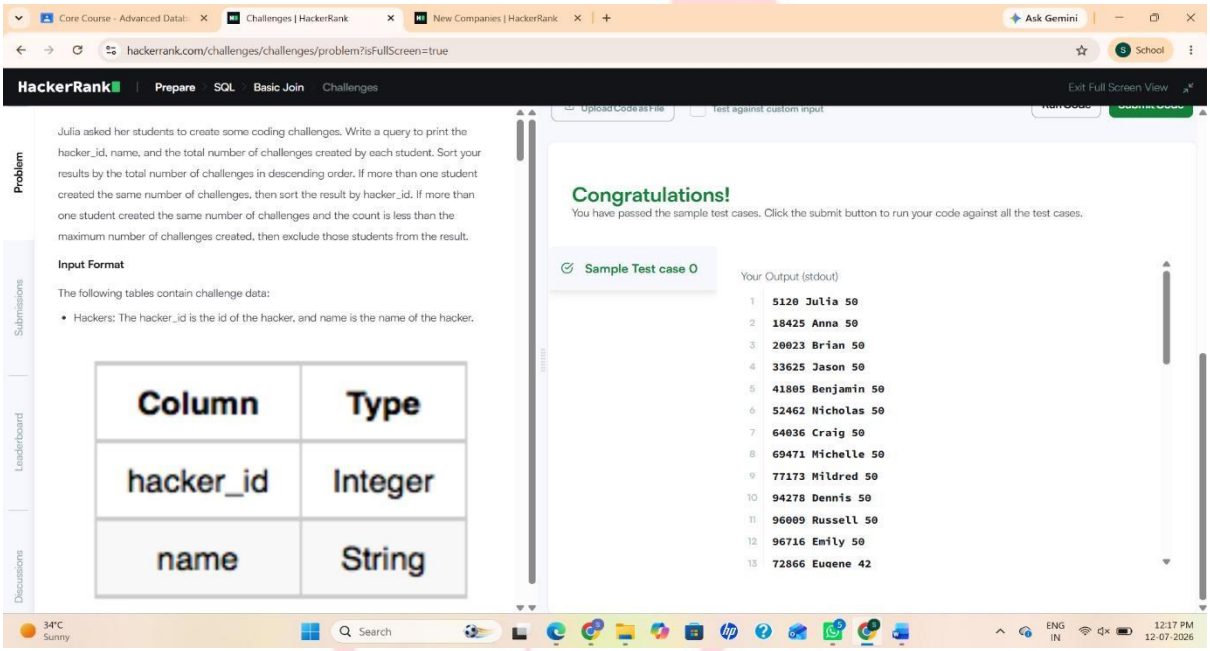
46.CHALLENGES

PROGRAM

```
SELECT h.hacker_id,
    h.name,
    COUNT(c.challenge_id) AS total_challenges
FROM Hackers h
JOIN Challenges c
ON h.hacker_id = c.hacker_id
GROUP BY h.hacker_id, h.name
HAVING COUNT(c.challenge_id) = (
        SELECT MAX(cnt)
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
            GROUP BY hacker_id
        ) t
    )
    OR COUNT(c.challenge_id) IN (
        SELECT cnt
        FROM (
            SELECT COUNT(*) AS cnt
            FROM Challenges
           GROUP BY hacker_id
        ) t
        GROUP BY cnt
```

```
    HAVING COUNT(*) = 1
)
ORDER BY total_challenges DESC,
    h.hacker_id ASC;
```

OUTPUT

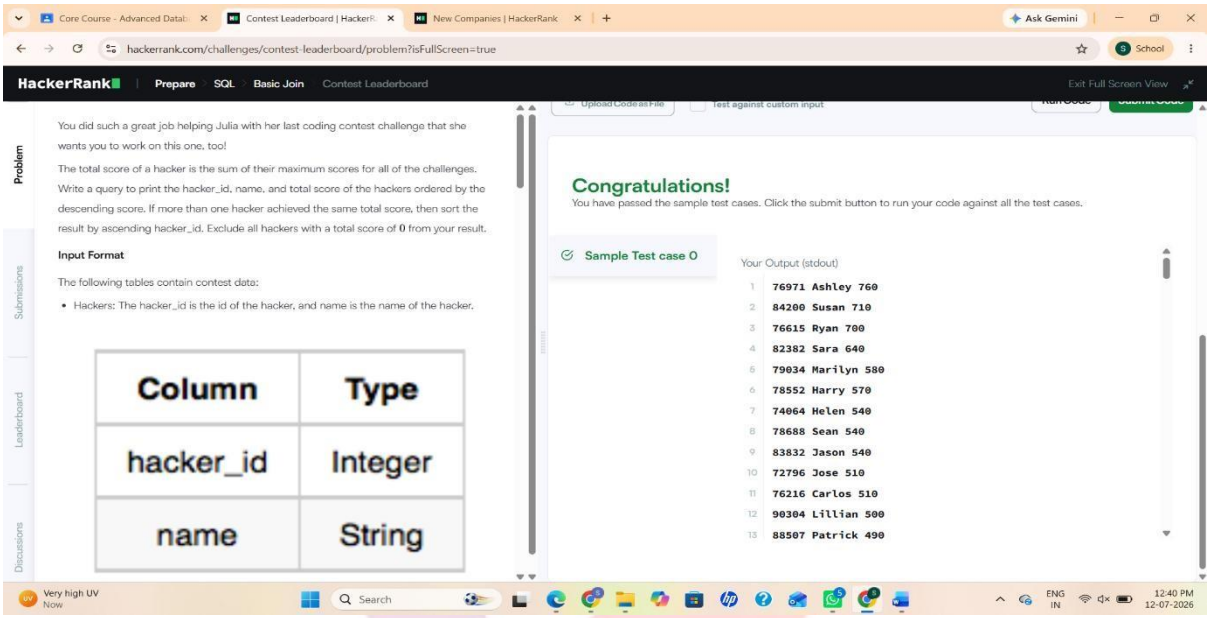


47.CONTEST LEADERBOARD

PROGRAM

```
SELECT
    h.hacker_id,
    h.name,
    SUM(ms.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT
        hacker_id,        challenge_id,
        MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) ms
ON h.hacker_id = ms.hacker_id
GROUP BY h.hacker_id, h.name
HAVING SUM(ms.max_score) > 0 ORDER BY
total_score DESC,
        h.hacker_id ASC;
```

OUTPUT

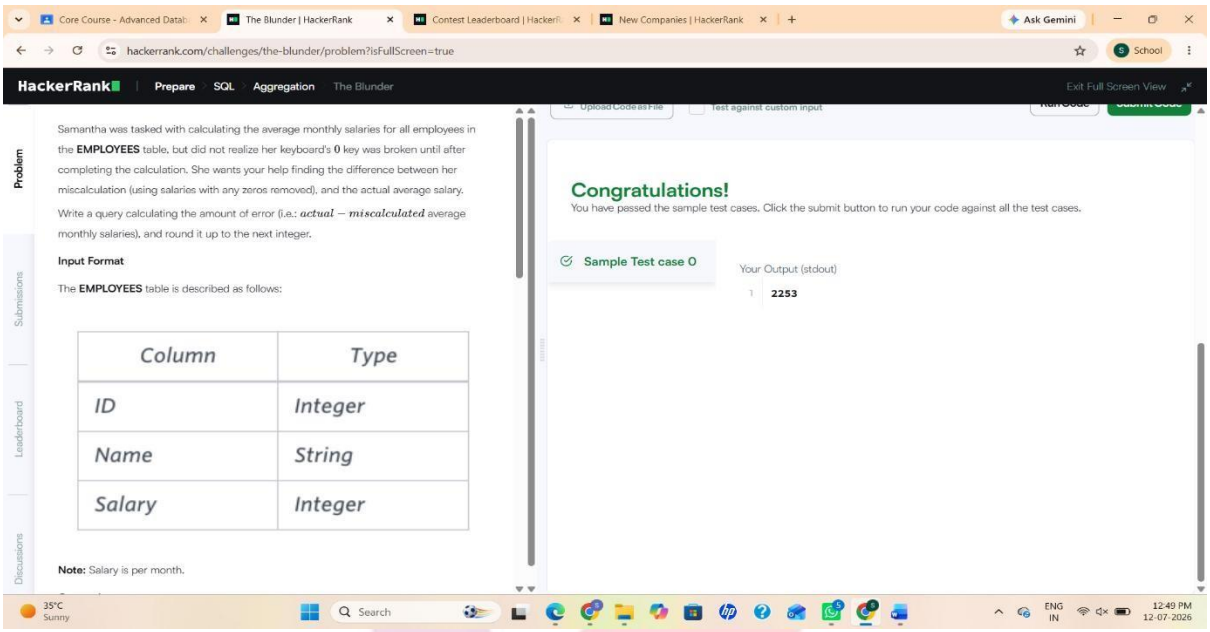


48.THE BLUNDER

PROGRAM

```
SELECT CEIL(
    AVG(Salary) - AVG(REPLACE(Salary, '0', ''))
)
FROM EMPLOYEES;
```

OUTPUT



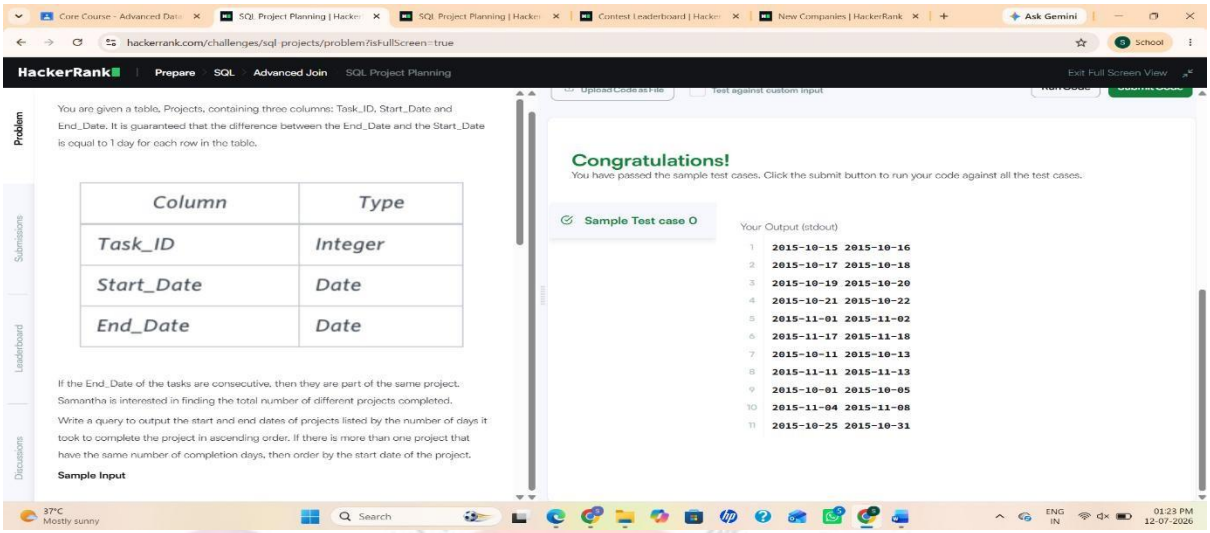
49.SQL PROJECT PLANNING

PROGRAM

```
SELECT
    MIN(Start_Date) AS Start_Date,
    MAX(End_Date) AS End_Date
FROM (
    SELECT
        Start_Date,
        End_Date,
        DATE_SUB(Start_Date, INTERVAL ROW_NUMBER() OVER (ORDER BY
Start_Date) DAY) AS grp
    FROM Projects
) p
```

```
GROUP BY grp
ORDER BY DATEDIFF(MAX(End_Date), MIN(Start_Date)),
        MIN(Start_Date);
```

OUTPUT



50.PLACEMENTS

PROGRAM

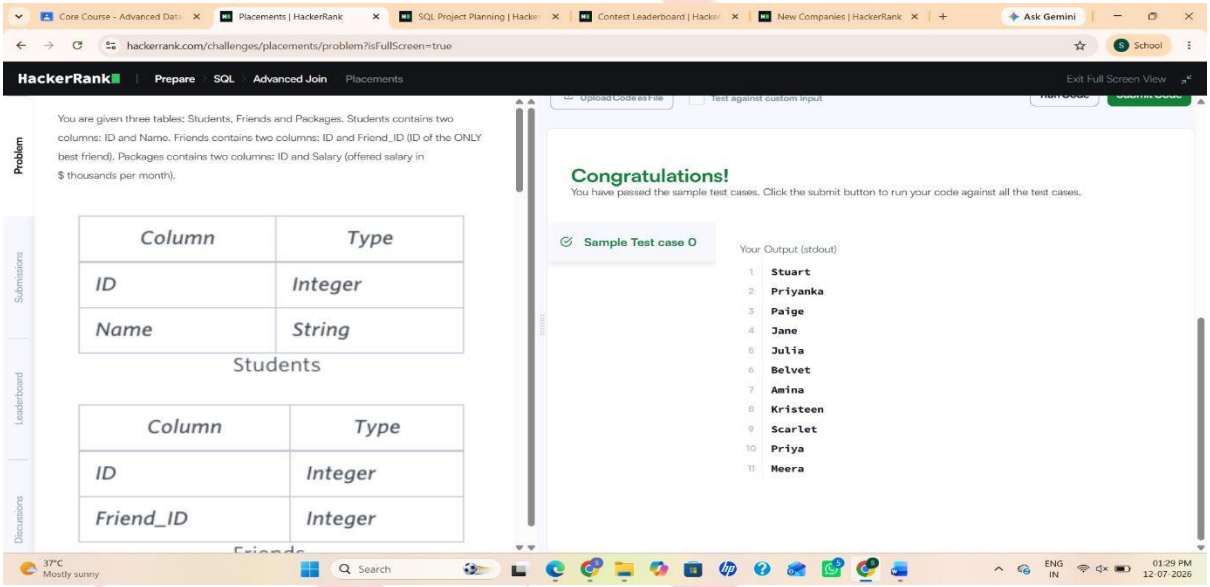
```
SELECT s.Name
FROM Students s
JOIN Friends f
    ON s.ID = f.ID
JOIN Packages p1
    ON s.ID = p1.ID
JOIN Packages p2
```

ON f.Friend_ID = p2.ID

WHERE p2.Salary > p1.Salary

ORDER BY p2.Salary;

OUTPUT



51.SYMMETRIC PAIRS

PROGRAM

SELECT DISTINCT f1.X, f1.Y

FROM Functions f1

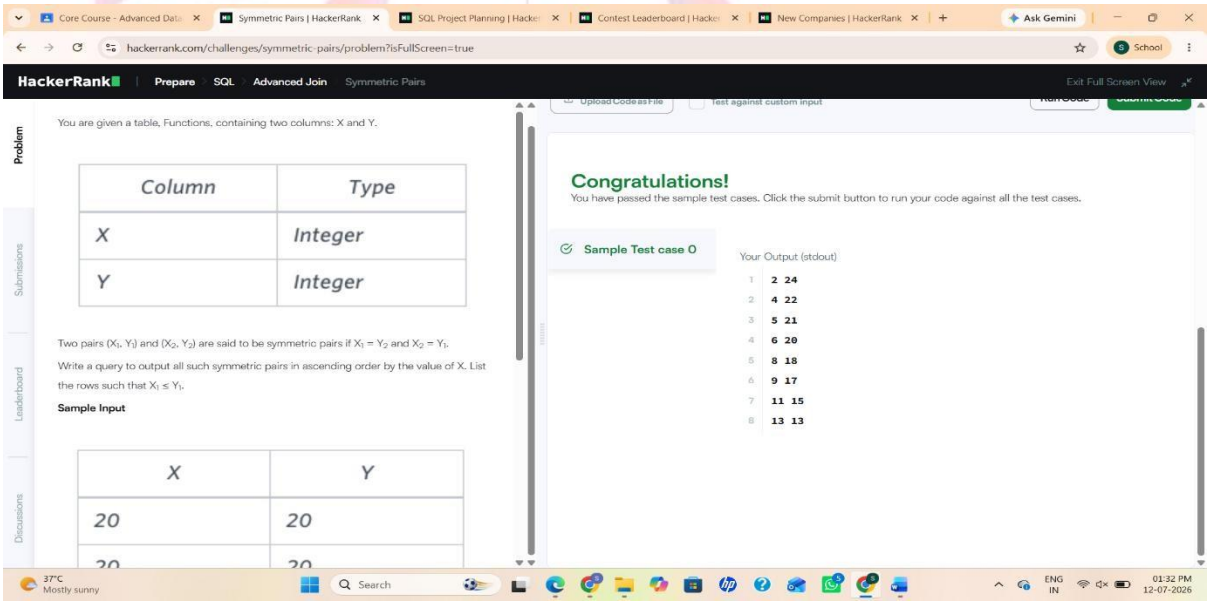
JOIN Functions f2

ON f1.X = f2.Y

AND f1.Y = f2.X

```
WHERE f1.X < f1.Y
  OR (       f1.X =
f1.Y
      AND (
          SELECT COUNT(*)
FROM Functions f3
          WHERE f3.X = f1.X
            AND f3.Y = f1.Y
      ) > 1
  )
ORDER BY f1.X, f1.Y;
```

OUTPUT



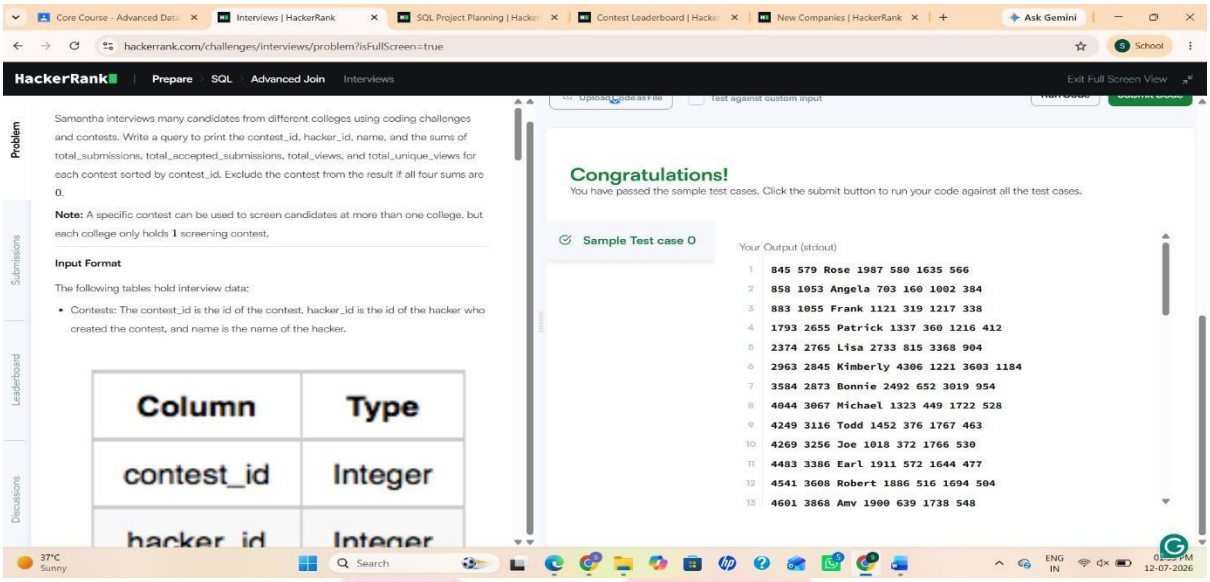
52.INTERVIEWS

PROGRAM

```
SELECT
    c.contest_id,
    c.hacker_id,
    c.name,
    SUM(IFNULL(s.total_submissions, 0)) AS total_submissions,
    SUM(IFNULL(s.total_accepted_submissions, 0)) AS total_accepted_submissions,
    SUM(IFNULL(v.total_views, 0)) AS total_views,
    SUM(IFNULL(v.total_unique_views, 0)) AS total_unique_views
FROM Contests c
JOIN Colleges co
    ON c.contest_id = co.contest_id
JOIN Challenges ch
    ON co.college_id = ch.college_id
LEFT JOIN (     SELECT
        challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) s
ON ch.challenge_id = s.challenge_id
LEFT JOIN (    SELECT
```

```
    challenge_id,
    SUM(total_views) AS total_views,
     SUM(total_unique_views) AS total_unique_views
  FROM View_Stats
  GROUP BY challenge_id
) v
ON ch.challenge_id = v.challenge_id
GROUP BY
  c.contest_id,
  c.hacker_id,
  c.name
HAVING
  total_submissions +
total_accepted_submissions +
total_views +    total_unique_views > 0
ORDER BY
  c.contest_id;
```

OUTPUT



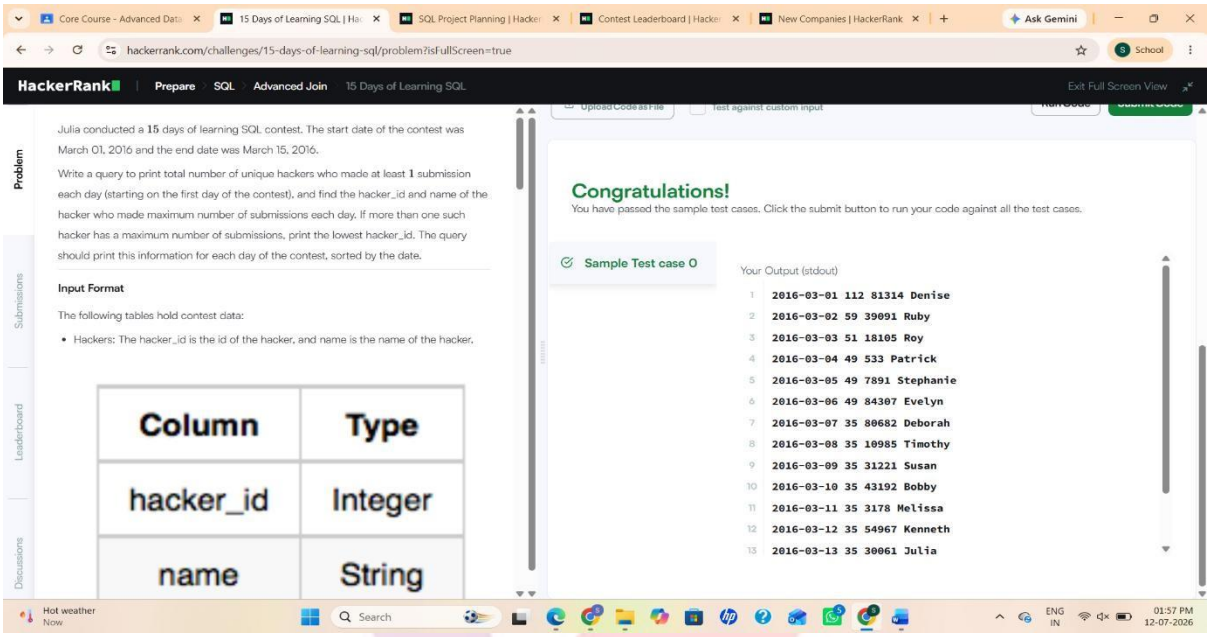
53. 15 DAYS OF LEARNING SQL

PROGRAM

```
SELECT
   d.submission_date,
   (
      SELECT COUNT(*)
      FROM (
         SELECT hacker_id
         FROM Submissions s
         WHERE s.submission_date <= d.submission_date
         GROUP BY hacker_id
         HAVING COUNT(DISTINCT s.submission_date)
               = DATEDIFF(d.submission_date,'2016-03-01') + 1
      ) x
```

```
    ) AS total_hackers,
    h.hacker_id,
    h.name
FROM
(
    SELECT submission_date,
        hacker_id,
        ROW_NUMBER() OVER
        (
            PARTITION BY submission_date
            ORDER BY COUNT(*) DESC, hacker_id
        ) rn
    FROM Submissions
    GROUP BY submission_date,hacker_id
) d
JOIN Hackers h
ON d.hacker_id = h.hacker_id
WHERE rn = 1
ORDER BY submission_date;
```

OUTPUT



54.PRINT PRIME NUMBERS

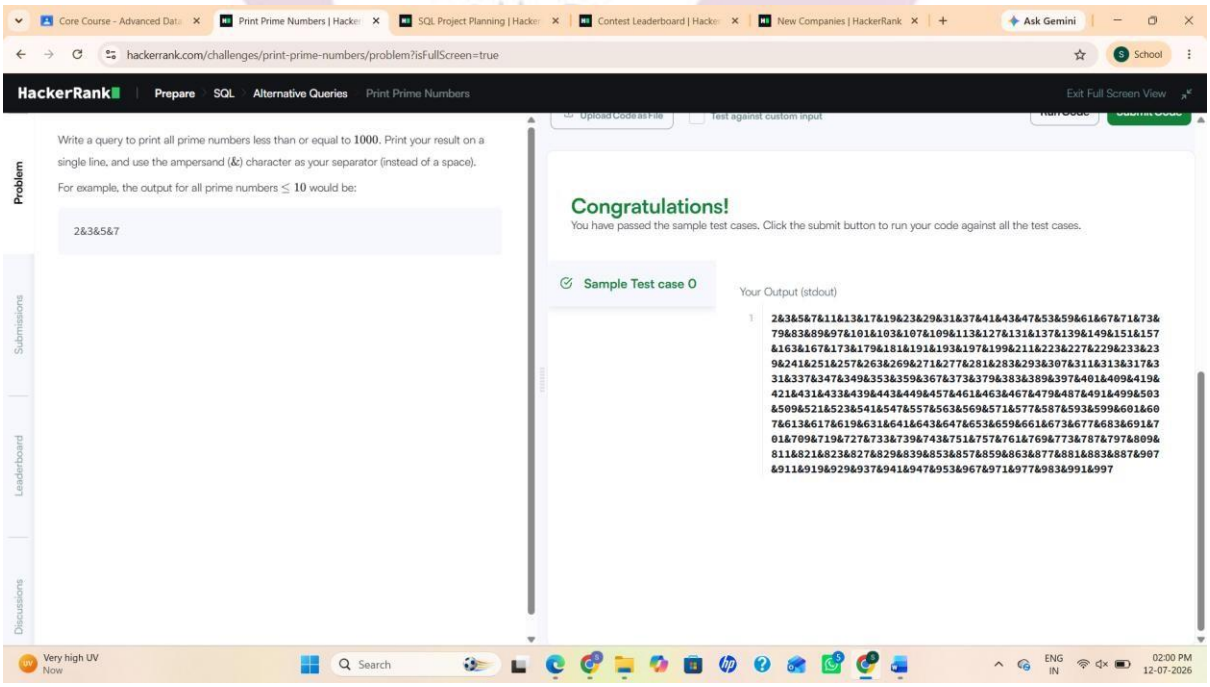
PROGRAM

```
SET @num = 1;


SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM (
   SELECT @num := @num + 1 AS n
   FROM information_schema.tables t1,        information_schema.tables t2
   LIMIT 998
) numbers
WHERE NOT EXISTS (
   SELECT 1
   FROM (
```

```
    SELECT @div := @div + 1 AS d
    FROM information_schema.tables,
        (SELECT @div := 1) init
    LIMIT 998
) divisors
WHERE d < n
  AND d > 1
  AND MOD(n, d) = 0
);
```

OUTPUT



55. OCCUPATIONS

PROGRAM

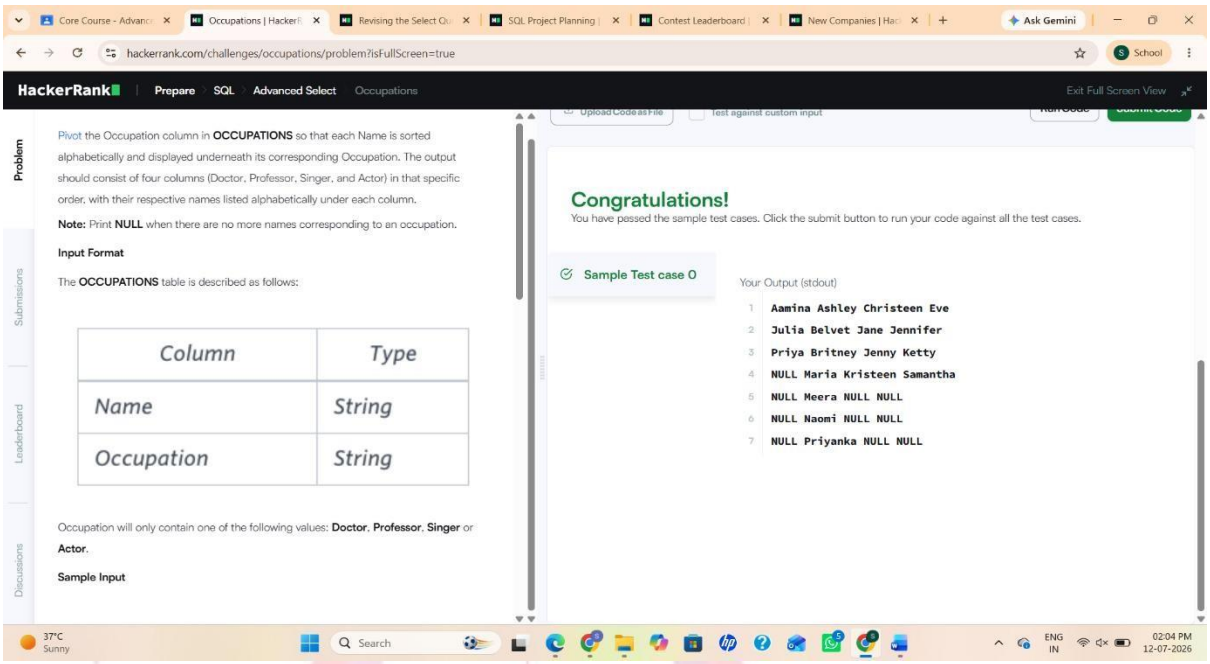
```
SET @d = 0, @p = 0, @s = 0, @a = 0;


SELECT
   MAX(Doctor),
   MAX(Professor),
   MAX(Singer),
   MAX(Actor)
FROM
(
   SELECT
      CASE
         WHEN Occupation = 'Doctor' THEN (@d := @d + 1)
         WHEN Occupation = 'Professor' THEN (@p := @p + 1)
         WHEN Occupation = 'Singer' THEN (@s := @s + 1)
         WHEN Occupation = 'Actor' THEN (@a := @a + 1)
      END AS rn,
      CASE WHEN Occupation = 'Doctor' THEN Name END AS Doctor,
      CASE WHEN Occupation = 'Professor' THEN Name END AS Professor,
      CASE WHEN Occupation = 'Singer' THEN Name END AS Singer,
      CASE WHEN Occupation = 'Actor' THEN Name END AS Actor
   FROM OCCUPATIONS
   ORDER BY Occupation, Name
) t
```

GROUP BY rn

ORDER BY rn;

OUTPUT

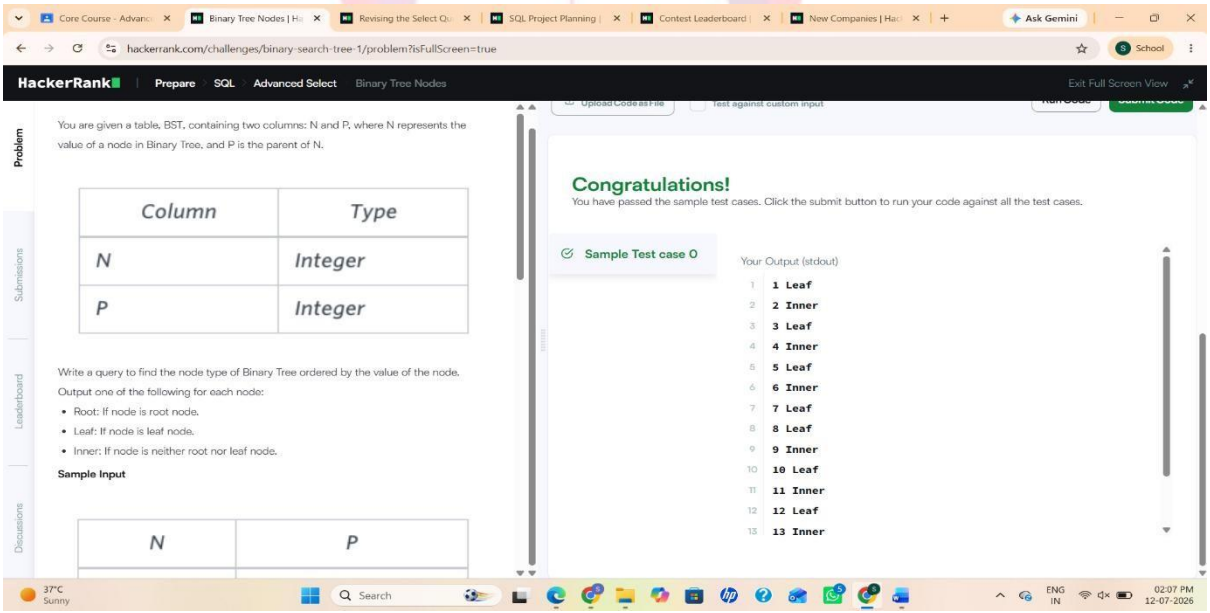


56.BINARY TREE NODES

PROGRAM

```
SELECT
   N,
   CASE
      WHEN P IS NULL THEN 'Root'
      WHEN N NOT IN (SELECT DISTINCT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'
      ELSE 'Inner'
   END AS NodeType
FROM BST
ORDER BY N;
```

OUTPUT





9. KARAOKE SONG SELECTOR

PROGRAM

```
CREATE TABLE songs ( id
    INTEGER PRIMARY KEY,
    title TEXT, artist
```

```
    TEXT, mood TEXT,
    duration INTEGER,
    released
    INTEGER);

INSERT INTO songs (title, artist, mood, duration, released) VALUES
    ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released) VALUES
    ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released) VALUES ("I will
   survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released) VALUES
    ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released) VALUES ("La
    Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992); INSERT
INTO songs (title, artist, mood, duration, released)
    VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969); INSERT
INTO songs (title, artist, mood, duration, released)
    VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);

SELECT title
FROM songs;


SELECT title FROM
songs
WHERE mood = "epic" OR
```

released > 1990;

SELECT title FROM

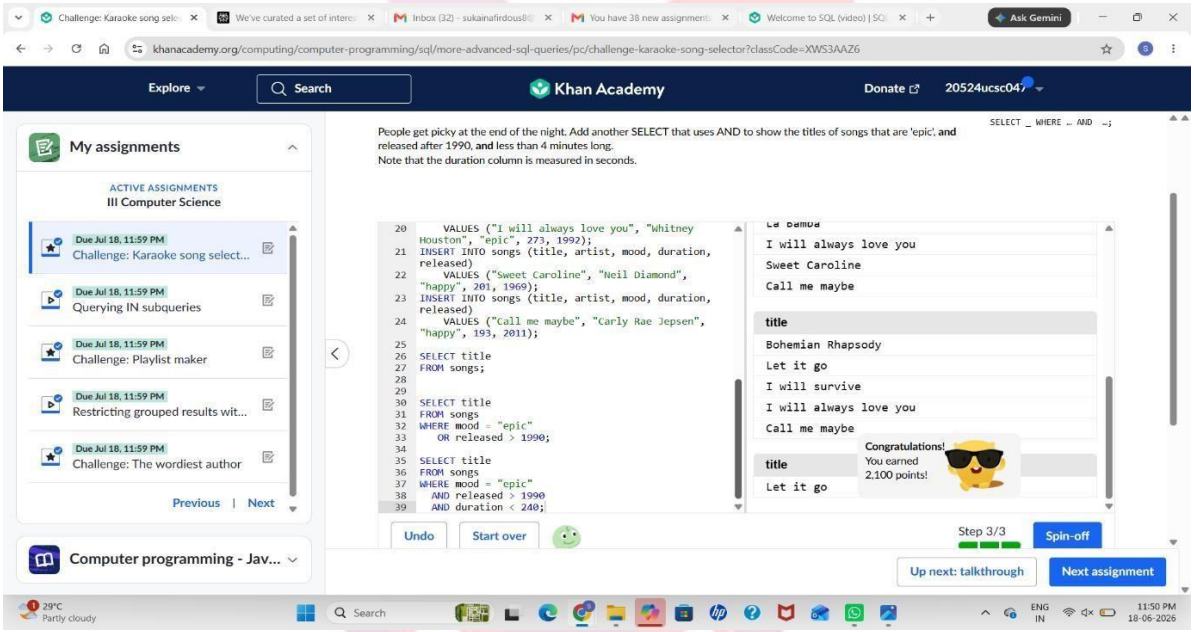
songs

WHERE mood = "epic" AND

released > 1990 AND duration <

240;

OUTPUT



10. QUERYING IN SUBQUERIES

PROGRAM

CREATE TABLE exercise_logs

```
(id INTEGER PRIMARY KEY AUTOINCREMENT, type TEXT,
minutes
INTEGER, calories INTEGER,
heart_rate INTEGER);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);

SELECT * FROM exercise_logs WHERE type = "biking" OR type = "hiking" OR type = "tree climbing" OR
type = "rowing";

/* IN */
SELECT * FROM exercise_logs WHERE type IN ("biking", "hiking", "tree climbing", "rowing");

CREATE TABLE drs_favorites (id
INTEGER PRIMARY KEY, type
TEXT, reason
TEXT);

INSERT INTO drs_favorites(type, reason) VALUES ("biking", "Improves endurance and flexibility.");
```

```
INSERT INTO drs_favorites(type, reason) VALUES ("hiking", "Increases cardiovascular health.");


SELECT type FROM drs_favorites;

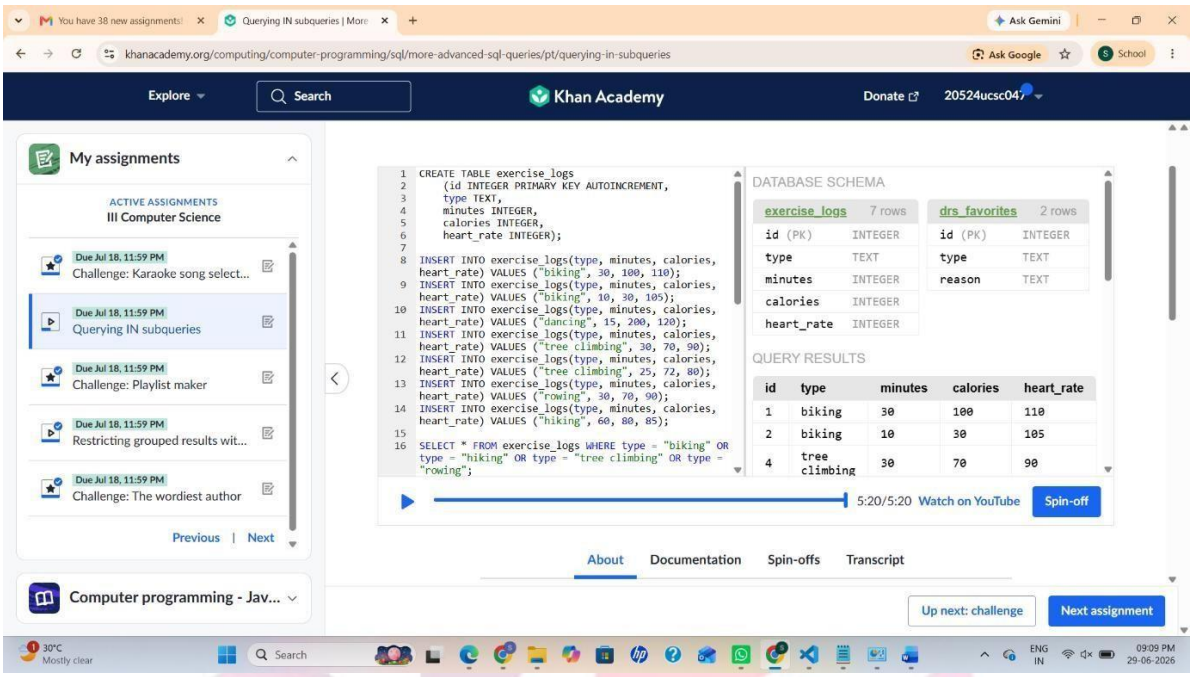

SELECT * FROM exercise_logs WHERE type IN ( SELECT type
    FROM drs_favorites);


SELECT * FROM exercise_logs WHERE type IN (
    SELECT type FROM drs_favorites WHERE reason = "Increases cardiovascular health");


/* LIKE */


SELECT * FROM exercise_logs WHERE type IN (
            SELECT type FROM drs_favorites WHERE reason LIKE "%cardiovascular%");
```

OUTPUT



11. PLAYLIST MAKER

PROGRAM

```
CREATE TABLE artists ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, name TEXT, country
    TEXT, genre TEXT);


      INSERT INTO artists (name, country, genre)
VALUES ("Taylor Swift", "US", "Pop"); INSERT
          INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard rock"); INSERT
```

```
INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre) VALUES
    ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre) VALUES
    ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre) VALUES
    ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre) VALUES
    ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre) VALUES
    ("Rihanna", "US", "Pop");


INSERT INTO artists (name, country, genre) VALUES
("Guns N' Roses", "US", "Hard rock");

INSERT INTO artists (name, country, genre)
VALUES ("Gloria Estefan", "US", "Pop");
 INSERT INTO artists (name, country, genre)
VALUES ("Bob Marley", "Jamaica", "Reggae");


CREATE TABLE songs ( id INTEGER PRIMARY KEY
    AUTOINCREMENT,
    artist TEXT, title
```

```
    TEXT);


INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title) VALUES
    ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on"); INSERT
INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come"); INSERT
INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two"); INSERT
INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");


INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title) VALUES
    ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody"); INSERT
INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");
```

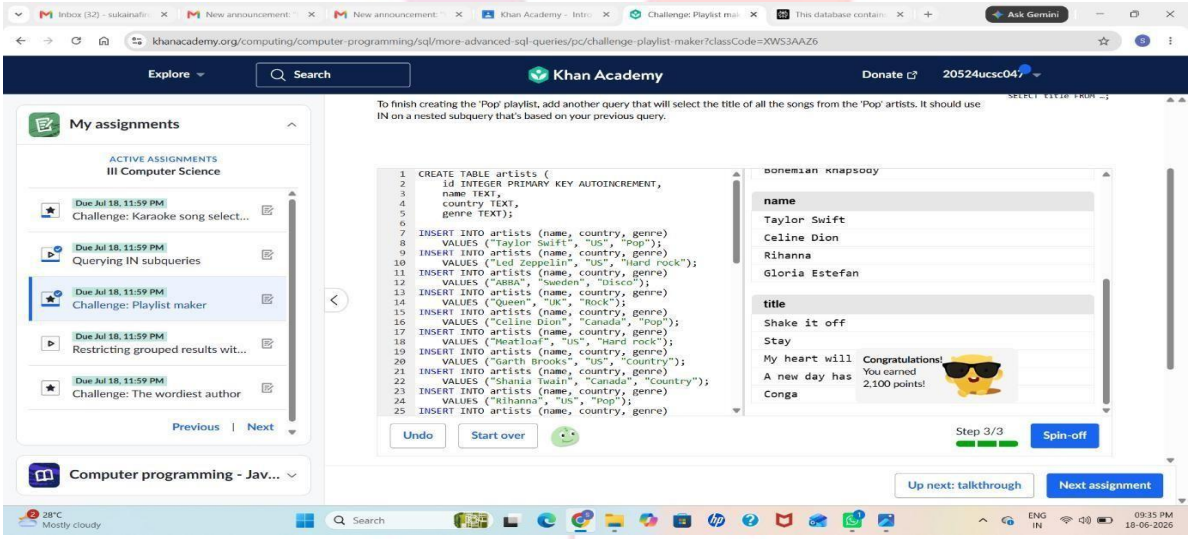
SELECT title FROM songs WHERE

artist = "Queen";

SELECT name FROM artists WHERE genre = 'Pop';

SELECT title FROM songs WHERE artist IN (SELECT name FROM artists WHERE genre = 'Pop');

OUTPUT



12. RESTRICTING GROUPING RESULTS WITH HAVING

PROGRAM

CREATE TABLE exercise_logs

```
    (id INTEGER PRIMARY KEY AUTOINCREMENT, type TEXT,
    minutes
    INTEGER, calories INTEGER,
    heart_rate INTEGER);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 115, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 45, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 165, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70,
90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);

SELECT * FROM exercise_logs;

SELECT type, SUM(calories) AS total_calories FROM exercise_logs GROUP BY type;

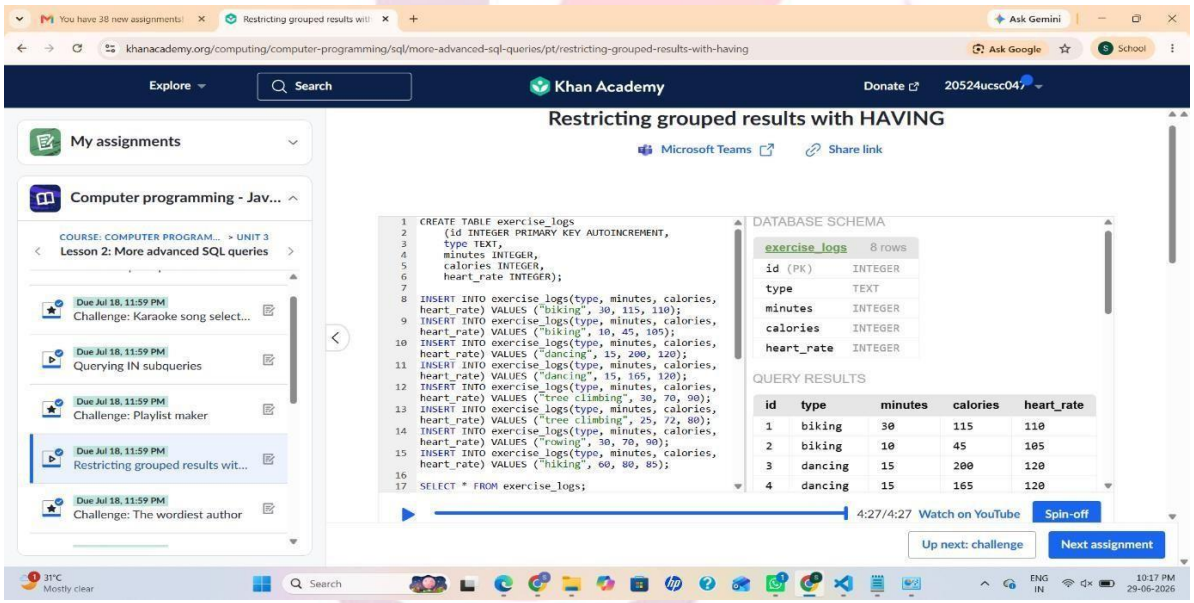
SELECT type, SUM(calories) AS total_calories FROM exercise_logs

    GROUP BY type
    HAVING total_calories > 150
    ;
```

```
SELECT type, AVG(calories) AS avg_calories FROM exercise_logs
    GROUP BY type
    HAVING avg_calories > 70
    ;
```

```
SELECT type FROM exercise_logs GROUP BY type HAVING COUNT(*) >= 2;
```

OUTPUT



13. THE WORDIEST AUTHOR

PROGRAM

```
CREATE TABLE books ( id INTEGER PRIMARY KEY
    AUTOINCREMENT,
    author TEXT, title
    TEXT,
    words INTEGER);


INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944); INSERT INTO
books (author, title, words)
        VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637); INSERT
INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045); INSERT
INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);


INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Twilight", 118501); INSERT
INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "New Moon", 132807);
```

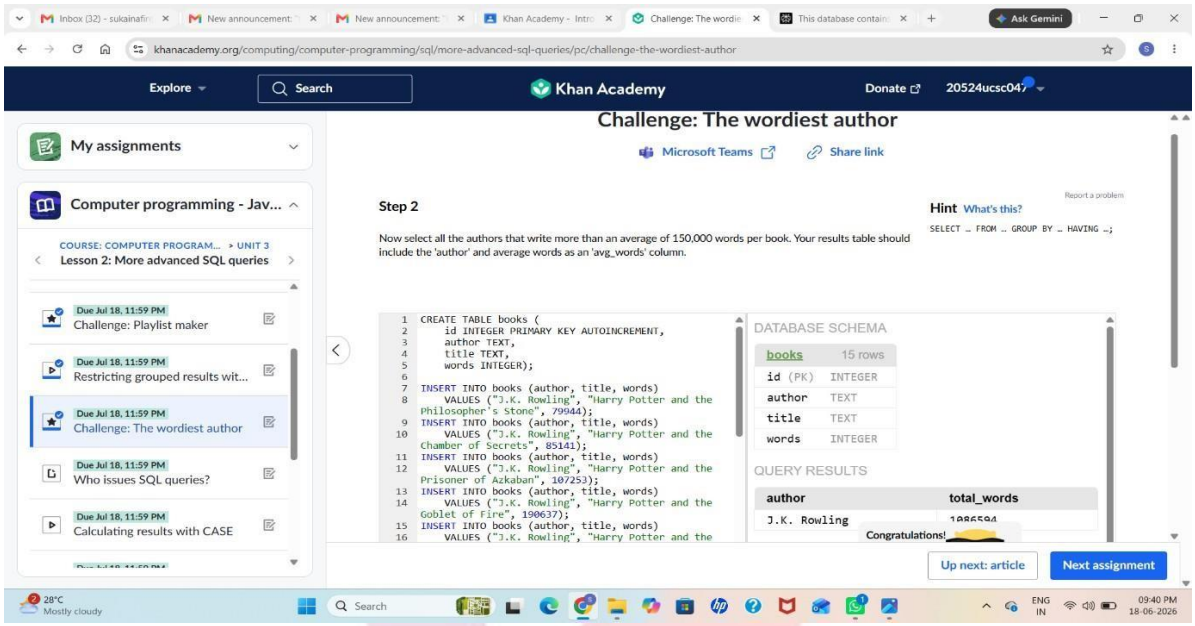
```
    INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930); INSERT
INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022); INSERT
INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227); INSERT
INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);

SELECT author, SUM(words) AS total_words FROM books
GROUP BY author
HAVING total_words > 1000000;

SELECT author, AVG(words) AS avg_words FROM books
GROUP BY author
HAVING avg_words > 150000;
```

OUTPUT



14. CALCULATING RESULTS WITH CASE

PROGRAM

```
CREATE TABLE exercise_logs
    (id INTEGER PRIMARY KEY AUTOINCREMENT, type TEXT,
    minutes
    INTEGER, calories INTEGER,
    heart_rate INTEGER);
```

```
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 165, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70, 90);
```

```
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72, 80);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);


SELECT * FROM exercise_logs;


SELECT COUNT(*) FROM exercise_logs WHERE heart_rate > 220 - 30;

/* 50-90% of max*/


SELECT COUNT(*) FROM exercise_logs WHERE heart_rate

>= ROUND(0.50 * (220-30))

    AND heart_rate <= ROUND(0.90 * (220-30));


/* CASE */

SELECT type, heart_rate, CASE

        WHEN heart_rate > 220-30 THEN "above max"

        WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target" WHEN

        heart_rate > ROUND(0.50 * (220-30)) THEN "within target" ELSE "below

        target"

    END as "hr_zone" FROM

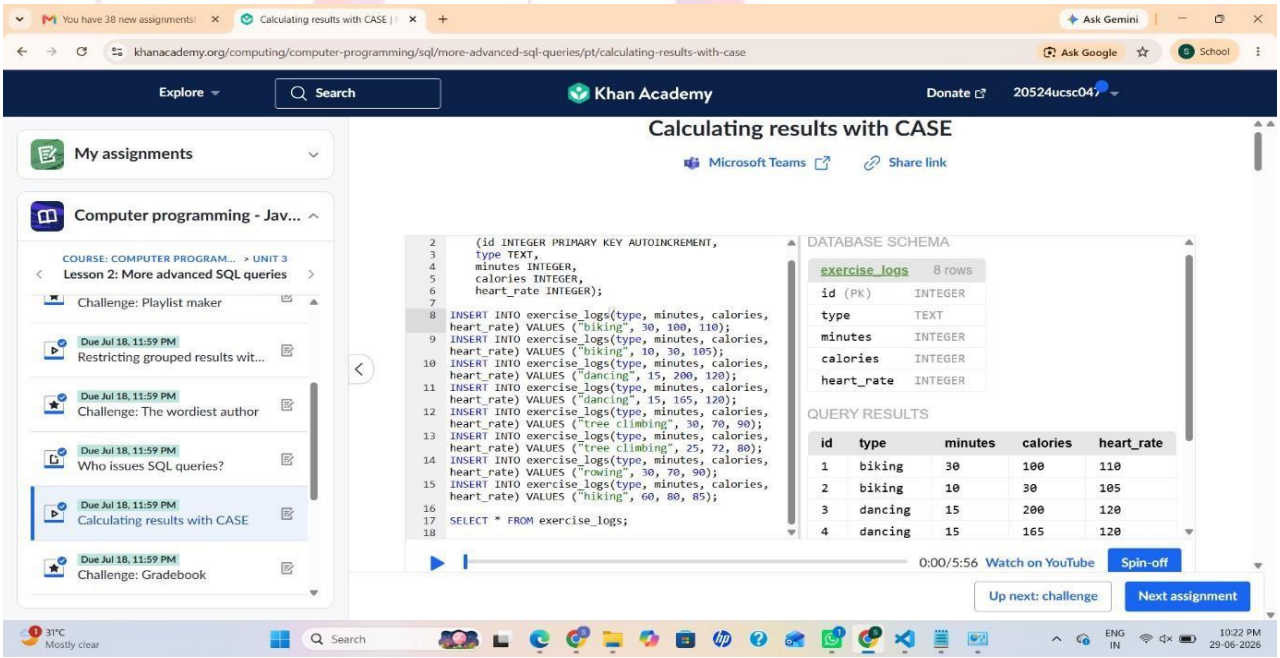
exercise_logs;


SELECT COUNT(*), CASE

        WHEN heart_rate > 220-30 THEN "above max"
```

```
    WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target" WHEN
    heart_rate > ROUND(0.50 * (220-30)) THEN "within target" ELSE "below
    target"
  END as "hr_zone" FROM
exercise_logs GROUP BY
hr_zone;
```

OUTPUT



15.GRADEBOOK

PROGRAM

```
 CREATE TABLE student_grades ( id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT,
 number_grade INTEGER,
fraction_completed REAL);


INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES

    ("Winston", 90, 0.805);

INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES

    ("Winnefer", 95, 0.901);

INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES

    ("Winsteen", 85, 0.906);

INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES

    ("Wincifer", 66, 0.7054);

INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES ("Winster", 76,

    0.5013);

INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES ("Winstonia", 82,

    0.9045);


SELECT name, number_grade, ROUND(100 * fraction_completed) AS percent_completed FROM
student_grades;




SELECT COUNT(*) AS number_of_students, CASE

    WHEN number_grade > 90 THEN "A" WHEN

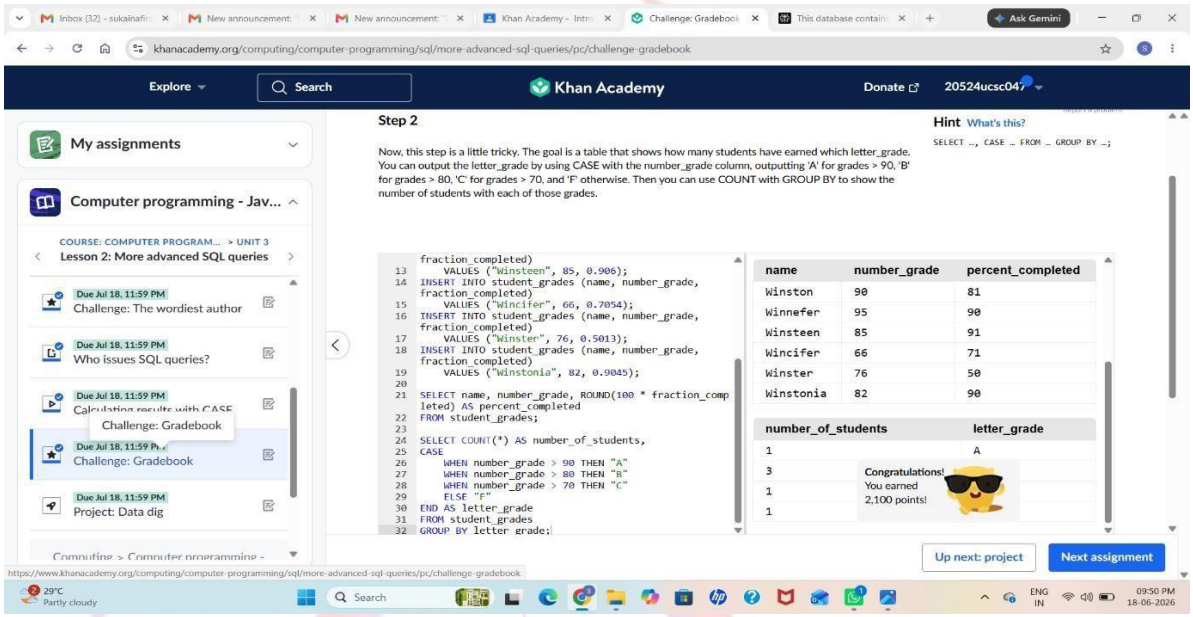
number_grade > 80 THEN "B" WHEN

number_grade > 70 THEN "C" ELSE "F" END AS
```

letter_grade FROM student_grades GROUP BY

letter_grade;

OUTPUT



16. PROJECT: DATA DIG

PROGRAM

```
CREATE TABLE astronauts ( Name
  TEXT PRIMARY KEY, Year
  INTEGER,
  GroupNum INTEGER, Status
  TEXT, Birth_Date TEXT,
  Birth_Place TEXT, Gender
  TEXT, Alma_Mater TEXT,
  Undergraduate_Major TEXT,
  Graduate_Major TEXT, Military_Rank
  TEXT, Military_Branch TEXT,
  Space_Flights INTEGER,
  Space_Flight_hr INTEGER,
  Space_Walks INTEGER,
  Space_Walks_hr REAL, Missions
  TEXT,
  Death_Date TEXT,
  Death_Mission TEXT
);

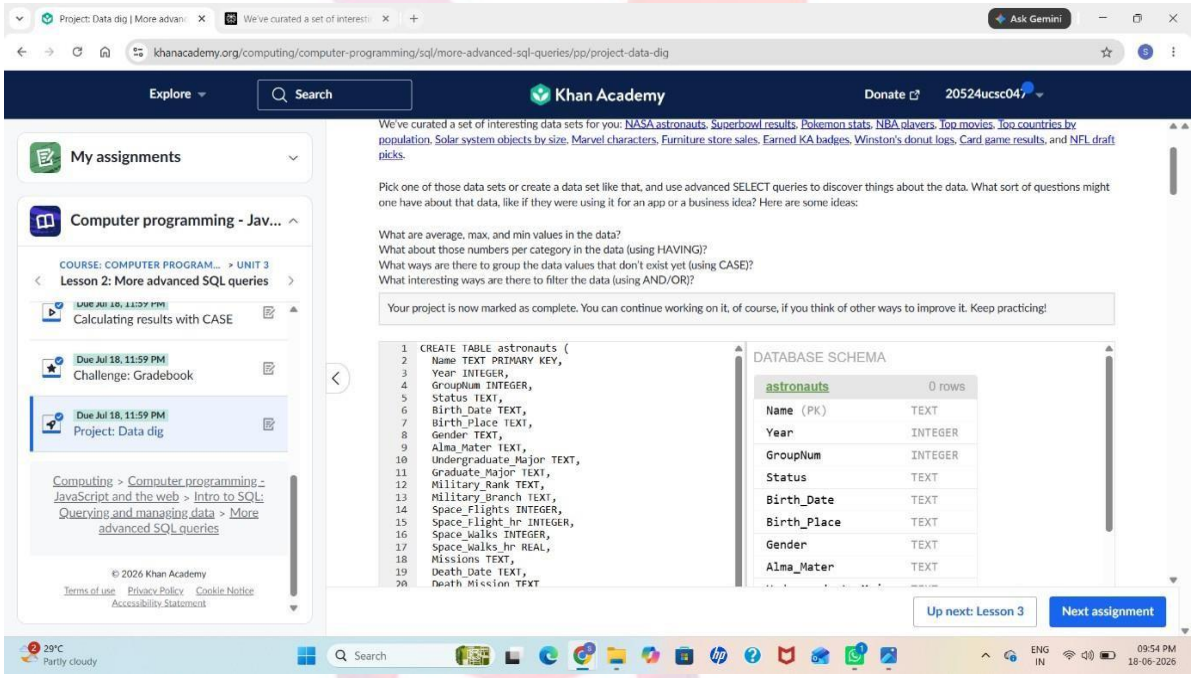
SELECT
  Gender,
  AVG(Space_Flights) AS avg_flights, MAX(Space_Flights)
```

AS max_flights,

MIN(Space_Flights) AS min_flights FROM astronauts

GROUP BY Gender;

OUTPUT



17. JOINING RELATED TABLES

PROGRAM

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
    first_name TEXT, last_name TEXT, email TEXT, phone
    TEXT, birthdate TEXT);


INSERT INTO students (first_name, last_name, email, phone, birthdate) VALUES
    ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444", "2002-07-04");


CREATE TABLE student_grades (id INTEGER PRIMARY KEY, student_id
    INTEGER, test TEXT,
    grade INTEGER);


INSERT INTO student_grades (student_id, test, grade)
    VALUES (1, "Nutrition", 95);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (2, "Nutrition", 92);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (1, "Chemistry", 85);
```

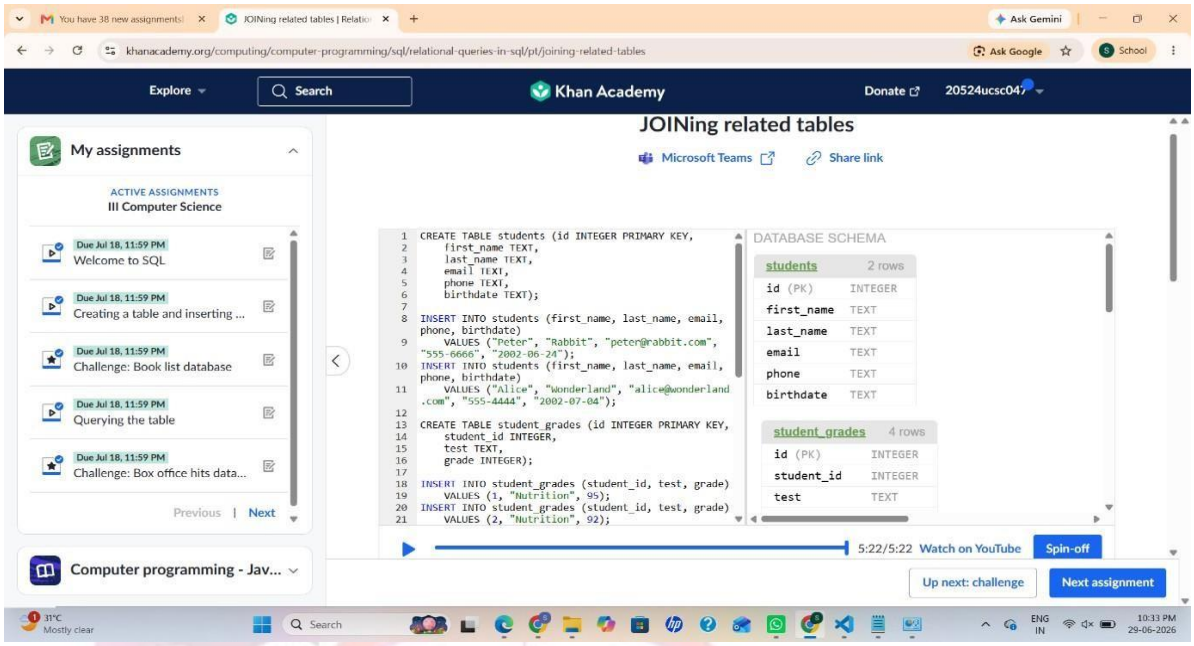
```
INSERT INTO student_grades (student_id, test, grade)
    VALUES (2, "Chemistry", 95);
SELECT * FROM student_grades;


/* cross join */
SELECT * FROM student_grades, students;


/* implicit inner join */
SELECT * FROM student_grades, students WHERE student_grades.student_id =
    students.id;


/* explicit inner join - JOIN */
SELECT students.first_name, students.last_name, students.email, student_grades.test,
student_grades.grade FROM students
    JOIN student_grades
    ON students.id = student_grades.student_id WHERE
    grade > 90;
```

OUTPUT



AUTONOMOUS

18.BOBBY'S HOBBIES

PROGRAM

```
CREATE TABLE persons ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, name TEXT, age INTEGER
);


INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);

CREATE TABLE hobbies ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, person_id INTEGER, name TEXT
);


INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");  INSERT INTO
hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");

 INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
 INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
 INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
```

INSERT INTO hobbies (person_id, name) VALUES (4, "meowing"); INSERT INTO persons (name, age) VALUES ("Aisha Khan", 20);

INSERT INTO hobbies (person_id, name) VALUES (6, "coding");

SELECT persons.name, hobbies.name FROM

persons

JOIN hobbies

ON persons.id = hobbies.person_id;

SELECT persons.name, hobbies.name FROM

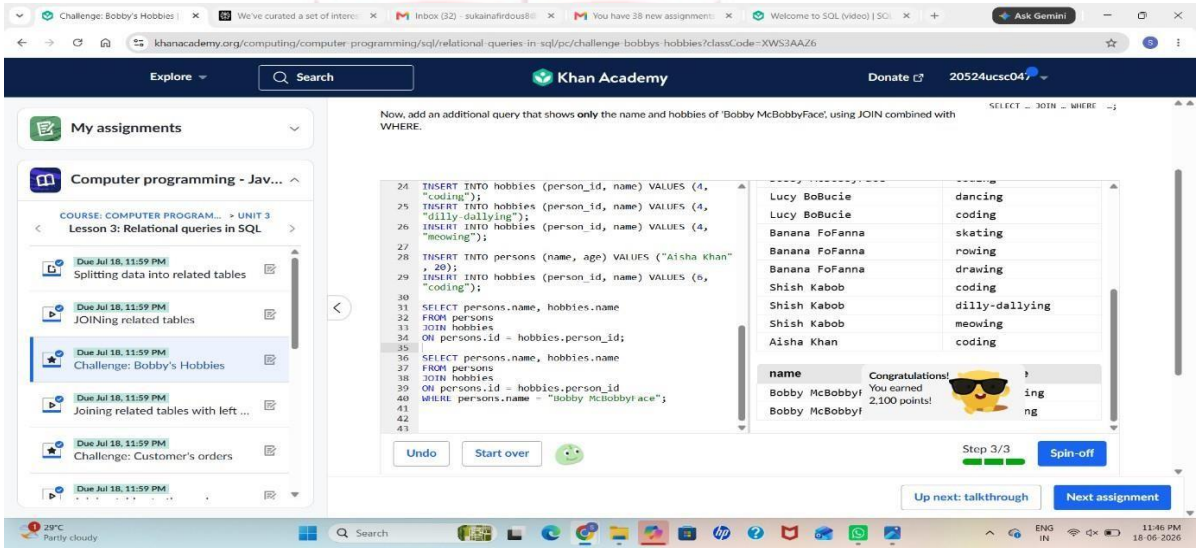
persons

JOIN hobbies

ON persons.id = hobbies.person_id

WHERE persons.name = "Bobby McBobbyFace";

OUTPUT



19. JOINING RELATED TABLES WITH LEFT OUTER JOINS

PROGRAM

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
    first_name TEXT, last_name TEXT, email TEXT, phone
    TEXT, birthdate TEXT);


INSERT INTO students (first_name, last_name, email, phone, birthdate) VALUES
    ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444", "2002-07-04");


CREATE TABLE student_grades (id INTEGER PRIMARY KEY, student_id
    INTEGER, test TEXT,
    grade INTEGER);


INSERT INTO student_grades (student_id, test, grade)
    VALUES (1, "Nutrition", 95);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (2, "Nutrition", 92);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (1, "Chemistry", 85);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (2, "Chemistry", 95);
```

```
CREATE TABLE student_projects (     id INTEGER

PRIMARY KEY, student_id INTEGER, title TEXT);


INSERT INTO student_projects (student_id, title) VALUES

    (1,"Carrotapault");


/* outer join */

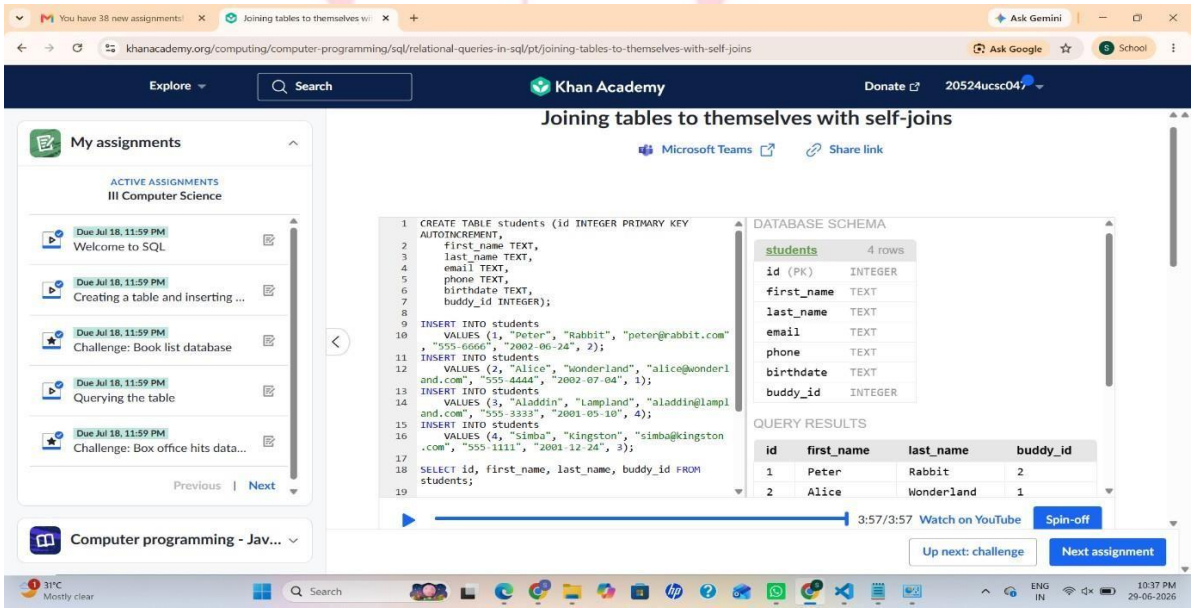
SELECT students.first_name, students.last_name, student_projects.title

FROM students

    LEFT OUTER JOIN student_projects

    ON students.id = student_projects.student_id;
```

OUTPUT



20. CUSTOMER'S ORDERS

PROGRAM

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT,
    email
    TEXT);

INSERT INTO customers (name, email) VALUES ("Doctor Who", "doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome", "captain@awesome.com");

CREATE TABLE orders ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, customer_id INTEGER, item
    TEXT,
    price REAL);

      INSERT INTO orders (customer_id, item, price)
 VALUES (1, "Sonic Screwdriver", 1000.00); INSERT
            INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00); INSERT
INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);
```

SELECT customers.name, customers.email, orders.item, orders.price

FROM customers

LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;

SELECT customers.name, customers.email, SUM(orders.price) AS total_spent FROM customers

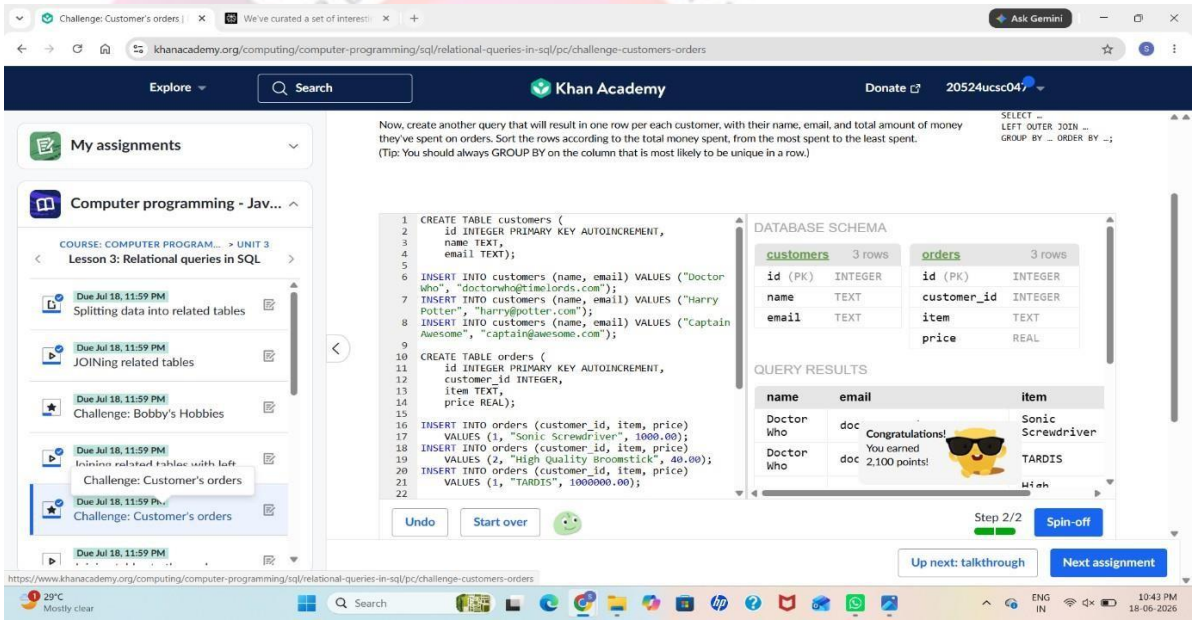
LEFT OUTER JOIN orders

ON customers.id = orders.customer_id

GROUP BY customers.id, customers.name, customers.email ORDER BY

total_spent DESC;

OUTPUT



21. SEQUELS IN SQL

PROGRAM

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT,
    released INTEGER, sequel_id INTEGER);


INSERT INTO movies
    VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2); INSERT
INTO movies
    VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
    VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
    VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
    VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6); INSERT
INTO movies
    VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7); INSERT
INTO movies
    VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8); INSERT INTO movies
    VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);

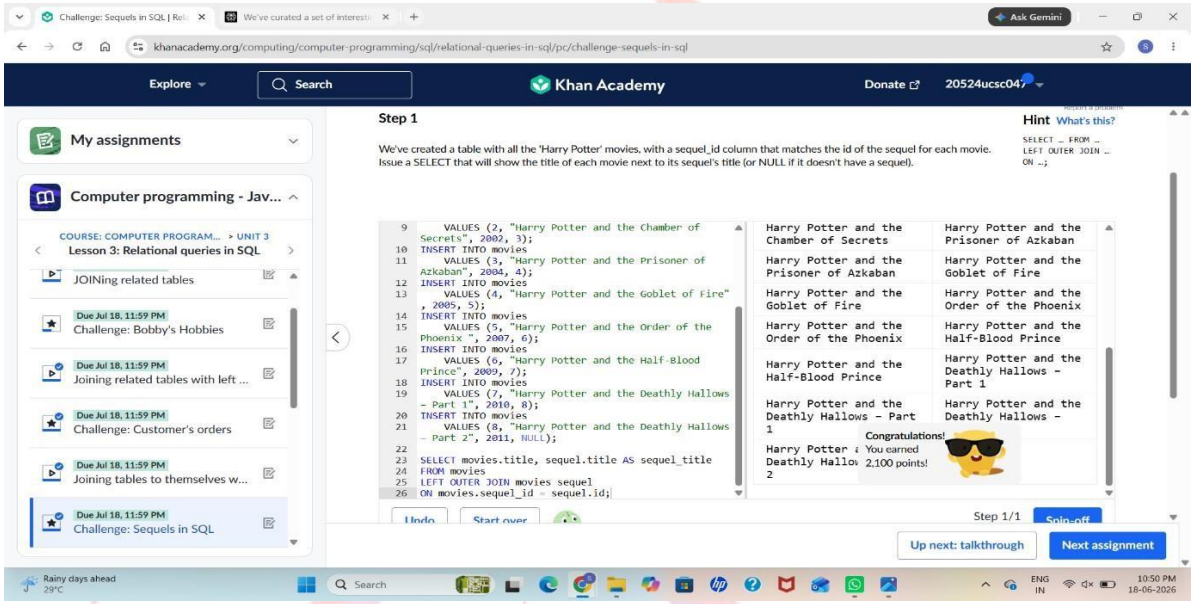


SELECT movies.title, sequel.title AS sequel_title
FROM movies
```

LEFT OUTER JOIN movies sequel ON movies.sequel_id =

sequel.id;

OUTPUT



22. COMBINING MULTIPLE JOINS

PROGRAM

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
    first_name TEXT, last_name TEXT, email TEXT, phone
    TEXT,
    birthdate TEXT);

INSERT INTO students (first_name, last_name, email, phone, birthdate) VALUES
    ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444", "2002-07-04"); INSERT
INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Aladdin", "Lampland", "aladdin@lampland.com", "555-3333", "2001-05-10"); INSERT
INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Simba", "Kingston", "simba@kingston.com", "555-1111", "2001-12-24");

CREATE TABLE student_projects (id INTEGER PRIMARY KEY, student_id
    INTEGER,
    title TEXT);

INSERT INTO student_projects (student_id, title) VALUES (1,
    "Carrotapault");
```

```
INSERT INTO student_projects (student_id, title) VALUES (2, "Mad
    Hattery");

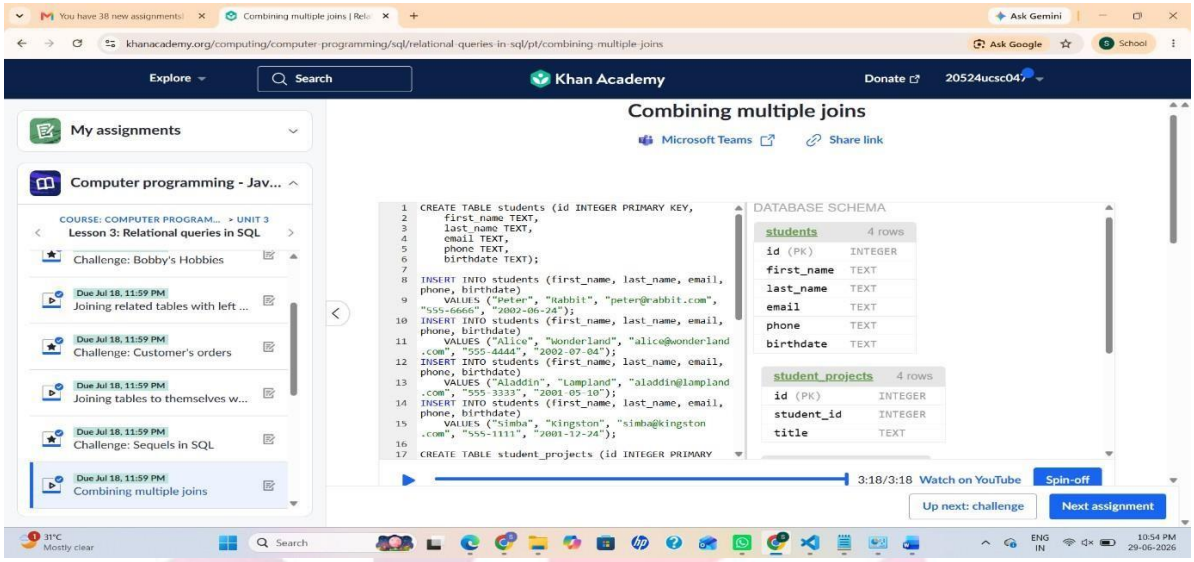
INSERT INTO student_projects (student_id, title) VALUES (3,
"Carpet Physics");
INSERT INTO student_projects (student_id, title) VALUES (4, "Hyena
    Habitats");

CREATE TABLE project_pairs (id INTEGER PRIMARY KEY, project1_id
    INTEGER,
    project2_id INTEGER);

INSERT INTO project_pairs (project1_id, project2_id) VALUES(1, 2);
INSERT INTO project_pairs (project1_id, project2_id) VALUES(3, 4);

SELECT a.title, b.title FROM project_pairs JOIN student_projects
    a
    ON project_pairs.project1_id = a.id JOIN
    student_projects b
    ON project_pairs.project2_id = b.id;
```

OUTPUT



23.FRIENDBOOK

PROGRAM

```
CREATE TABLE persons ( id INTEGER PRIMARY KEY
    AUTOINCREMENT,
    fullname TEXT, age
    INTEGER
);


INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", 12);
```

```
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", 25); INSERT INTO
persons (fullname, age) VALUES ("Banana FoFanna", 14); INSERT INTO persons
(fullname, age) VALUES ("Shish Kabob", 20); INSERT INTO persons
(fullname, age) VALUES ("Fluffy Sparkles", 8);

CREATE TABLE hobbies ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, person_id INTEGER, name TEXT
);

INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");

INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying"); INSERT INTO hobbies
(person_id, name) VALUES (4, "meowing");

CREATE TABLE friends ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, person1_id INTEGER,
    person2_id INTEGER
);
```

INSERT INTO friends (person1_id, person2_id) VALUES (1, 4); INSERT INTO

friends (person1_id, person2_id) VALUES (2, 3);

SELECT persons.fullname, hobbies.name FROM persons

JOIN hobbies

ON persons.id = hobbies.person_id;

SELECT a.fullname, b.fullname FROM friends

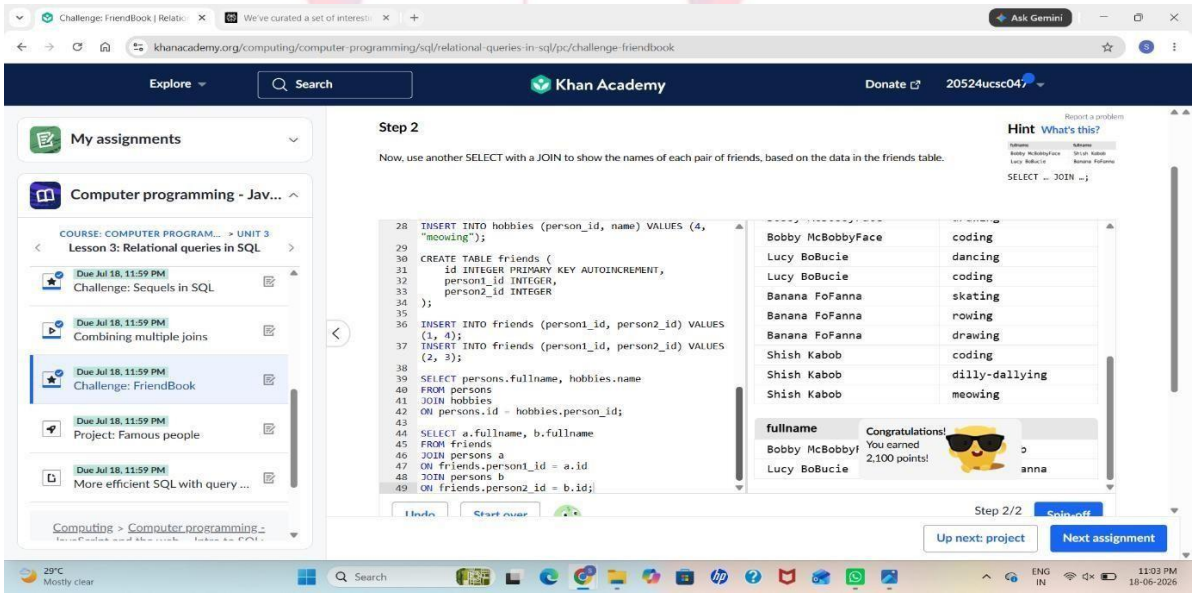
JOIN persons a

ON friends.person1_id = a.id JOIN

persons b

ON friends.person2_id =
b.id;

OUTPUT



24. PROJECT: FAMOUS PEOPLE

PROGRAM

```
CREATE TABLE people ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, name TEXT, age INTEGER,
    profession TEXT
);


INSERT INTO people (name, age, profession) VALUES ("A. P. J. Abdul Kalam", 83, "Scientist");
INSERT INTO people (name, age, profession) VALUES ("M. S. Subbulakshmi", 88, "Singer");
INSERT INTO people (name, age, profession) VALUES ("C. V. Raman", 82, "Scientist");
INSERT INTO people (name, age, profession) VALUES ("Lata Mangeshkar", 92, "Singer");
INSERT INTO people (name, age, profession) VALUES ("Dr. B. R. Ambedkar", 65, "Lawyer");

CREATE TABLE works ( id INTEGER PRIMARY KEY
    AUTOINCREMENT, person_id INTEGER, work_title
    TEXT, work_type
    TEXT
);


INSERT INTO works (person_id, work_title, work_type) VALUES (1, "Missile Man of India", "Book");
INSERT INTO works (person_id, work_title, work_type) VALUES (2, "Vande Mataram", "Song");



INSERT INTO works (person_id, work_title, work_type) VALUES (3, "Raman Effect Research",
"Discovery");
```

INSERT INTO works (person_id, work_title, work_type) VALUES (4, "Aye Mere Watan Ke Logon", "Song");

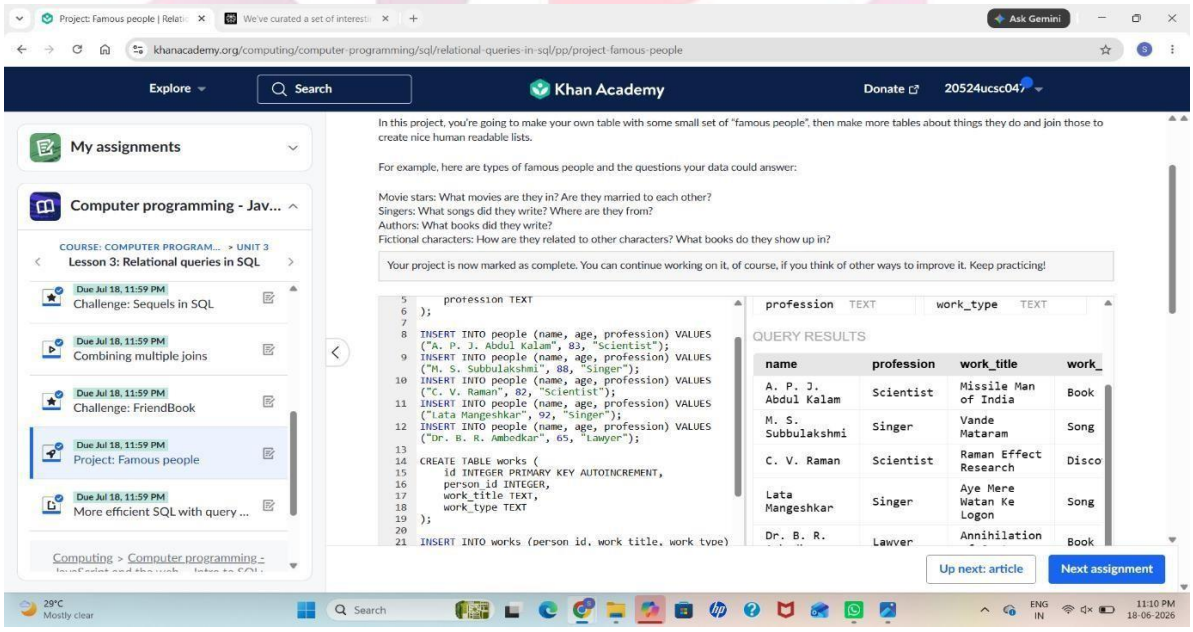
INSERT INTO works (person_id, work_title, work_type) VALUES (5, "Annihilation of Caste", "Book");

SELECT people.name, people.profession, works.work_title, works.work_type FROM people

JOIN works

ON people.id = works.person_id;

OUTPUT



25. CHANGING ROWS WITH UPDATE AND DELETE

PROGRAM

```
CREATE TABLE users ( id
    INTEGER PRIMARY KEY,
    name TEXT);


CREATE TABLE diary_logs ( id
    INTEGER PRIMARY KEY,
    user_id INTEGER, date
    TEXT, content TEXT
    );

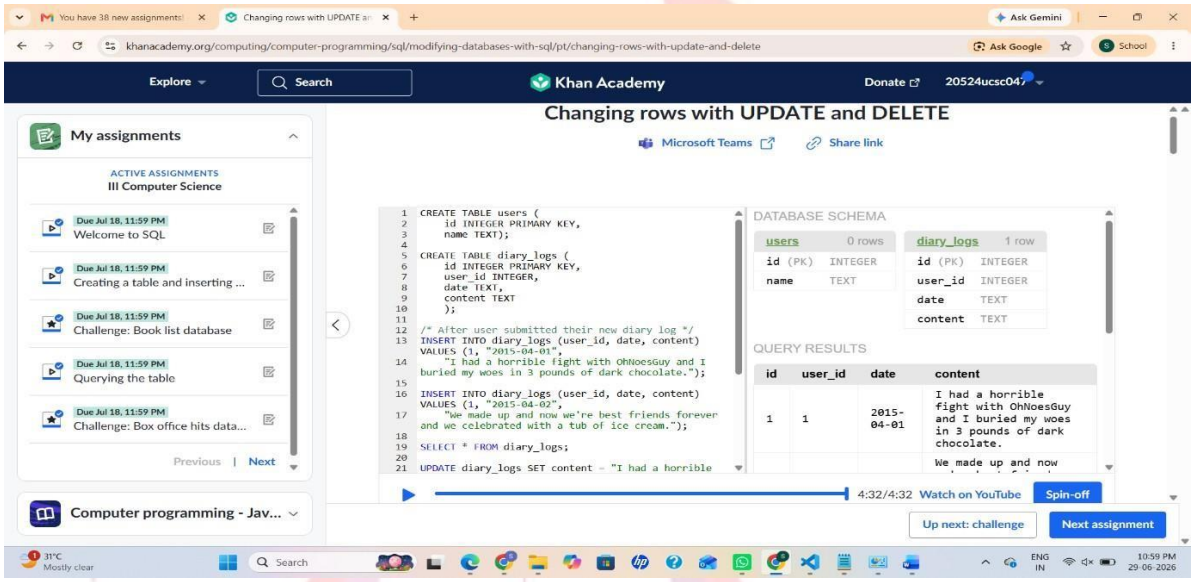
/* After user submitted their new diary log */
INSERT INTO diary_logs (user_id, date, content) VALUES (1, "2015-04-01",
    "I had a horrible fight with OhNoesGuy and I buried my woes in 3 pounds of dark chocolate.");

INSERT INTO diary_logs (user_id, date, content) VALUES (1, "2015-04-02",
    "We made up and now we're best friends forever and we celebrated with a tub of ice cream.");

SELECT * FROM diary_logs;
UPDATE diary_logs SET content = "I had a horrible fight with OhNoesGuy" WHERE id = 1;
SELECT * FROM diary_logs;
DELETE FROM diary_logs WHERE id = 1;
```

SELECT * FROM diary_logs;

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS